

ICPC Notebook – UIT.BTC

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM

Vu Gia Bao, Nguyen Khanh Tai, Huynh Trung Cuong | Ngày 7 tháng 10 năm 2026

Mục lục

C++: template và typename

1 Cấu trúc dữ liệu

1.1	Iterative Segment Tree	1
1.2	Lazy Segment Tree (lưu ý cài đặt)	1
1.3	Sparse Segment Tree	1
1.4	Cartesian Tree	1
1.5	Persistent Segment Tree	2
1.6	Parallel Binary Search	2
1.7	Wavelet Tree	2
1.8	Segment Tree Beat	2
1.9	Kd Tree	2
1.10	BIT2D	3
1.11	Treap	3
1.12	Splay Tree / Link - cut Tree	3
1.13	Farach - Colton - Bender	3
1.14	Trie	4
1.15	Lichao Tree	4
1.16	Line Container	4
1.17	Arpa Trick	4
1.18	Chia căn (Square Root Decomposition)	5
1.19	Hash Map (SplitMix64)	5
1.20	Erasable Priority Queue	5
1.21	Cấu trúc mở rộng (Binary Doubling)	5
1.22	Interval Set	5

2 Đồ thị

2.1	Minimum Cost Flow	5
2.2	MinCut - MaxFlow	6
2.3	Bipartite Matching	6
2.4	Hungarian Matching	7
2.5	General Matching	7
2.6	Biconnected Component	7
2.7	Functional Cycle	7
2.8	Strongly Connected Components	7
2.9	Xóa offline (DSU Rollback)	8
2.10	2-SAT	8
2.11	Floyd	8
2.12	Eulerian Path	8
2.13	Định lý Vizing	8
2.14	Kiểm tra hai phía online	9
2.15	Gomory-Hu Tree	9
2.16	Critical (Dominator Tree)	9
2.17	Prufer	9
2.18	Steiner Tree	9

3 Cây

2.19	Stable Marriage	9
3.1	Tree Isomorphism	9
3.2	Centroid Decomposition	10
3.3	Heavy-Light Decomposition	10

4 Chuỗi

4.1	Chiến lược giải bài chuỗi	10
4.2	Rolling Hash (61-bit Mersenne)	10
4.3	KMP	11
4.4	Z-Algorithm	11
4.5	Aho Corasick	11
4.6	Suffix Array	11
4.7	Suffix Automaton	11
4.8	Lyndon Factorization	12
4.9	Manacher	12
4.10	Palindrome Tree	12
4.11	De Bruijn (FKM)	12

5 Quy hoạch động (DP)

5.1	Reroot DP	12
5.2	Divide-and-Conquer DP và bất đẳng thức tứ giác	12
5.3	Tối ưu DP 1D/1D	13
5.4	Alien DP	13
5.5	Convex Hull Trick	13
5.6	Slope Trick	13
5.7	Bitmask DP	13
5.8	SOS DP	13
5.9	Tối ưu bằng monotonic queue	13
5.10	Expected Value Trick	13

6 Toán học

6.1	Công thức số học	14
6.2	Like-Euclid Floor Sum	14
6.3	Rabin-Miller	14
6.4	Sàng nhanh (Linear Sieve)	14
6.5	Đếm nguyên tố (Prime Counting)	14
6.6	Pollard Rho	14
6.7	Primitive Root	14
6.8	Baby-step Giant-Step	15
6.9	Căn modulo (Tonelli-Shanks)	15
6.10	Extended GCD	15
6.11	Diophantine	15
6.12	Stern-Brocot Tree	15
6.13	Hàm nhân tính và Möbius Inversion	15

6.14	Công thức tổ hợp	15
6.15	Hàm sinh (Generating Functions) và chuỗi Taylor	16
6.16	Truy hồi tuyến tính (Berlekamp-Massey)	16
6.17	FWHT	16
6.18	Lagrange	16
6.19	NTT (Number Theoretic Transform)	16
6.20	FFT (Fast Fourier Transform)	16
6.21	Công thức Formal Power Series (FPS)	16
6.22	Dại số tuyến tính (Gauss)	17
6.23	XOR Linear Basis	17
6.24	Burnside Lemma & Pólya Theorem	17
6.25	Chiến lược và suy luận trạng thái game	17
6.26	Xác suất và kỳ vọng	18
6.27	Bignum	18

7 Hình học

7.1	Primitive Geometry	18
7.2	Convex Hull	18
7.3	Rotating Calipers	18
7.4	Minkowski Sum	18
7.5	Point in Convex Polygon	18
7.6	Line Hull Intersection	18
7.7	Polygon Raycast	19
7.8	Half-plane Intersection	19
7.9	Smallest Enclosing Circle	19
7.10	Circle Tangents	19
7.11	Công thức hình học	19
7.12	AREA	20
7.13	Manhattan MST	20
7.14	Planar Dual Graph	20

8 Tiện ích và kỹ thuật

8.1	Template	20
8.2	Debugging	21
8.3	Script biên dịch và chạy	21
8.4	Phân tích biểu thức (Expression Parsing)	21
8.5	Lập lịch Johnson	21
8.6	Monte Carlo và Las Vegas	21
8.7	Lịch (Calendar)	21
8.8	Josephus	21
8.9	Kỹ thuật interactive	22
8.10	Random, Bit Tricks, Bitset	22
8.11	Biến đổi bài toán (Transformations)	22
8.12	Checklist trước khi nộp	24

C++: template và typename

Kiểu dữ liệu thay đổi: viết `template<typename T>` trước hàm hoặc `struct`; thay `T` bằng kiểu cụ thể khi dùng. `typename` và `class` tương đương trong khai báo tham số kiểu.

Cách gọi: hàm thường suy ra `T` từ đối số; ghi `<long long>` để chọn kiểu tính toán. Với `struct`, ghi rõ kiểu như `Sum<long long>`. Mỗi kiểu được biên dịch thành phiên bản riêng, không mất thêm chi phí gọi lúc chạy.

```
1 // C++17; requires <array> and <vector>.
2 template<typename T>
3 T bigger(T a, T b) { return a > b ? a : b; }
4
5 template<typename T>
6 struct Sum {
7     T value{};
8     void add(T x) { value += x; }
9 };
10
11 template<typename T, int N>
12 struct Buffer {
13     std::array<T, N> data{};
14 };
15
16 template<typename C>
17 typename C::value_type total(const C& a) {
18     typename C::value_type answer{};
19     for (const auto& x : a) answer += x;
20     return answer;
21 }
22
23 inline void typename_examples() {
24     auto a = bigger(3, 5); // T = int
25     auto b = bigger<long long>(1, 2LL);
26     Sum<long long> s;
27     s.add(a); s.add(b); // s.value = 7
28     Buffer<int, 4> buf;
29     buf.data[0] = a;
30     auto c = total(std::vector<int>{1, 2, 3});
31     (void)c; // c = 6
32 }
```

Phân biệt tham số: `typename T` là kiểu; `int N` là giá trị hằng lúc biên dịch. `typename Merge` cũng là kiểu, dùng để nhận một phép gộp (`struct` có `operator()` hoặc `lambda`).

Tên phụ thuộc: trong C++17, thêm `typename` trước kiểu lồng phụ thuộc tham số, như `typename C::value_type`; không thêm trước giá trị như `C::size`. Khi gọi hàm `template` thành viên qua đối tượng có kiểu phụ thuộc, viết `obj.template get<T>()`.

Lỗi hay gặp: `bigger(1, 2LL)` không suy ra cùng một `T`; dùng `bigger<long long>(1, 2LL)`. `Template` chỉ dùng được khi kiểu đáp ứng các phép trong thân hàm (ví dụ `>` hoặc `+=`). Đặt cả định nghĩa trong file được include, trước chỗ sử dụng.

1 Cấu trúc dữ liệu

Quy ước C++17: Chỉ số từ 0, đoạn $[l, r)$. File nguồn có sẵn header chuẩn và namespace; phần bọc lặp lại được ẩn khi in.

1.1 Iterative Segment Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn/cập nhật, $\mathcal{O}(n)$ dựng Segment tree trên monoid có thứ tự: gán điểm, gộp đoạn, tìm biên; bản dual cộng đoạn/truy vấn điểm

```
1 // Associative merge with identity; order is preserved (noncommutative
2 // supported).
3 template<typename T, typename Merge> struct SegmentTree {
4     int n, base = 1; T identity; Merge merge; vector<T> tree;
5     SegmentTree(const vector<T>&a, T identity, Merge merge): n(a.size()), identity(
6         identity), merge(merge) {
7         while (base < n) base *= 2;
8         while (2*base < n) copy(a.begin(), a.end(), tree.begin()+base);
9         for (int u=base-1; u; --u) tree[u]=merge(tree[u*2], tree[u*2+1]);
10    }
11    void set(int p, T value) {
12        assert(0 <= p && p < n); tree[p += base] = value;
13        while (p /= 2) tree[p] = merge(tree[p * 2], tree[p * 2 + 1]);
14    }
15    T fold(int l, int r) const {
16        assert(0 <= l && l <= r && r < n); T left=identity, right=identity;
17        for (l += base, r += base; l < r; l /= 2, r /= 2) {
18            if (l & 1) left = merge(left, tree[l++]);
19            if (r & 1) right = merge(tree[--r], right);
20        }
21        return merge(left, right);
22    }
23    // First failing position r for predicate(fold(l,r+1)); n if none.
24    // predicate(identity) true, predicate monotone under appending elements.
25    template<typename Predicate> int max_right(int l, Predicate predicate) const {
26        assert(0 <= l && l <= n && predicate(identity));
27        if (l == n) return n;
28        T value = identity; int u = l + base;
29        do {
30            while (!(u & 1)) u /= 2;
31            T candidate = merge(value, tree[u]);
32            if (!predicate(candidate)) {
33                while (u < base) {
34                    u *= 2; candidate = merge(value, tree[u]);
35                    if (predicate(candidate)) { value = candidate; ++u; }
36                }
37                return min(n, u - base);
38            }
39            value = candidate; ++u;
40        } while ((u & -u) != u);
41        return n;
42    }
43 };
44 struct RangeAddPointQuery {
45     int n; vector<long long> tree;
46     explicit RangeAddPointQuery(int n) : n(n), tree(2 * max(1, n)) {}
47     void add(int l, int r, long long value) {
48         assert(0 <= l && l <= r && r <= n);
49         for (l += n, r += n; l < r; l /= 2, r /= 2) {
50             if (l & 1) tree[l++] += value;
51             if (r & 1) tree[--r] += value;
52         }
53     }
54     long long get(int p) const {
55         assert(0 <= p && p < n); long long value = 0;
56         for (p += n; p; p /= 2) value += tree[p];
57         return value;
58     }
59 };
```

1.2 Lazy Segment Tree (lưu ý cài đặt)

Quy tắc lazy propagation, hợp thành tag (Affine, Range Set) và Tree Walk $\mathcal{O}(\log n)$

- Quy tắc push/pull:** Gọi `push(id, l, r)` đầu `update/query`; gọi `pull(id)` (`st[id] = merge(st[2*id], st[2*id+1])`) sau khi cập nhật các con.
- Nhiều lazy tag:**

– **Affine** ($x \mapsto a \cdot x + b$): Áp dụng tag mới (c, d) sau $(a, b) \implies c(ax + b) + d = (ca)x + (cb + d)$.
Tag: `laz_a = laz_a * c; laz_b = laz_b * c + d;`
Node: `val = val * c + (r - l + 1) * d;`

– **Range Set và Range Add:** Range Set ($x \leftarrow v$) ghi đè Range Add trước đó $\implies$ đặt lại `add = 0`.

- Tree Walk** ($\mathcal{O}(\log n)$): Nếu con trái thỏa ($\max \geq X$ hoặc tổng $\geq S$), đẩy lazy và đi trái; nếu không, điều chỉnh mục tiêu rồi đi phải.
- Giới hạn:** Cấp phát $4N$ khi đánh số từ 1 (`id < 1` và `id < 1 | 1`).

1.3 Sparse Segment Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn
Segment tree động, chỉ cấp phát node khi cần

```
1 // Sparse point-add / range-sum on integer domain [lo,hi); O(log(domain))
2 // operations.
3 struct SparseRangeSum {
4     struct Node { int left=-1, right=-1; long long sum=0; }; vector<Node> t {1};
5     long long lo, hi;
6     int child(int u, bool right) {
7         int v = right ? t[u].right : t[u].left;
8         if (v < 0) {
9             v = t.size(); t.emplace_back();
10            if (right) t[u].right = v;
11            else t[u].left = v;
12        }
13        return v;
14    }
15    void add(int u, long long l, long long r, long long p, long long x) {
16        t[u].sum += x;
17        if (r - l == 1) return;
18        long long m = l + (r - l) / 2;
19        if (p < m) add(child(u, false), l, m, p, x);
20        else add(child(u, true), m, r, p, x);
21    }
22    long long query(int u, long long l, long long r, long long a, long long b) const {
23        if (u < 0 || b <= l || r <= a) return 0;
24        if (a <= l && r <= b) return t[u].sum;
25        long long m = l + (r - l) / 2; return query(t[u].left, l, m, a, b) + query(t[u].right, m, r, a, b);
26    }
27    SparseRangeSum(long long lo, long long hi) : lo(lo), hi(hi) {
28        assert(lo < hi && (__int128) hi - lo <= LLONG_MAX);
29    }
30    void add(long long p, long long x) { assert(lo <= p && p < hi); add(0, lo, hi, p, x); }
31    long long sum(long long l, long long r) const {
32        assert(lo <= l && l <= r && r <= hi); return query(0, lo, hi, l, r);
33    }
34    size_t node_count() const { return t.size(); }
35 };
```

1.4 Cartesian Tree

Độ phức tạp: $\mathcal{O}(n)$ dựng bằng monotonic stack
Cây nhị phân có thứ tự heap theo giá trị và thứ tự BST theo chỉ số

- Tính chất:** Min-heap theo giá trị ($val[parent] \leq val[child]$) + duyệt in-order cho chỉ số mảng gốc (1..n). Gốc là giá trị nhỏ nhất. Cây con u ứng với đoạn cực đại $[L_u, R_u]$ mà $A[u]$ là nhỏ nhất ($size(u) = R_u - L_u + 1$).
- $\mathcal{O}(n)$ Dựng bằng stack:** Giữ nhánh phải trong stack. Với $i = 1..n$: pop u có $A[u] > A[i]$; node pop cuối thành $left[i] = u$; đỉnh còn lại có $right[top] = i$. Push i .

- Ứng dụng chính:**

– **RMQ $\iff$ LCA:** Giá trị nhỏ nhất trên đoạn $\min_{l \leq k \leq r} A[k] = A[LCA(l, r)]$.

– **Hình chữ nhật lớn nhất trong histogram:** $\max_u (A[u] \cdot size(u))$.

– D&C $\rightarrow$ Tree DP: Chuyển D&C trên mảng, chia tại phần tử nhỏ nhất, thành DP trên cây con.

1.5 Persistent Segment Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn/cập nhật

Persistent segment tree: cập nhật điểm, truy vấn các phiên bản cũ

```
1 // Immutable point-add versions; root 0 is the all-zero array. Ranges [l,r].
2 struct PersistentRangeSum {
3     struct Node { int left=0,right=0; long long sum=0; }; vector<Node> t{1};
4     int n;
5     int update(int root, int l, int r, int p, long long x) {
6         int u = t.size(); t.push_back(t[root]); t[u].sum += x;
7         if (r - l > 1) {
8             int m = (l + r) / 2;
9             if (p < m) {
10                 int v = update(t[u].left, l, m, p, x); t[u].left = v;
11             } else { int v = update(t[u].right, m, r, p, x); t[u].right = v; }
12         }
13         return u;
14     }
15     long long query(int u, int l, int r, int a, int b) const {
16         if (!u || b <= l || r <= a) return 0;
17         if (a <= l && r <= b) return t[u].sum;
18         int m=(l+r)/2; return query(t[u].left,l,m,a,b)+query(t[u].right,m,r,a,b);
19     }
20     explicit PersistentRangeSum(int n) : n(n) { assert(n >= 0); }
21     int add(int root, int p, long long x) {
22         assert(0<root&&root<(int)t.size())&&0<=p&&p<n; return update(root,0,n,p,x);
23     }
24     long long sum(int root, int l, int r) const {
25         assert(0<root&&root<(int)t.size())&&0<=l&&l<=r&&r<=n; return query(root,0,n,l,r);
26     }
27 };
```

1.6 Parallel Binary Search

Độ phức tạp: $\mathcal{O}((N + Q) \log(\text{range}) \log N)$

Binary search song song offline cho nhiều truy vấn đơn điệu

```
1 // Monotone test(query) after an event prefix. Returns first prefix length
2 // 0..M,
3 // or M+1 if never true. reset() restores prefix 0; apply(i) applies event i.
4 template<typename Reset,typename Apply,typename Test>vector<int>
5 parallel_binary_search(int M,int Q,Reset reset,Apply apply,Test test){
6     assert(M >= 0 && Q >= 0); vector<int> lo(Q, 0), hi(Q, M + 1);
7     for (;) {
8         vector<vector<int>> bucket(M + 1); bool active = false;
9         for (int q = 0; q < Q; ++q) if (lo[q] < hi[q]) {
10             active = true; bucket[(lo[q] + hi[q]) / 2].push_back(q);
11         }
12         if (!active) return lo;
13         reset();
14         for (int prefix = 0; prefix <= M; ++prefix) {
15             if (prefix) apply(prefix - 1);
16             for (int q : bucket[prefix]) if (test(q)) hi[q] = prefix;
17             else lo[q] = prefix + 1;
18         }
19     }
```

1.7 Wavelet Tree

Độ phức tạp: $\mathcal{O}(\log(\text{alphabet}))$ mỗi truy vấn

Wavelet tree: phần tử thứ k , tần suất và đếm giá trị trong $[L, R]$ trên đoạn

```
1 // Immutable array; ranges [l,r], k is zero-based.  $\mathcal{O}(\log \text{sigma})$  queries.
2 struct WaveletTree {
3     struct Node { int lo,hi,left=-1,right=-1; vector<int>pref; }; vector<Node>
4     nodes;
5     vector<long long> values; int n;
6     int build(vector<int> a, int lo, int hi) {
7         int id = nodes.size(); nodes.push_back( { lo, hi, -1, -1, {0} } );
8         if (hi - lo <= 1 || a.empty()) return id;
9         int mid = (lo + hi) / 2; vector<int> left, right;
10         for (int x : a) {
11             nodes[id].pref.push_back(nodes[id].pref.back() + (x < mid));
12             (x < mid ? left : right).push_back(x);
13         }
14         int l=build(move(left),lo,mid),r=build(move(right),mid,hi); nodes[id].
15         left=l;
16         nodes[id].right = r; return id;
17     }
18     int less(int id, int l, int r, int rank) const {
19         const auto& t = nodes[id];
20         if (l == r || rank <= t.lo) return 0;
21         if (t.hi <= rank) return r - l;
22         int a = t.pref[l], b = t.pref[r];
23         return less(t.left, a, b, rank) + less(t.right, l - a, r - b, rank);
24     }
25     explicit WaveletTree(const vector<long long>&a):values(a),n(a.size()){
26         sort(values.begin(), values.end());
27         values.erase(unique(values.begin(),values.end()),values.end()); vector<
28         int>ranks;
29         for(auto x:a)ranks.push_back(lower_bound(values.begin(),values.end(),x)-
30         values.begin());
31         build(move(ranks), 0, max(1, int(values.size())));
32     }
33     long long kth(int l, int r, int k) const {
34         assert(0<=l&&l<=r&&r<=n&&0<=k&&k<=r-l); int id=0;
35         while (nodes[id].hi - nodes[id].lo > 1) {
36             const auto& t = nodes[id]; int a = t.pref[l], b = t.pref[r];
37             if (k<b-a){ l=a; r=b; id=t.left; } else { k-=b-a; l=a; r=b; id=t.right
38             ; }
39         }
40         return values[nodes[id].lo];
41     }
42     int count(int l, int r, long long low, long long high) const {
43         assert(0 <= l && l <= r && r <= n);
44         if (low > high) return 0;
45         int a=lower_bound(values.begin(),values.end(),low)-values.begin();
46         int b=upper_bound(values.begin(),values.end(),high)-values.begin();
47         return less(0, l, r, b) - less(0, l, r, a);
48     }
49 };
```

1.8 Segment Tree Beat

Độ phức tạp: $\mathcal{O}((n + q) \log^2 n)$ khâu hao

Segment Tree Beats: chmin/chmax, cộng và truy vấn tổng trên đoạn

```
1 // [l,r]; values, shifts and sums must remain well inside +/- INF.
2 // chmin/chmax/sum: amortized logarithmic; with add: log^2 bound.
3 struct SegmentTreeBeats {
4     static constexpr long long INF = 1LL << 60;
5     struct Node {
6         long long sum=0,mx=-INF,mx2=-INF,mn=INF,mn2=INF,add=0; int cmx=0,cmn=0,
7         len=0;
8     }; int n; vector<Node> t;
9     void pull(int p) {
10         const auto&a=t[p*2],&b=t[p*2+1]; auto&c=t[p]; c.sum=a.sum+b.sum;
11         c.len=a.len+b.len; c.mx=max(a.mx,b.mx); c.mn=min(a.mn,b.mn);
12         c.cmx = (a.mx == c.mx ? a.cmx : 0) + (b.mx == c.mx ? b.cmx : 0);
13         c.cmn = (a.mn == c.mn ? a.cmn : 0) + (b.mn == c.mn ? b.cmn : 0);
14         c.mx2=max(a.mx==c.mx?a.mx2:a.mx,b.mx==c.mx?b.mx2:b.mx);
15         c.mn2=min(a.mn==c.mn?a.mn2:a.mn,b.mn==c.mn?b.mn2:b.mn);
16     }
17     void add_node(int p, long long x) {
18         auto&a=t[p]; a.sum+=x*a.len; a.mx+=x; a.mn+=x; a.add+=x;
19         if (a.mx2 != -INF) a.mx2 += x;
20         if (a.mn2 != INF) a.mn2 += x;
21     }
```

```
21 void cap(int p, long long x) {
22     auto&a = t[p];
23     if (x >= a.mx) return;
24     a.sum += (x - a.mx) * a.cmx;
25     if (a.mn == a.mx) a.mn = x;
26     else if (a.mn2 == a.mx) a.mn2 = x;
27     a.mx = x;
28 }
29 void floor_node(int p, long long x) {
30     auto&a = t[p];
31     if (x <= a.mn) return;
32     a.sum += (x - a.mn) * a.cmn;
33     if (a.mx == a.mn) a.mx = x;
34     else if (a.mx2 == a.mn) a.mx2 = x;
35     a.mn = x;
36 }
37 void push(int p) {
38     for (int c : {p * 2, p * 2 + 1}) {
39         if (t[p].add) add_node(c, t[p].add);
40         cap(c, t[p].mx); floor_node(c, t[p].mn);
41     }
42     t[p].add = 0;
43 }
44 void build(int p, int l, int r, const vector<long long>&a) {
45     if (r-l==1){ t[p].sum=t[p].mx=t[p].mn=a[l]; t[p].cmx=t[p].cmn=t[p].len=1;
46     return; }
47     int m=(l+r)/2; build(p*2,l,m,a); build(p*2+1,m,r,a); pull(p);
48 }
49 void update(int p,int l,int r,int ql,int qr,long long x,int kind){
50     if (qr<=l||r<=ql||(kind==0&&t[p].mx<=x)|| (kind==1&&t[p].mn>=x))return;
51     if (ql <= l && r <= qr) {
52         if (kind == 2) { add_node(p, x); return; }
53         if (kind == 0 && t[p].mx2 < x) { cap(p, x); return; }
54         if (kind == 1 && x < t[p].mn2) { floor_node(p, x); return; }
55     }
56     push(p); int m = (l + r) / 2; update(p * 2, l, m, ql, qr, x, kind);
57     update(p * 2 + 1, m, r, ql, qr, x, kind); pull(p);
58 }
59 long long query(int p, int l, int r, int ql, int qr) {
60     if (qr <= l || r <= ql) return 0;
61     if (ql <= l && r <= qr) return t[p].sum;
62     push(p); int m=(l+r)/2; return query(p*2,l,m,ql,qr)+query(p*2+1,m,r,ql,qr);
63 }
64 explicit SegmentTreeBeats(const vector<long long>&a):n(a.size()),t(4*max(1,
65     n)){
66     if (n) build(1, 0, n, a);
67 }
68 void chmin(int l, int r, long long x) {
69     if (l < r) update(1, 0, n, l, r, x, 0);
70 }
71 void chmax(int l, int r, long long x) {
72     if (l < r) update(1, 0, n, l, r, x, 1);
73 }
74 void add(int l, int r, long long x) {
75     if (l < r) update(1, 0, n, l, r, x, 2);
76 }
77 long long sum(int l,int r){ return l<r?query(1,0,n,l,r):0; }
```

1.9 Kd Tree

Độ phức tạp: $\mathcal{O}(n)$ mỗi truy vấn trong trường hợp xấu nhất; cấu trúc không gian trên cây cân bằng

Cây k chiều tĩnh: điểm gần nhất và truy vấn hộp đóng song song trục

```
1 // Balanced static k-d tree. Worst-case nearest/range query  $\mathcal{O}(n)$ .
2 template<size_t D> struct KDTree {
3     static_assert(D > 0); using Point = array<double, D>;
4     struct Node { int id,left=-1,right=-1; Point low,high; }; vector<Point>
5     points;
6     vector<Node> t; int root;
7     int build(vector<int>& ids, int l, int r, int depth) {
8         if (l == r) return-1;
9         int m = (l + r) / 2, axis = depth % D;
```

```

9     nth_element(ids.begin()+l,ids.begin()+m,ids.begin()+r,[&](int a,int b){
10         return points[a][axis] < points[b][axis];
11     }); int u=t.size(); t.push_back( {ids[m],-1,-1,points[ids[m]],points[ids[
12     m]]});
13     int a=build(ids,l,m,depth+1),b=build(ids,m+1,r,depth+1); t[u].left=a; t[u
14     ].right=b;
15     for (int v : {a, b}) if (v >= 0) for (size_t d = 0; d < D; ++d) {
16         t[u].low[d] = min(t[u].low[d], t[v].low[d]);
17         t[u].high[d] = max(t[u].high[d], t[v].high[d]);
18     }
19     return u;
20 }
21 double lower_bound(int u, const Point& q) const {
22     if (u < 0) return numeric_limits<double>::infinity();
23     double ans = 0;
24     for (size_t d = 0; d < D; ++d) {
25         double z=max( {0.,t[u].low[d]-q[d],q[d]-t[u].high[d]}); ans+=z*z;
26     }
27     return ans;
28 }
29 explicit KDTree(vector<Point> p) : points(move(p)) {
30     vector<int> ids(points.size()); iota(ids.begin(), ids.end(), 0);
31     root = build(ids, 0, ids.size(), 0);
32 }
33 optional<pair<int,double>>nearest(const Point&q,int excluded=-1)const {
34     int best = -1; double distance = numeric_limits<double>::infinity();
35     function<void(int)> visit = [&](int u) {
36         if (u < 0 || lower_bound(u, q) > distance) return;
37         int id = t[u].id;
38         if (id != excluded) {
39             double v = 0;
40             for(size_t d=0;d<D;++d)v+=(points[id][d]-q[d])*(points[id][d]-q[d]);
41             if(v<distance||(v==distance&&(best<0||id<best)))distance=v,best=id;
42         }
43         int a = t[u].left, b = t[u].right;
44         if (lower_bound(a, q) > lower_bound(b, q)) swap(a, b);
45         visit(a); visit(b);
46     }; visit(root);
47     if (best < 0) return nullopt;
48     return pair {best, distance};
49 }
50 vector<int> range(const Point& low, const Point& high) const {
51     vector<int> out;
52     function<void(int)> visit = [&](int u) {
53         if (u < 0) return;
54         for(size_t d=0;d<D;++d)if (t[u].high[d]<low[d]||high[d]<t[u].low[d])
55             return;
56         bool inside = true;
57         for(size_t d=0;d<D;++d)inside&=low[d]<=points[t[u].id][d]&&points[t[u].
58         id][d]<=high[d];
59         if (inside) out.push_back(t[u].id);
60         visit(t[u].left); visit(t[u].right);
61     }; visit(root); return out;
62 }
63 }
64 };

```

1.10 BIT2D

Độ phức tạp: $\mathcal{O}(\log^2 n)$ mỗi truy vấn

Fenwick 2D: tổng hình chữ nhật và cập nhật điểm

```

1 struct Fenwick2D {
2     int n, m; vector<vector<long long>>> t;
3     Fenwick2D(int n,int m):n(n),m(m),t(n+1,vector<long long>(m+1)){
4         assert(n >= 0 && m >= 0);
5     }
6     void add(int x, int y, long long value) {
7         assert(0 <= x && x < n && 0 <= y && y < m);
8         for(int i=x+1;i<=n;i+=i&-i)for(int j=y+1;j<=m;j+=j&-j)t[i][j]+=value;
9     }
10    long long prefix(int x, int y) const {
11        assert(0 <= x && x <= n && 0 <= y && y <= m); long long ans = 0;
12        for(int i=x; i; i-=i&-i)for(int j=y; j; j-=j&-j)ans+=t[i][j];
13        return ans;
14    }
15    long long sum(int x1, int y1, int x2, int y2) const {

```

```

16    assert(x1 <= x2 && y1 <= y2);
17    return prefix(x2,y2)-prefix(x1,y2)-prefix(x2,y1)+prefix(x1,y1);
18 }
19 };

```

1.11 Treap

Độ phức tạp: $\mathcal{O}(\log n)$ kỳ vọng mỗi thao tác

BST ngẫu nhiên: split/merge mảng và thao tác theo khóa ngẫu

```

1 // Implicit sequence with insert/erase/reverse/range-sum. RNG seed is
2 // explicit.
3 // Erased nodes remain in the arena; storage O(total insertions), no dangling
4 // pointers.
5 struct ImplicitTreap {
6     struct Node {
7         long long value,sum; uint64_t priority; int left=-1,right=-1,size=1; bool
8         rev=false;
9     }; vector<Node> t; int root = -1; mt19937_64 rng;
10    int size(int u) const { return u < 0 ? 0 : t[u].size; }
11    long long sum(int u) const { return u < 0 ? 0 : t[u].sum; }
12    void flip(int u){ if(u==0){ swap(t[u].left,t[u].right); t[u].rev^=1; } }
13    void push(int u) {
14        if(u>=0&&t[u].rev){ flip(t[u].left); flip(t[u].right); t[u].rev=false; }
15    }
16    void pull(int u) {
17        t[u].size = 1 + size(t[u].left) + size(t[u].right);
18        t[u].sum = t[u].value + sum(t[u].left) + sum(t[u].right);
19    }
20    pair<int, int> split(int u, int k) {
21        if (u < 0) return {-1, -1};
22        push(u);
23        if (size(t[u].left) >= k) {
24            auto[a,b]=split(t[u].left,k); t[u].left=b; pull(u); return {a,u};
25        }
26        auto[a,b]=split(t[u].right,k-size(t[u].left)-1); t[u].right=a; pull(u);
27        return {u, b};
28    }
29    int merge(int a, int b) {
30        if (a < 0) return b;
31        if (b < 0) return a;
32        if (t[a].priority > t[b].priority) {
33            push(a); t[a].right = merge(t[a].right, b); pull(a); return a;
34        }
35        push(b); t[b].left = merge(a, t[b].left); pull(b); return b;
36    }
37    explicit ImplicitTreap(uint64_t seed) : rng(seed) {}
38    int size() const { return size(root); }
39    void insert(int p, long long x) {
40        assert(0<=p&&p<=size()); int u=t.size(); t.push_back( {x,x,rng()});
41        auto[a, b] = split(root, p); root = merge(merge(a, u), b);
42    }
43    void erase(int l, int r) {
44        assert(0<=l&&l<=r&&r<=size()); auto[a,c]=split(root,r); auto[b,discard]=
45        split(a,l);
46        (void) discard; root = merge(b, c);
47    }
48    void reverse(int l, int r) {
49        assert(0<=l&&l<=r&&r<=size()); auto[a,c]=split(root,r); auto[b,m]=split(a
50        ,l);
51        flip(m); root = merge(merge(b, m), c);
52    }
53    long long sum(int l, int r) {
54        assert(0<=l&&l<=r&&r<=size()); auto[a,c]=split(root,r); auto[b,m]=split(a
55        ,l);
56        long long value=sum(m); root=merge(merge(b,m),c); return value;
57    }
58 }
59 };

```

1.12 Splay Tree / Link - cut Tree

Độ phức tạp: $\mathcal{O}(\log n)$ khấu hao mỗi thao tác

Link-Cut Tree: truy vấn đường đi động, link, cut và đổi gốc trên rừng

```

1 // Dynamic forest, vertex values, path sum. 0-based IDs; -1 is no node.
2 struct LinkCutTree {
3     struct Node {
4         int child[2] {-1,-1},parent=-1; bool rev=false; long long value=0,sum=0;
5         vector<Node> t;
6         bool is_root(int u) const {
7             int p=t[u].parent; return p<0||(t[p].child[0]!=u&&t[p].child[1]!=u);
8         }
9         long long sum(int u) const { return u < 0 ? 0 : t[u].sum; }
10        void pull(int u){ t[u].sum=t[u].value+sum(t[u].child[0])+sum(t[u].child[1])
11        ; }
12        void flip(int u){ if(u==0){ swap(t[u].child[0],t[u].child[1]); t[u].rev^=1;
13        } }
14        void push(int u) {
15            if(t[u].rev){ flip(t[u].child[0]); flip(t[u].child[1]); t[u].rev=false; }
16        }
17        void rotate(int u) {
18            int p=t[u].parent,g=t[p].parent,d=t[p].child[1]==u,v=t[u].child[d^1];
19            if (!is_root(p)) t[g].child[t[g].child[1] == p] = u;
20            t[u].parent=g; t[u].child[d^1]=p; t[p].parent=u; t[p].child[d]=v;
21            pull(p); pull(u);
22        }
23        void splay(int u) {
24            vector<int> path {u};
25            for (int v = u; !is_root(v);) path.push_back(v = t[v].parent);
26            for (auto it = path.rbegin(); it != path.rend(); ++it) push(*it);
27            while (!is_root(u)) {
28                int p = t[u].parent, g = t[p].parent;
29                if (!is_root(p))rotate((t[p].child[1]==u)==(t[g].child[1]==p)?p:u);
30                rotate(u);
31            }
32        }
33        int access(int u) {
34            int last = -1;
35            for (int v = u; v >= 0; v = t[v].parent) {
36                splay(v); t[v].child[1] = last;
37                if (last >= 0) t[last].parent = v;
38                pull(v); last = v;
39            }
40            splay(u); return last;
41        }
42        int root(int u) {
43            access(u);
44            while (push(u), t[u].child[0] >= 0) u = t[u].child[0];
45            splay(u); return u;
46        }
47        explicit LinkCutTree(const vector<long long>&values):t(values.size()){
48            for(int i=0; i<(int)t.size(); ++i)t[i].value=t[i].sum=values[i];
49        }
50        void make_root(int u) { access(u); flip(u); }
51        bool connected(int u, int v) { return u == v || root(u) == root(v); }
52        bool link(int u, int v) {
53            make_root(u);
54            if (root(v) == u) return false;
55            t[u].parent = v; return true;
56        }
57        bool cut(int u, int v) {
58            make_root(u); access(v);
59            if (t[v].child[0] != u || t[u].child[1] >= 0) return false;
60            t[v].child[0] = -1; t[u].parent = -1; pull(v); return true;
61        }
62        void set(int u,long long value){ access(u); t[u].value=value; pull(u); }
63        optional<long long> path_sum(int u, int v) {
64            if (!connected(u, v)) return nullopt;
65            make_root(u); access(v); return t[v].sum;
66        }
67        optional<int> lca(int rooted_at, int u, int v) {
68            if (!connected(rooted_at, u) || !connected(u, v)) return nullopt;
69            make_root(rooted_at); access(u); return access(v);
70        }
71    };

```

1.13 Farach - Colton - Bender

Độ phức tạp: $\mathcal{O}(n)$ dựng, $\mathcal{O}(1)$ truy vấn

$\mathcal{O}(n), \mathcal{O}(1)$ LCA qua Euler Tour, RMQ ± 1 và kỹ thuật Four Russians

```
1 // Nonempty tree, root=0 default; O(n) preprocessing, O(1) LCA.
2 struct FCBLCA {
3     vector<int> euler, depth, first, profile, log2; vector<vector<int>> sparse;
4     vector<vector<unsigned char>> micro; int block;
5     int better(int a, int b) const { return depth[a] <= depth[b] ? a : b; }
6     int inside(int b, int l, int r) const { return b * block + micro[profile[b]][l * block + r]; }
7     explicit FCBLCA(const vector<vector<int>> &adj, int root = 0) : first(adj.size() - 1) {
8         assert(!adj.empty()); struct Frame { int u, p, next, d; };
9         vector<Frame> stack { {root, -1, 0, 0} }; euler.push_back(root); depth.push_back(0);
10        first[root] = 0;
11        while (!stack.empty()) {
12            auto & f = stack.back();
13            if (f.next == int(adj[f.u].size())) {
14                stack.pop_back();
15                if (!stack.empty()) {
16                    euler.push_back(stack.back().u); depth.push_back(stack.back().d);
17                }
18                continue;
19            }
20            int v = adj[f.u][f.next++];
21            if (v == f.p) continue;
22            int u = f.u, d = f.d + 1; first[v] = euler.size(); euler.push_back(v); depth.push_back(d);
23            stack.push_back({v, u, 0, d});
24        }
25        int m = euler.size(), lg = 0;
26        while ((1LL << (lg + 1)) <= m) ++lg;
27        block = max(1, lg / 2); int count = (m + block - 1) / block;
28        micro.resize(1 << (block - 1), vector<unsigned char>(block * block));
29        for (int mask = 0; mask < int(micro.size()); ++mask) {
30            vector<int> d(block);
31            for (int i = 1; i < block; ++i) d[i] = d[i - 1] + ((mask >> (i - 1) & 1) ? 1 : -1);
32            for (int l = 0; l < block; ++l) {
33                int best = l;
34                for (int r = l; r < block; ++r) {
35                    if (d[r] < d[best]) best = r;
36                    micro[mask][l * block + r] = best;
37                }
38            }
39        }
40        profile.resize(count); vector<int> minima(count);
41        for (int b = 0; b < count; ++b) {
42            int start = b * block, end = min(m, start + block), best = start;
43            for (int i = start + 1; i < end; ++i) {
44                if (depth[i] > depth[i - 1]) profile[b] |= 1 << (i - start - 1);
45                best = better(best, i);
46            }
47            minima[b] = best;
48        }
49        log2.resize(count + 1);
50        for (int i = 2; i <= count; ++i) log2[i] = log2[i / 2] + 1;
51        sparse.push_back(move(minima));
52        for (int k = 1; (1 << k) <= count; ++k) {
53            sparse.push_back(vector<int>(count - (1 << k) + 1));
54            for (int i = 0; i < int(sparse[k].size()); ++i) sparse[k][i] = better(sparse[k - 1][i], sparse[k - 1][i + (1 << (k - 1))]);
55        }
56    }
57    int lca(int u, int v) const {
58        int l = first[u], r = first[v];
59        if (l > r) swap(l, r);
60        int a = l / block, b = r / block;
61        if (a == b) return euler[inside(a, l % block, r % block)];
62        int best = better(inside(a, l % block, block - 1), inside(b, 0, r % block));
63        if (a + 1 < b) {
64            int k = log2[b - a - 1]; best = better(best, better(sparse[k][a + 1], sparse[k][b - (1 << k)]));
65        }
66        return euler[best];
67    }
68 };
```

1.14 Trie

Độ phức tạp: $\mathcal{O}(\text{length})$ mỗi truy vấn

Trie nhị phân multiset 64-bit: thêm, xóa, đếm, XOR lớn nhất

```
1 // Multiset of unsigned 64-bit keys; duplicates and erasure supported.
2 struct BinaryTrie {
3     struct Node { int child[2] {-1, -1}; int count = 0; }; vector<Node> t {1};
4     int size() const { return t[0].count; }
5     int count(uint64_t x) const {
6         int u = 0;
7         for (int b = 63; b >= 0; --b) {
8             u = t[u].child[x >> b & 1];
9             if (u < 0) return 0;
10        }
11        return t[u].count;
12    }
13    void insert(uint64_t x) {
14        int u = 0; ++t[u].count;
15        for (int b = 63; b >= 0; --b) {
16            int c = x >> b & 1, v = t[u].child[c];
17            if (v < 0) { v = t.size(); t.emplace_back(); t[u].child[c] = v; }
18            u = v; ++t[u].count;
19        }
20    }
21    bool erase(uint64_t x) {
22        if (!count(x)) return false;
23        int u = 0; --t[u].count;
24        for (int b = 63; b >= 0; --b) { u = t[u].child[x >> b & 1]; --t[u].count; }
25        return true;
26    }
27    optional<uint64_t> max_xor(uint64_t x) const {
28        if (!size()) return nullopt;
29        uint64_t answer = 0; int u = 0;
30        for (int b = 63; b >= 0; --b) {
31            int c = x >> b & 1, v = t[u].child[c ^ 1];
32            if (v >= 0 && t[v].count) { answer |= uint64_t(1) << b; u = v; } else u = t[u].child[c];
33        }
34        return answer;
35    }
36 };
```

1.15 Lichao Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn/thêm

Li Chao Tree: thêm đường thẳng online, truy vấn min/max

```
1 // Integer-domain minimum envelope [lo, hi]; query nullopt before first insertion.
2 struct LiChao {
3     struct Line {
4         long long m, b;
5         __int128 at(long long x) const { return (__int128) m * x + b; }
6     }; struct Node { Line line; int left = -1, right = -1; }; vector<Node> t; int root = -1;
7     long long lo, hi;
8     int add(int u, long long l, long long r, Line line) {
9         if (u < 0) { u = t.size(); t.push_back({line}); return u; }
10        long long mid = l + (r - l) / 2;
11        bool left = line.at(l) < t[u].line.at(l), middle = line.at(mid) < t[u].line.at(mid);
12        if (middle) swap(line, t[u].line);
13        if (r - l == 1) return u;
14        if (left != middle) {
15            int v = add(t[u].left, l, mid, line); t[u].left = v;
16        } else { int v = add(t[u].right, mid, r, line); t[u].right = v; }
17        return u;
18    }
19    optional<__int128> query(int u, long long l, long long r, long long x) const {
20        if (u < 0) return nullopt;
21        __int128 ans = t[u].line.at(x);
22        if (r - l > 1) {
23            long long m = l + (r - l) / 2;
24            auto child = x < m ? query(t[u].left, l, m, x) : query(t[u].right, m, r, x);
25            if (child) ans = min(ans, *child);
26        }
27    }
```

```
26     }
27     return ans;
28 }
29 LiChao(long long lo, long long hi) : lo(lo), hi(hi) {
30     assert(lo < hi && (__int128) hi - lo <= LLONG_MAX);
31 }
32 void add(long long m, long long b) { root = add(root, lo, hi, {m, b}); }
33 optional<__int128> minimum(long long x) const {
34     assert(lo <= x && x < hi); return query(root, lo, hi, x);
35 }
36 };
```

1.16 Line Container

Độ phức tạp: $\mathcal{O}(\log n)$ khâu hao mỗi thao tác

Convex hull của các đường thẳng động, hệ số góc và điểm truy vấn x tùy ý

```
1 // Arbitrary slopes and x, maximum queries. Integer intersections use int128.
2 struct LineContainer {
3     struct Line {
4         long long m, b; mutable __int128 end;
5         bool operator< (const Line & other) const { return m < other.m; }
6         bool operator< ( __int128 x) const { return end < x; }
7     }; multiset<Line, less<>> h; static constexpr __int128 INF = (__int128) 1 << 126;
8     static __int128 floor_div(__int128 a, __int128 b) {
9         __int128 q = a / b;
10        if (a % b && ((a < 0) != (b < 0))) --q;
11        return q;
12    }
13    bool intersect(multiset<Line, less<>>::iterator x, multiset<Line, less<>>::iterator y) {
14        if (y == h.end()) { x->end = INF; return false; }
15        if (x->m == y->m) x->end = x->b > y->b ? INF : -INF;
16        else x->end = floor_div((__int128) y->b - x->b, (__int128) x->m - y->m);
17        return x->end >= y->end;
18    }
19    void add(long long m, long long b) {
20        auto z = h.insert({m, b, 0}).y = z++, x = y;
21        while (intersect(y, z)) z = h.erase(z);
22        if (x != h.begin() && intersect(--x, y)) intersect(x, y = h.erase(y));
23        while ((y = x) != h.begin() && (--x->end > y->end) intersect(x, h.erase(y)));
24    }
25    optional<__int128> maximum(long long x) const {
26        if (h.empty()) return nullopt;
27        auto line = h.lower_bound((__int128) x); return (__int128) line->m * x + line->b;
28    }
29 };
```

1.17 Arpa Trick

Độ phức tạp: $\mathcal{O}(na(n))$

Truy vấn min/max đoạn offline bằng DSU

```
1 // Offline maximum indices; queries [l, r] nonempty; ties choose rightmost.
2 template<typename T> vector<int> range_max_indices(const vector<T> & a, const vector<pair<int, int>> & queries) {
3     int n = a.size(); vector<vector<int, int>>> ending(n);
4     for (int i = 0; i < int(queries.size()); ++i) {
5         auto [l, r] = queries[i]; assert(0 <= l && l < r && r < n); ending[r - 1].push_back({l, i});
6     }
7     vector<int> parent(n), stack, answer(queries.size());
8     iota(parent.begin(), parent.end(), 0);
9     auto find = [&](int u) {
10        int v = u;
11        while (parent[v] != v) v = parent[v];
12        while (parent[u] != u) { int next = parent[u]; parent[u] = v; u = next; }
13        return v;
14    };
15    for (int r = 0; r < n; ++r) {
16        while (!stack.empty() && a[stack.back()] <= a[r]) {
17            parent[stack.back()] = r; stack.pop_back();
18        }
19    }
```

```
19     stack.push_back(r);
20     for (auto [l, id] : ending[r]) answer[id] = find(l);
21 }
22 return answer;
23 }
```

1.18 Chia căn (Square Root Decomposition)

Độ phức tạp: $\mathcal{O}(n\sqrt{n})$

Chia căn: mảng chia block và thuật toán Mo theo thứ tự Hilbert

```
1 struct BlockArray {
2     int n, block; vector<long long> a, lazy;
3     explicit BlockArray(vector<long long> values : n(values.size()), block(max(1, (
4         int)sqrt(n))), a(move(values)), lazy((n+block-1)/block){}
5     void add(int l, int r, long long x) {
6         assert(0 <= l && l <= r && r <= n);
7         while (l < r && l % block) a[l++] += x;
8         while (l + block <= r) { lazy[l / block] += x; l += block; }
9         while (l < r) a[l++] += x;
10    }
11    long long get(int i) const { assert(0 <= i && i < n); return a[i] + lazy[i/block]; }
12 };
13 uint64_t hilbert_order(uint32_t x, uint32_t y, int power, int rotation=0) {
14     assert(0 <= power && power <= 31);
15     if (!power) return 0;
16     uint32_t half = uint32_t(1) << (power-1); int segment = x < half ? (y < half ? 0 : 3) : (y <
17         half ? 1 : 2);
18     segment = (segment + rotation) & 3; static constexpr int delta[] {3, 0, 0, 1};
19     uint64_t size = uint64_t(1) << (2 * power - 2);
20     auto sub = hilbert_order(x < (half-1), y < (half-1), power-1, (rotation + delta[
21         segment]) & 3);
22     return segment * size + ((segment == 1 || segment == 2) ? sub : size - sub - 1);
23 }
24 // Current range starts empty; add/remove receive array indices, answer
25 // receives query ID.
26 template<typename Add, typename Remove, typename Answer> auto mo_queries(int n,
27     const vector<pair<int, int>> & ranges, Add add, Remove remove, Answer answer) {
28     int power = 0;
29     while ((uint64_t(1) << power) <= (uint64_t) n) ++power;
30     assert(power <= 31); vector<pair<uint64_t, int>> order;
31     for (int id = 0; id < (int) ranges.size(); ++id) {
32         auto [l, r] = ranges[id]; assert(0 <= l && l <= r && r <= n);
33         order.push_back({ hilbert_order(l, r, power), id });
34     }
35     sort(order.begin(), order.end());
36     vector<decay_t<invoke_result_t<Answer, int>>> result(ranges.size()); int l=0,
37         r=0;
38     for (auto [key, id] : order) {
39         (void) key; auto [a, b] = ranges[id];
40         while (l > a) add(--l);
41         while (r < b) add(++r);
42         while (l < a) remove(l++);
43         while (r > b) remove(--r);
44         result[id] = answer(id);
45     }
46     return result;
47 }
```

1.19 Hash Map (SplitMix64)

Độ phức tạp: $\mathcal{O}(1)$ kỳ vọng mỗi lần tra cứu

Hash map khóa không dấu, hash SplitMix64 với seed do người gọi chọn

```
1 struct SplitMixHash {
2     uint64_t seed;
3     explicit SplitMixHash(uint64_t seed = 0x712367) : seed(seed) {}
4     static uint64_t mix(uint64_t x) {
5         x += 0x9e3779b97f4a7c15ULL; x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
6         x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL; return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const { return mix(x + seed); }
```

```
9 }; template<typename Value> using HashMap = unordered_map<uint64_t, Value,
10     SplitMixHash>;
11 // Example: HashMap<int> counts{0, SplitMixHash(seed)}; counts.reserve(
12     expected_size);
```

1.20 Erasable Priority Queue

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi thao tác

Priority queue max/min, xóa lazy phần tử tùy ý

```
1 // erase removes one occurrence if present.  $\mathcal{O}(\log n)$  insert/erase/pop
2 // amortized.
3 template<typename T, typename Compare = less<T>> struct ErasablePriorityQueue {
4     priority_queue<T, vector<T>, Compare> q, deleted; map<T, size_t, Compare> counts;
5     size_t live = 0;
6     void flush() {
7         while (!deleted.empty() && !q.empty() && q.top() == deleted.top()) {
8             q.pop(); deleted.pop();
9         }
10    }
11    explicit ErasablePriorityQueue(Compare cmp = Compare {}): q(cmp), deleted(cmp),
12        counts(cmp) {}
13    size_t size() const { return live; }
14    bool empty() const { return live == 0; }
15    void insert(const T& x) { q.push(x); ++counts[x]; ++live; }
16    bool erase(const T& x) {
17        auto it = counts.find(x);
18        if (it == counts.end()) return false;
19        if (!--it->second) counts.erase(it);
20        deleted.push(x); --live; flush(); return true;
21    }
22    const T& top() { flush(); assert(live); return q.top(); }
23    void pop() {
24        T value = top(); q.pop(); auto it = counts.find(value);
25        if (!--it->second) counts.erase(it);
26        --live; flush();
27    }
28 }
```

1.21 Cấu trúc mở rộng (Binary Doubling)

Độ phức tạp: $\mathcal{O}(A \log n)$ thêm khâu hao, với cấu trúc tĩnh dựng được trong $\mathcal{O}(A)$

Kỹ thuật gộp block theo lũy thừa 2: thêm online cho cấu trúc tĩnh (Aho, CHT)

```
1 // Binary-counter buckets. Builder(const vector<T>&) returns a static index
2 // by value.
3 // visit(index) called once per occupied bucket; caller combines query
4 // results.
5 template<typename T, typename Builder> struct ExpandableIndex {
6     using Index = decay_t<invoke_result_t<Builder, const vector<T>&>>; Builder
7     build;
8     vector<vector<T>> buckets; vector<optional<Index>> indices;
9     explicit ExpandableIndex(Builder builder) : build(move(builder)) {}
10    void insert(T value) {
11        vector<T> carry; carry.push_back(move(value)); size_t level = 0;
12        for (;;) ++level {
13            if (level == buckets.size()) { buckets.emplace_back(); indices.emplace_back
14                (); }
15            if (buckets[level].empty()) {
16                auto index = build(carry); buckets[level] = move(carry);
17                indices[level].emplace(move(index)); break;
18            }
19            for (auto& x : buckets[level]) carry.push_back(move(x));
20            buckets[level].clear(); indices[level].reset();
21        }
22    }
23    template<typename Visit> void query(Visit visit) const {
24        for (const auto& index : indices) if (index) visit(*index);
25    }
26 }
```

1.22 Interval Set

Độ phức tạp: $\mathcal{O}((k+1) \log s)$ mỗi lần gán, k đoạn bị xóa, s đoạn đang lưu

Phân hoạch đoạn: gán đoạn, gộp các đoạn kề nhau cùng giá trị.

```
1 // Complete interval partition of [lo, hi), adjacent equal values merged.
2 // Assign takes  $\mathcal{O}((\text{erased\_intervals}+1) \cdot \log(\text{number\_of\_intervals}))$  worst case.
3 template<typename T, typename Equal = equal_to<T>> struct IntervalSet {
4     struct Entry { long long end; T value; }; map<long long, Entry> segments;
5     long long lo, hi; Equal equal;
6     auto split(long long x) {
7         if (x == hi) return segments.end();
8         auto it = prev(segments.upper_bound(x));
9         if (it->first == x) return it;
10        long long end = it->second.end; T value = it->second.value; it->second.end = x;
11        return segments.emplace(x, Entry {end, move(value)}).first;
12    }
13    struct Interval { long long l, r; T value; };
14    IntervalSet(long long lo, long long hi, T initial, Equal eq = Equal {}): lo(lo),
15        hi(hi), equal(move(eq)) {
16        assert(lo < hi); segments.emplace(lo, Entry {hi, move(initial)});
17    }
18    const T& get(long long x) const {
19        assert(lo <= x && x < hi); return prev(segments.upper_bound(x))->second.value;
20    }
21    void assign(long long l, long long r, T value) {
22        assert(lo <= l && l <= r && r <= hi);
23        if (l == r) return;
24        auto right = split(r, left = split(l); segments.erase(left, right);
25        auto it = segments.emplace(l, Entry {r, move(value)}).first;
26        if (it != segments.begin()) {
27            auto p = prev(it);
28            if (equal(p->second.value, it->second.value)) {
29                p->second.end = it->second.end; segments.erase(it); it = p;
30            }
31        }
32        auto next = std::next(it);
33        if (next != segments.end() && equal(it->second.value, next->second.value)) {
34            it->second.end = next->second.end; segments.erase(next);
35        }
36    }
37    vector<Interval> intervals() const {
38        vector<Interval> out;
39        for (auto& [l, e] : segments) out.push_back({ l, e.end, e.value });
40        return out;
41    }
```

2 Đồ thị

2.1 Minimum Cost Flow

Độ phức tạp: $\mathcal{O}(nm + Am \log n)$, A lần tăng luồng

Mín-cost k -flow: chuyển k đơn vị qua đồ thị có capacity và chi phí

```
1 // Capacities nonnegative; no reachable negative-cost cycle in residual graph
2 // Bellman-Ford initializes potentials; Dijkstra for each augmentation.
3 struct MinCostFlow {
4     struct Edge { int to, rev; long long cap, cost; }; vector<vector<Edge>> g;
5     struct Result { long long flow, cost; };
6     explicit MinCostFlow(int n) : g(n) {}
7     void add_edge(int u, int v, long long cap, long long cost) {
8         assert(cap >= 0); int a = g[u].size(), b = g[v].size();
9         g[u].push_back({ v, b+(u==v), cap, cost }); g[v].push_back({ u, a, 0, -cost });
10    }
11    Result send(int s, int t, long long limit) {
12        assert(s != t && limit >= 0); int n = g.size(); constexpr long long inf = 1LL << 60;
13        vector<long long> potential(n, inf); potential[s] = 0;
14        for (int iter = 0; iter < n; ++iter) {
15            bool changed = false;
16            for (int u = 0; u < n; ++u) if (potential[u] < inf) for (auto& e : g[u]) if (e.cap &&
17                potential[e.to] > potential[u] + e.cost) {
18                potential[e.to] = potential[u] + e.cost; changed = true;
19            }
```

```

18     }
19     if (!changed) break;
20     if (iter == n - 1) throw invalid_argument("negative cost cycle");
21 }
22 for (auto& x : potential) if (x == inf) x = 0;
23 Result result {0, 0};
24 while (result.flow < limit) {
25     vector<long long>dist(n,inf); vector<int>pv(n),pe(n); dist[s]=0;
26     priority_queue<pair<long long,int>,vector<pair<long long,int>>,greater<
pair<long long,int>>>q;
27     q.push( {0, s});
28     while (!q.empty()) {
29         auto [d, u] = q.top(); q.pop();
30         if (d != dist[u]) continue;
31         for (int i = 0; i < int(g[u].size()); ++i) {
32             auto& e = g[u][i];
33             if (!e.cap) continue;
34             long long nd = d + e.cost + potential[u] - potential[e.to];
35             if (nd < dist[e.to]) {
36                 dist[e.to]=nd; pv[e.to]=u; pe[e.to]=i; q.push( {nd,e.to});
37             }
38         }
39     }
40     if (dist[t] == inf) break;
41     for(int u=0; u<n; ++u)if(dist[u]<inf)potential[u]+=dist[u];
42     long long f = limit - result.flow, cost = 0;
43     for(int v=t; v!=s; v=pv[v])f=min(f,g[pv[v]][pe[v]].cap);
44     for (int v = t; v != s; v = pv[v]) {
45         auto&e=g[pv[v]][pe[v]]; cost+=e.cost; e.cap-=f; g[v][e.rev].cap+=f;
46     }
47     result.flow += f; result.cost += f * cost;
48 }
49 return result;
50 }
51 };

```

2.2 MinCut - MaxFlow

Độ phức tạp: $\mathcal{O}(EV^2)$

Dinic: số đường đi/đường đi không chung đỉnh lớn nhất từ source đến sink.

• **Tô hai màu lưới:** Chia ô theo $(r + c) \bmod 2 \implies$ đồ thị hai phía. Dùng lát domino, đặt quân không tấn công nhau, bắn tia laser trên lưới.

• **Tách đỉnh (Vertex Splitting)** ($u_{in} \rightarrow u_{out}$): Giới hạn capacity đỉnh hoặc buộc đi qua đỉnh (capacity c_u , chi phí w_u).

• **Max-Weight Closure (Project Selection):**

- Quy tắc: chọn u thì phải chọn v (cạnh có hướng $u \rightarrow v$ với $\text{cap} = \infty$).
- Trọng số dương $W_u > 0$: thêm cạnh $S \rightarrow u$ với $\text{cap} = W_u$.
- Trọng số âm $W_v < 0$: thêm cạnh $v \rightarrow T$ với $\text{cap} = -W_v$.
- Max Profit = $\sum_{W_u > 0} W_u - \text{MinCut}(S, T)$.

• **Phân nhóm với hai lựa chọn:** Gán i vào S (chi phí A_i) hoặc T (B_i). Nếu $i \in S, j \in T$, phạt $C_{i,j}$. Thêm $S \rightarrow i$ (B_i), $i \rightarrow T$ (A_i), $i \rightarrow j$ ($C_{i,j}$). Tổng chi phí = min-cut.

• **Circulation có cận dưới/trên** $[L_e, R_e]$: Thay cạnh $e = (u, v)$ bằng capacity $R_e - L_e$. Đặt $d(u) = \sum_{in} L - \sum_{out} L$. Thêm super-source S^* , super-sink T^* . Với $d(u) > 0$, thêm $S^* \rightarrow u$ capacity $d(u)$; với $d(u) < 0$, thêm $u \rightarrow T^*$ capacity $-d(u)$. Khả thi $\iff$ mọi cạnh từ S^* bão hòa.

```

1 // Directed residual network. add_edge returns forward edge ID; self-loops
  supported.
2 struct Dinic {
3     struct Edge { int from,to; long long capacity,flow; }; vector<Edge>edges;

```

```

4     vector<vector<int>> adj; vector<int> level, next;
5     long long dfs(int u, int sink, long long pushed) {
6         if (u == sink) return pushed;
7         for (int& i = next[u]; i < int(adj[u].size()); ++i) {
8             int id = adj[u][i]; auto& e = edges[id];
9             if (level[e.to] != level[u] + 1 || e.flow == e.capacity) continue;
10            long long f = dfs(e.to, sink, min(pushed, e.capacity - e.flow));
11            if (f) { e.flow += f; edges[id ^ 1].flow -= f; return f; }
12        }
13        return 0;
14    }
15    explicit Dinic(int n) : adj(n), level(n), next(n) {}
16    int add_edge(int u, int v, long long capacity) {
17        assert(capacity >= 0); int id = edges.size(); adj[u].push_back(id);
18        adj[v].push_back(id + 1); edges.push_back( {u, v, capacity, 0});
19        edges.push_back( {v, u, 0, 0}); return id;
20    }
21    long long max_flow(int source, int sink, long long limit = LLONG_MAX){
22        assert(source != sink); long long total = 0;
23        while (total < limit) {
24            fill(level.begin(),level.end(),-1); queue<int>q; q.push(source); level[
source]=0;
25            while (!q.empty()) {
26                int u = q.front(); q.pop();
27                for (int id : adj[u]) {
28                    auto& e = edges[id];
29                    if (level[e.to] < 0 && e.flow < e.capacity) {
30                        level[e.to] = level[u] + 1; q.push(e.to);
31                    }
32                }
33            }
34            if (level[sink] < 0) break;
35            fill(next.begin(), next.end(), 0);
36            while (total < limit) {
37                long long f = dfs(source, sink, limit - total);
38                if (!f) break;
39                total += f;
40            }
41        }
42        return total;
43        // additional flow; mutates residual network.
44    }
45    vector<bool> source_side(int source) const {
46        vector<bool>seen(adj.size()); queue<int>q; q.push(source); seen[source]=
true;
47        while (!q.empty()) {
48            int u = q.front(); q.pop();
49            for (int id : adj[u]) {
50                auto& e = edges[id];
51                if(!seen[e.to]&&e.flow<e.capacity){ seen[e.to]=true; q.push(e.to); }
52            }
53        }
54        return seen;
55    }
56 };

```

2.3 Bipartite Matching

Độ phức tạp: $\mathcal{O}(E\sqrt{V})$

Hopcroft-Karp: vertex cover, path cover không chung đỉnh/tổng quát, antichain (Dilworth).

• **Định lý König:** Max Matching = Min Vertex Cover.

• **Max Independent Set:** $|V| - \text{Min Vertex Cover} = |V| - \text{Max Matching}$.

• **Min Edge Cover** (không có đỉnh cô lập): $|V| - \text{Max Matching}$.

• **Path cover nhỏ nhất trên DAG:**

– *Không chung đỉnh:* Tách $u \rightarrow u_{out}, u_{in}$; dựng matching hai phía $u_{out} \rightarrow v_{in}$ cho cạnh $u \rightarrow v$. Kết quả: $|V| - \text{Max Matching}$.

– *Tổng quát (được chung đỉnh):* Tính transitive closure (Floyd-Warshall/reachability), rồi giải path cover không chung đỉnh.

• **Định lý Dilworth:** Trong poset hữu hạn (reachability trên DAG), kích thước **maximum antichain** (tập lớn nhất gồm các đỉnh không đến được nhau) bằng **minimum path cover**.

• **Định lý Hall:** Đồ thị hai phía (X, Y) có matching phủ hết $X \iff \forall S \subseteq X, |N(S)| \geq |S|$.

• **Kích thước maximum matching qua Hall:** $|X| - \max_{S \subseteq X} (|S| - |N(S)|)$.

```

1 // Left vertices [0,L), right [0,R). solve() returns maximum cardinality.
2 struct HopcroftKarp {
3     vector<vector<int>> g; vector<int> distance; int shortest;
4     bool augment(int u) {
5         for (int v : g[u]) {
6             int next = right[v];
7             if ((next<0&&distance[u]+1==shortest)|| (next>=0&&distance[next]==
distance[u]+1&&augment(next))) {
8                 left[u] = v; right[v] = u; return true;
9             }
10        }
11        distance[u] = -1; return false;
12    }
13    vector<int> left, right;
14    HopcroftKarp(int L,int R):g(L),distance(L),left(L,-1),right(R,-1){}
15    void add_edge(int u, int v) { g[u].push_back(v); }
16    int solve() {
17        int count=count_if(left.begin(),left.end(),[](int x){ return x>=0; });
18        for (;;) {
19            fill(distance.begin(),distance.end(),-1); queue<int>q; shortest=INT_MAX
;
20            for (int u = 0; u < int(g.size()); ++u) if (left[u] < 0) {
21                distance[u] = 0; q.push(u);
22            }
23            while (!q.empty()) {
24                int u = q.front(); q.pop();
25                if (distance[u] + 1 > shortest) continue;
26                for (int v : g[u]) if (right[v] < 0) shortest = distance[u] + 1;
27                else if (distance[right[v]] < 0) {
28                    distance[right[v]] = distance[u] + 1; q.push(right[v]);
29                }
30            }
31            if (shortest == INT_MAX) break;
32            for(int u=0; u<int(g.size()); ++u)if(left[u]<0&&augment(u))++count;
33        }
34        return count;
35    }
36    // Call after solve(); returns {left IDs,right IDs} in a minimum vertex
cover.
37    pair<vector<int>, vector<int>> min_vertex_cover() const {
38        vector<bool> l(left.size()), r(right.size()); queue<int> q;
39        for (int u = 0; u < int(left.size()); ++u) if (left[u] < 0) {
40            l[u] = true; q.push(u);
41        }
42        while (!q.empty()) {
43            int u = q.front(); q.pop();
44            for (int v : g[u]) if (left[u] != v && !r[v]) {
45                r[v] = true; int next = right[v];
46                if (next >= 0 && !l[next]) { l[next] = true; q.push(next); }
47            }
48        }
49        pair<vector<int>, vector<int>> answer;
50        for(int u=0; u<int(l.size()); ++u)if(!l[u])answer.first.push_back(u);
51        for(int v=0; v<int(r.size()); ++v)if(r[v])answer.second.push_back(v);
52        return answer;
53    }
54 };

```

2.4 Hungarian Matching

Độ phức tạp: $\mathcal{O}(V^3)$

Ghép cặp hoàn hảo chi phí nhỏ nhất trên đồ thị hai phía

```
1 struct Assignment { long long cost; vector<int> column; };
2 // Minimum rectangular assignment, rows<=columns, costs may be negative.
3 Assignment hungarian(const vector<vector<long long>>& a) {
4     int n = a.size();
5     if (!n) return {
6         0, {}
7     }; int m=a[0].size(); assert(n<=m); vector<long long> u(n+1), v(m+1);
8     vector<int> p(m + 1), way(m + 1);
9     for (int i = 1; i <= n; ++i) {
10         p[0]=i; int j0=0; vector<long long> best(m+1, 1LL<<60); vector<bool> used(m+1);
11         do {
12             used[j0]=true; int row=p[j0], j1=0; long long delta=1LL<<60;
13             for (int j = 1; j <= m; ++j) if (!used[j]) {
14                 long long value = a[row - 1][j - 1] - u[row] - v[j];
15                 if (value < best[j]) { best[j] = value; way[j] = j0; }
16                 if (best[j] < delta) { delta = best[j]; j1 = j; }
17             }
18             for (int j = 0; j <= m; ++j) if (used[j]) {
19                 u[p[j]] += delta; v[j] -= delta;
20             } else best[j] -= delta;
21             j0 = j1;
22         } while (p[j0]);
23         while (p[j0]);
24         do { int previous = way[j0]; p[j0] = p[previous]; j0 = previous; }
25         while (j0);
26     }
27     vector<int> column(n);
28     for (int j = 1; j <= m; ++j) if (p[j]) column[p[j] - 1] = j - 1;
29     return {-v[0], column};
30 }
```

2.5 General Matching

Độ phức tạp: $\mathcal{O}(V^3)$

Edmonds Blossom: maximum matching trên đồ thị tổng quát.

- **Tutte Matrix & Rabin-Vazirani** ($\mathcal{O}(V^3)$): Với $(u, v) \in E$ ($u < v$), đặt $T_{u,v} = r \in [1, p-1]$, $T_{v,u} = -r \pmod{p}$. Kịch thước $= \frac{1}{2} \text{rank}(T)$ (Gauss mod p).

– *Khôi phục Rabin-Vazirani*: Tính $A = T^{-1} \pmod{p}$ (trên cơ sở $2K \times 2K$). Cạnh (u, v) thuộc một matching $\iff A_{u,v} \neq 0$. Chọn cặp đó, ghép lại rồi cập nhật A trong $\mathcal{O}(V^2)$ bằng Schur complement: $A'_{x,y} = A_{x,y} + \frac{A_{x,u}A_{y,v} - A_{x,v}A_{y,u}}{A_{u,v}} \pmod{p}$. Lặp cho K cặp $\implies$ tổng $\mathcal{O}(V^3)$.

- **Quy về max b -matching** ($\deg(i) \leq a_i$): Đỉnh $i \rightarrow a_i$ slot ($a_i \leftarrow \min(a_i, \deg(i))$). Cạnh $e = (u, v) \rightarrow L_e, R_e$, nối $L_e - R_e$, $L_e - \text{slot}(u)$, $R_e - \text{slot}(v)$. Bỏ e : match $L_e - R_e$ (+1). Chọn e : match L_e, R_e với slot (+2, lợi thêm +1). Đồ thị phụ $|V'| \leq 4M \implies$ Blossom.

```
1 // Edmonds blossom; simple undirected adjacency. Returns mate[u] or -1.
2 vector<int> general_matching(const vector<vector<int>>& adjacency) {
3     int n=adjacency.size(); std::mt19937 rng(712367); vector<int> match(n,-1);
4     vector<int> aux(n, -1); vector<int> label(n); vector<int> orig(n);
5     vector<int> parent(n, -1); queue<int> q; int aux_time = -1;
6     auto lca = [&](int v, int u) {
7         aux_time++;
8         while (true) {
9             if (v != -1) {
10                 if (aux[v] == aux_time) { return v; }
11                 aux[v] = aux_time;
12                 if (match[v]==-1){ v=-1; } else { v=orig[parent[match[v]]]; }
13             }
14             swap(v, u);
15         }
16     }
```

```
15     }
16 };
17 // lca
18 auto blossom = [&](int v, int u, int a) {
19     while (orig[v] != a) {
20         parent[v] = u; u = match[v];
21         if (label[u] == 1) { label[u] = 0; q.push(u); }
22         orig[v] = orig[u] = a; v = parent[u];
23     }
24 };
25 // blossom
26 auto augment = [&](int v) {
27     while (v != -1) {
28         int pv=parent[v]; int next_v=match[pv]; match[v]=pv; match[pv]=v;
29         v = next_v;
30     }
31 };
32 // augment
33 auto bfs = [&](int root) {
34     fill(label.begin(), label.end(), -1); iota(orig.begin(), orig.end(), 0);
35     while (!q.empty()) { q.pop(); }
36     q.push(root); label[root] = 0;
37     while (!q.empty()) {
38         int v = q.front(); q.pop();
39         for (int u : adjacency[v]) {
40             if (label[u] == -1) {
41                 label[u] = 1; parent[u] = v;
42                 if (match[u] == -1) { augment(u); return true; }
43                 label[match[u]] = 0; q.push(match[u]); continue;
44             } else if (label[u] == 0 && orig[v] != orig[u]) {
45                 int a=lca(orig[v], orig[u]); blossom(u,v,a); blossom(v,u,a);
46             }
47         }
48     }
49     return false;
50 };
51 // bfs
52 auto greedy = [&]() {
53     vector<int> order(n); iota(order.begin(), order.end(), 0);
54     shuffle(order.begin(), order.end(), rng);
55     for (int i : order) {
56         if (match[i] == -1) {
57             for (int to : adjacency[i]) {
58                 if (match[to] == -1) { match[i] = to; match[to] = i; break; }
59             }
60         }
61     }
62     greedy();
63     for (int i = 0; i < n; i++) { if (match[i] == -1) { bfs(i); } }
64     return match;
65 }
```

2.6 Biconnected Component

Độ phức tạp: $\mathcal{O}(V + E)$

BCC: khớp, cầu và thành phần song liên thông

```
1 struct BiconnectedResult {
2     vector<int> bridges; vector<bool> articulation; vector<vector<int>>
3     edge_blocks;
4 };
5 // Undirected multigraph; IDs are positions in edges. Self-loops form
6 // singleton blocks.
7 BiconnectedResult biconnected_components(int n, const vector<pair<int, int>>&
8     edges) {
9     vector<vector<pair<int, int>>> g(n);
10    BiconnectedResult result { {}, vector<bool>(n), {} };
11    for (int id = 0; id < int(edges.size()); ++id) {
12        auto [u, v] = edges[id];
13        if (u == v) { result.edge_blocks.push_back( {id} ); continue; }
14        g[u].push_back( {v, id} ); g[v].push_back( {u, id} );
15    }
16    vector<int> tin(n, -1), low(n), stack; int timer = 0;
17    auto dfs = [&](auto&& self, int u, int parent_edge)->void {
18        tin[u] = low[u] = timer++; int children = 0;
19        for (auto [v, id] : g[u]) if (id != parent_edge) {
```

```
17         if (tin[v] < 0) {
18             ++children; stack.push_back(id); self(self, v, id); low[u]=min(low[u],
19             low[v]);
20             if (low[v] > tin[u]) result.bridges.push_back(id);
21             if (low[v] >= tin[u]) {
22                 if (parent_edge >= 0) result.articulation[u] = true;
23                 result.edge_blocks.push_back( {} ); int e;
24                 do {
25                     e=stack.back(); stack.pop_back(); result.edge_blocks.back().
26                     push_back(e);
27                 } while (e != id);
28             } else if (tin[v] < tin[u]) { stack.push_back(id); low[u]=min(low[u], tin[v]
29             ); }
30             if (parent_edge < 0) result.articulation[u] = children > 1;
31         };
32         for (int u = 0; u < n; ++u) if (tin[u] < 0) dfs(dfs, u, -1);
33         return result;
34     }
```

2.7 Functional Cycle

Độ phức tạp: $\mathcal{O}(\text{dist} + \text{len})$

Floyd Tortoise and Hare: tìm chu trình trong functional graph $f(x)$

```
1 template<typename State> struct CycleInfo {
2     State entry; size_t prefix_length, cycle_length;
3 };
4 // f must eventually cycle; equality compares complete states. 0(1) extra
5 // space.
6 template<typename State, typename Next> CycleInfo<State> find_cycle(State start,
7     Next f) {
8     State slow = f(start), fast = f(f(start));
9     while (slow != fast) { slow = f(slow); fast = f(f(fast)); }
10    size_t prefix = 0; slow = start;
11    while (slow != fast) { slow = f(slow); fast = f(fast); ++prefix; }
12    size_t length = 1; fast = f(slow);
13    while (slow != fast) { fast = f(fast); ++length; }
14    return {slow, prefix, length};
15 }
```

2.8 Strongly Connected Components

Độ phức tạp: $\mathcal{O}(V + E)$

Tarjan SCC trên đồ thị có hướng

```
1 struct SCCResult { vector<int> component; vector<vector<int>> vertices; };
2 // 0(V+E). IDs assigned sink-first: every condensation edge has ID[from]>ID[to].
3 SCCResult strongly_connected_components(const vector<vector<int>>& g) {
4     int n=g.size(), timer=0; vector<int> number(n,-1), low(n), stack; vector<bool>
5     active(n);
6     SCCResult result { vector<int>(n, -1), {} };
7     auto dfs = [&](auto&& self, int u)->void {
8         number[u] = low[u] = timer++; stack.push_back(u); active[u] = true;
9         for (int v : g[u]) {
10             if (number[v] < 0) {
11                 self(self, v); low[u] = min(low[u], low[v]);
12             } else if (active[v]) low[u] = min(low[u], number[v]);
13         }
14         if (low[u] == number[u]) {
15             result.vertices.push_back( {} ); int id=result.vertices.size()-1;
16             for (;;) {
17                 int v=stack.back(); stack.pop_back(); active[v]=false; result.
18                 component[v]=id;
19                 result.vertices.back().push_back(v);
20                 if (v == u) break;
21             }
22         }
23         for (int u = 0; u < n; ++u) if (number[u] < 0) dfs(dfs, u);
24         return result;
25     }
```

```
24 }
```

2.9 Xóa offline (DSU Rollback)

Độ phức tạp: $\mathcal{O}(T(n) \log n)$

DSU rollback kết hợp D&C trên segment tree

```
1 // No path compression;  $\mathcal{O}(\log n)$  find. Roll back to a snapshot, including no-
  op unions.
2 struct RollbackDSU {
3     vector<int> parent, size; struct Change { int child, root, old_size; };
4     vector<Change> history; int components;
5     explicit RollbackDSU(int n) : parent(n), size(n, 1), components(n) {
6         iota(parent.begin(), parent.end(), 0);
7     }
8     int find(int u) const {
9         while (parent[u] != u) u = parent[u];
10        return u;
11    }
12    size_t snapshot() const { return history.size(); }
13    bool unite(int u, int v) {
14        u = find(u); v = find(v);
15        if (u == v) return false;
16        if (size[u] < size[v]) swap(u, v);
17        history.push_back( {v, u, size[u]}); parent[v] = u; size[u] += size[v]; --
        components;
18        return true;
19    }
20    void rollback(size_t snapshot) {
21        assert(snapshot <= history.size());
22        while (history.size() > snapshot) {
23            auto c = history.back(); history.pop_back(); parent[c.child] = c.child;
24            size[c.root] = c.old_size; ++components;
25        }
26    }
27 };
```

2.10 2-SAT

Độ phức tạp: $\mathcal{O}(n + m)$

Giải 2-SAT bằng đồ thị kéo theo (implication graph) và SCC.

- **Ít nhất một biến đúng** ($X \vee Y$): Thêm các cạnh $\neg X \rightarrow Y$ và $\neg Y \rightarrow X$.

- **Nếu X thì Y** ($X \implies Y$): Thêm các cạnh $X \rightarrow Y$ và $\neg Y \rightarrow \neg X$.

- **Đúng một biến đúng** ($X \text{ XOR } Y = 1$): $(X \vee Y) \wedge (\neg X \vee \neg Y)$. Các cạnh: $X \leftrightarrow \neg Y$ và $\neg X \leftrightarrow Y$.

- **X và Y phải bằng nhau** ($X \text{ XOR } Y = 0$): $(X \vee \neg Y) \wedge (\neg X \vee Y)$. Các cạnh: $X \leftrightarrow Y$ và $\neg X \leftrightarrow \neg Y$.

- **Buộc X đúng/sai**: Thêm cạnh $\neg X \rightarrow X$ (buộc đúng), hoặc $X \rightarrow \neg X$ (buộc sai).

- **Nhiều nhất một biến trong $\{X_1, X_2, \dots, X_k\}$ đúng**: Thêm biến prefix $P_i = P_{i-1} \vee X_i$. Các cạnh: $X_i \rightarrow P_i$, $P_{i-1} \rightarrow P_i$, và $X_i \rightarrow \neg P_{i-1}$ (nếu X_i đúng thì mọi biến trước phải sai).

```
1 // Literal 2*v=true, 2*v+1=false. add_or accepts encoded literals.
2 struct TwoSAT {
3     vector<vector<int>> g;
4     explicit TwoSAT(int variables) : g(2 * variables) {}
5     static int literal(int variable, bool value) { return 2*variable+!value; }
6     void imply(int a, int b) { g[a].push_back(b); g[b^1].push_back(a^1); }
7     void add_or(int a, int b) { imply(a ^ 1, b); }
8     void force(int a) { add_or(a, a); }
9     optional<vector<bool>> solve() const {
10        auto s = strongly_connected_components(g); vector<bool> answer(g.size()/2);
11        for (int v = 0; v < int(answer.size()); ++v) {
12            if (s.component[2 * v] == s.component[2 * v + 1]) return nullopt;
13            answer[v] = s.component[2 * v] < s.component[2 * v + 1];
```

```
14        }
15        return answer;
16    }
17 };
```

2.11 Floyd

Độ phức tạp: $\mathcal{O}(n^3)$

Đường đi ngắn nhất mọi cặp/transitive closure/đường đi min-max

```
1 // Matrix includes diagonal 0, inf for absent edges. Reports whether a
  negative cycle exists.
2 // Distances involving a negative cycle require separate -infinity
  propagation.
3 bool floyd_warshall(vector<vector<long long>>& d, long long inf = 1LL < 60) {
4     int n = d.size();
5     for (int k = 0; k < n; ++k) for (int i = 0; i < n; ++i) if (d[i][k] < inf) for (int j = 0; j < n; ++j)
        if (d[k][j] < inf) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
6     for (int i = 0; i < n; ++i) if (d[i][i] < 0) return true;
7     return false;
8 }
```

2.12 Eulerian Path

Độ phức tạp: $\mathcal{O}(m)$

Hierholzer: tìm đường đi/chu trình Euler.

- **Nối từ/chuỗi (De Bruijn/domino)**: Nối từ sao cho đuôi từ trước khớp đầu từ sau. Định là *trạng thái* (prefix/suffix dài $L - 1$ hoặc nửa domino); cạnh có hướng là *đối tượng*. Đường đi Euler đi qua mỗi đối tượng đúng một lần.

- **Định hướng cạnh vô hướng**: Định hướng cạnh để cân bằng bậc vào/ra (ví dụ $|in - out| \leq 1$). Thêm cạnh giả (hoặc nối đỉnh bậc lẻ tới đỉnh giả) để mọi bậc chẵn; tìm chu trình Euler rồi định hướng theo lượt duyệt.

- **Phân rã chu trình không chung cạnh**: Phân rã thành chu trình không chung cạnh. Khả thi $\iff$ đồ thị liên thông và mọi đỉnh bậc chẵn. Tìm chu trình Euler rồi tách các chu trình.

- **Duyệt lưới bằng đồ thị hàng-cột**: Bài ma trận di chuyển luân phiên hàng/cột: dựng đồ thị hai phía gồm hàng và cột. Ô (r, c) là cạnh nối hàng r với cột c . Đường đi Euler duyệt các ô theo hàng-cột-hàng.

```
1 struct EulerTrail { vector<int> vertices, edge_ids; };
2 // Directed or undirected multigraph, self-loops allowed. nullopt if no
  complete trail.
3 optional<EulerTrail> euler_trail(int n, const vector<pair<int, int>>& edges, bool
  directed = false, optional<int> requested_start = nullopt) {
4     if (n == 0) return edges.empty() ? optional<EulerTrail>(EulerTrail {}) : nullopt;
5     vector<vector<pair<int, int>>> g(n); vector<int> in(n), out(n);
6     for (int id = 0; id < int(edges.size()); ++id) {
7         auto [u, v] = edges[id]; g[u].push_back( {v, id} ); ++out[u]; ++in[v];
8         if (!directed) { g[v].push_back( {u, id} ); ++out[v]; ++in[u]; }
9     }
10    int forced = -1, ends = 0;
11    if (directed) {
12        int sinks = 0;
13        for (int u = 0; u < n; ++u) {
14            if (out[u] - in[u] == 1) {
15                forced = u; ++ends;
16            } else if (in[u] - out[u] == 1) ++sinks;
17            else if (in[u] != out[u]) return nullopt;
18        }
19        if (ends != sinks || ends > 1) return nullopt;
20    } else for (int u = 0; u < n; ++u) if (out[u] & 1) {
21        ++ends; forced = u;
22    }
23    if (!directed && ends != 0 && ends != 2) return nullopt;
```

```
24    int start = requested_start.value_or(forced >= 0 ? forced : 0);
25    if (!requested_start && forced < 0) for (int u = 0; u < n; ++u) if (out[u] {
26        start = u; break;
27    }
28    if (start < 0 || start >= n) return nullopt;
29    if (directed && forced >= 0 && start != forced) return nullopt;
30    if (!directed && ends && !(out[start] & 1)) return nullopt;
31    vector<int> next(n); vector<bool> used(edges.size());
32    vector<pair<int, int>> stack { {start, -1} }; EulerTrail answer;
33    while (!stack.empty()) {
34        int u = stack.back().first;
35        while (next[u] < int(g[u].size()) && used[g[u][next[u]].second]) ++next[u];
36        if (next[u] == int(g[u].size())) {
37            auto [v, id] = stack.back(); stack.pop_back(); answer.vertices.push_back(v
            );
38            if (id >= 0) answer.edge_ids.push_back(id);
39            else { auto [v, id] = g[u][next[u]++]; used[id] = true; stack.push_back( {v,
            id} ); }
40        }
41        if (answer.edge_ids.size() != edges.size()) return nullopt;
42        reverse(answer.vertices.begin(), answer.vertices.end());
43        reverse(answer.edge_ids.begin(), answer.edge_ids.end()); return answer;
44    }
```

2.13 Định lý Vizing

Độ phức tạp: $\mathcal{O}(nm)$ thời gian, $\mathcal{O}(n^2)$ bộ nhớ

Tổ cạnh đồ thị đơn bằng tối đa $\Delta + 1$ màu

```
1 // Simple undirected graph; returns edge colors 0..Delta.  $\mathcal{O}(n^2)$  storage.
2 vector<int> vizing_coloring(int N, const vector<pair<int, int>>& edges) {
3     vector<pair<int, int>> E;
4     for (auto [u, v] : edges) { assert(u != v); E.push_back( {u+1, v+1} ); }
5     vector<vector<int>> C(N+1, vector<int>(N+2)), G(N+1, vector<int>(N+1));
6     vector<int> X(N+1); int a;
7     auto update = [&](int u) {
8         for (X[u] = 1; C[u][X[u]]; X[u]++);
9     };
10    auto color = [&](int u, int v, int c) {
11        int p = G[u][v];
12        if (p == c) return p;
13        G[u][v] = G[v][u] = c; C[u][c] = v; C[v][c] = u; C[u][p] = C[v][p] = 0;
14        if (p) X[u] = X[v] = p;
15        else update(u), update(v);
16        return p;
17    };
18    // end of function : color
19    auto flip = [&](int u, int c1, int c2) {
20        int p = C[u][c1]; swap(C[u][c1], C[u][c2]);
21        if (p) G[u][p] = G[p][u] = c2;
22        if (!C[u][c1]) X[u] = c1;
23        if (!C[u][c2]) X[u] = c2;
24        return p;
25    };
26    // end of function : flip
27    for (int i = 1; i <= N; i++) X[i] = 1;
28    for (int t = 0; t < (int) E.size(); t++) {
29        int u = E[t].first, v = E[t].second, v0 = E[t].c0 = X[u], c = c0, d; vector<pair<int, int>
        >> L;
30        vector<int> vst(N+2);
31        while (!G[u][v0]) {
32            L.emplace_back(v, d = X[v]);
33            if (!C[v][c]) for (a = (int) L.size() - 1; a >= 0; a--) c = color(u, L[a].first, c);
34            else if (!C[u][d]) for (a = (int) L.size() - 1; a >= 0; a--) color(u, L[a].first, L[a
        ].second);
35            else if (vst[d]) break;
36            else vst[d] = 1, v = C[u][d];
37        }
38        if (!G[u][v0]) {
39            for (; v; v = flip(v, c, d), swap(c, d));
40            if (C[u][c0]) {
41                for (a = (int) L.size() - 2; a >= 0 && L[a].second != c; a--);
42                for (c; a >= 0; a--) color(u, L[a].first, L[a].second);
43            } else t--;
44        }
45    }
```

```

46     vector<int> result;
47     for (auto [u, v] : E) result.push_back(G[u][v] - 1);
48     return result;
49 }

```

2.14 Kiểm tra hai phía online

Độ phức tạp: $\mathcal{O}(\alpha(n))$

Kiểm tra hai phía online khi thêm cạnh bằng parity DSU

```

1 // Append undirected edges; is_bipartite(u) tests the current component.
2 struct BipartiteDSU {
3     vector<int> parent, size, parity; vector<bool> good;
4     pair<int, int> find(int u) {
5         if (parent[u] == u) return {u, 0};
6         auto [root, p] = find(parent[u]); parity[u] ^= p; parent[u] = root;
7         return {root, parity[u]};
8     }
9     explicit BipartiteDSU(int n):parent(n),size(n,1),parity(n),good(n,true){
10         iota(parent.begin(), parent.end(), 0);
11     }
12     bool add_edge(int u, int v) {
13         auto [a, x] = find(u); auto [b, y] = find(v);
14         if (a == b) {
15             if (x == y) good[a] = false;
16             return good[a];
17         }
18         if (size[a] < size[b]) { swap(a, b); swap(x, y); }
19         parent[b]=a; parity[b]=x^y^1; size[a]+=size[b]; good[a]=good[a]&&good[b];
20         return good[a];
21     }
22     bool is_bipartite(int u) { return good[find(u).first]; }
23 };

```

2.15 Gomory-Hu Tree

Độ phức tạp: $\mathcal{O}(n \cdot \text{MaxFlow})$

Cây cut tương đương cho max-flow/min-cut mọi cặp

```

1 struct CutTreeEdge { int u, v; long long capacity; };
2 // Undirected nonnegative capacities; min edge on tree path gives pairwise
   min-cut.
3 vector<CutTreeEdge> gomory_hu(int n, const vector<CutTreeEdge>& edges){
4     vector<int> parent(n, 0); vector<long long> value(n);
5     for (int s = 1; s < n; ++s) {
6         int t = parent[s]; Dinic flow(n);
7         for (auto e : edges) {
8             flow.add_edge(e.u, e.v, e.capacity); flow.add_edge(e.v, e.u, e.capacity);
9         }
10        long long cut=flow.max_flow(s,t); auto side=flow.source_side(s);
11        for(int i=s+1; i<n; ++i)if(parent[i]==t&&side[i])parent[i]=s;
12        if (t != 0 && side[parent[t]]) {
13            parent[s]=parent[t]; parent[t]=s; value[s]=value[t]; value[t]=cut;
14        } else value[s] = cut;
15    }
16    vector<CutTreeEdge> tree;
17    for (int u = 1; u < n; ++u) tree.push_back( {u, parent[u], value[u]});
18    return tree;
19 }

```

2.16 Critical (Dominator Tree)

Độ phức tạp: $\mathcal{O}(m \log n)$

Lengauer-Tarjan: immediate dominator từ nguồn s

```

1 // Immediate dominators from source; source dominates itself; unreachable=-1.
2 // Lengauer-Tarjan; recursive eval compression,  $\mathcal{O}((n+m) \log n)$  bound.
3 vector<int>immediate_dominators(const vector<vector<int>>&g,int source){
4     int n=g.size(); vector<int>index(n),vertex(1,-1),parent(1),order {source};
5     struct Frame { int u,next; }; vector<Frame>stack { {source,0} }; index[
   source]=1;

```

```

6     vertex.push_back(source); parent.push_back(0);
7     while (!stack.empty()) {
8         auto& f = stack.back();
9         if (f.next == int(g[f.u].size())) { stack.pop_back(); continue; }
10        int v = g[f.u][f.next++];
11        if (index[v]) continue;
12        int p=index[f.u]; index[v]=vertex.size(); vertex.push_back(v); parent.
   push_back(p);
13        stack.push_back( {v, 0});
14    }
15    int count = vertex.size() - 1;
16    vector<int>semi(count+1),label(count+1),ancestor(count+1),idom(count+1);
17    iota(semi.begin(),semi.end(),0); iota(label.begin(),label.end(),0);
18    vector<vector<int>> pred(count + 1), bucket(count + 1);
19    for(int u=0;u<n;++u)if(index[u])for(int v:g[u])if(index[v])pred[index[v]].
   push_back(index[u]);
20    auto compress = [&](auto&& self, int v)->void {
21        if (ancestor[ancestor[v]]) {
22            self(self, ancestor[v]);
23            if (semi[label[ancestor[v]]]<semi[label[v]])label[v]=label[ancestor[v]];
24            ancestor[v] = ancestor[ancestor[v]];
25        }
26    };
27    auto eval = [&](int v) {
28        if (!ancestor[v]) return label[v];
29        compress(compress, v); return label[v];
30    };
31    for (int w = count; w > 1; --w) {
32        for (int v : pred[w]) semi[w] = min(semi[w], semi[eval(v)]);
33        bucket[semi[w]].push_back(w); ancestor[w] = parent[w];
34        for(int v:bucket[parent[w]]){ int u=eval(v); idom[v]=semi[u]<semi[v]?u:
   parent[w]; }
35        bucket[parent[w]].clear();
36    }
37    for(int w=2; w<=count; ++w)if(idom[w]!=semi[w])idom[w]=idom[idom[w]];
38    vector<int> answer(n, -1); answer[source] = source;
39    for (int w = 2; w <= count; ++w) answer[vertex[w]] = vertex[idom[w]];
40    return answer;
41 }

```

2.17 Prufer

Độ phức tạp: $\mathcal{O}(n \log n)$

Mã hóa/giải mã cây có nhãn gồm n đỉnh

```

1 // Labels 0..n-1; tree n==2. 0(n log n); decode uses n=code.size()+2.
2 vector<int> prufer_encode(const vector<vector<int>>& g) {
3     int n = g.size(); assert(n >= 2); vector<int> degree(n);
4     priority_queue<int, vector<int>, greater<int>> leaves;
5     for(int u=0; u<n; ++u)if((degree[u]=g[u].size()==1)leaves.push(u);
6     vector<int> code;
7     for (int i = 0; i < n - 2; ++i) {
8         int u = leaves.top(); leaves.pop(); degree[u] = 0;
9         for (int v : g[u]) if (degree[v]) {
10             code.push_back(v);
11             if (--degree[v] == 1) leaves.push(v);
12             break;
13         }
14     }
15     return code;
16 }
17 vector<pair<int, int>> prufer_decode(const vector<int>& code) {
18     int n = code.size() + 2; vector<int> degree(n, 1);
19     for (int u : code) { assert(0 <= u && u < n); ++degree[u]; }
20     priority_queue<int, vector<int>, greater<int>> leaves;
21     for (int u = 0; u < n; ++u) if (degree[u] == 1) leaves.push(u);
22     vector<pair<int, int>> edges;
23     for (int u : code) {
24         int v = leaves.top(); leaves.pop(); edges.push_back( {u, v});
25         if (--degree[u] == 1) leaves.push(u);
26     }
27     int a=leaves.top(); leaves.pop(); edges.push_back( {a,leaves.top()});
28     return edges;

```

2.18 Steiner Tree

Độ phức tạp: $\mathcal{O}(n \cdot 3^k + m \cdot 2^k \log m)$

Steiner tree nhỏ nhất nối k đỉnh terminal

```

1 // Nonnegative undirected weights; at most 20 terminals (practically <=12).
2 // Returns nullptr when disconnected, zero for no terminals.
3 optional<long long>steiner_tree(const vector<vector<pair<int,long long>>&g,
   const vector<int>&terminals){
4     int n = g.size(), k = terminals.size();
5     if (!k) return 0;
6     assert(k <= 20); constexpr long long inf = 1LL << 60;
7     vector<vector<long long>> dp(1 << k, vector<long long>(n, inf));
8     for (int i = 0; i < k; ++i) dp[1 << i][terminals[i]] = 0;
9     for (int mask = 1; mask < (1 << k); ++mask) {
10        for(int sub=(mask-1)&mask;sub=(sub-1)&mask;if(sub<(mask^sub))for(int
   u=0;u<n;++u)dp[mask][u]=min(dp[mask][u],dp[sub][u]+dp[mask^sub][u]);
11        priority_queue<pair<long long,int>,vector<pair<long long,int>>,greater<
   pair<long long,int>>>q;
12        for(int u=0; u<n; ++u)if(dp[mask][u]<inf)q.push( {dp[mask][u],u});
13        while (!q.empty()) {
14            auto [d, u] = q.top(); q.pop();
15            if (d != dp[mask][u]) continue;
16            for (auto [v, w] : g[u]) if (d + w < dp[mask][v]) {
17                dp[mask][v] = d + w; q.push( {d + w, v});
18            }
19        }
20    }
21    long long answer = *min_element(dp.back().begin(), dp.back().end());
22    return answer == inf ? nullptr : optional<long long>(answer);
23 }

```

2.19 Stable Marriage

Độ phức tạp: $\mathcal{O}(n^2)$

Gale-Shapley giải bài ghép đôi ổn định (Stable Marriage)

```

1 // Complete strict preference permutations, equal side sizes; proposer-
   optimal.
2 vector<int>stable_marriage(const vector<vector<int>>&men,const vector<vector<
   int>>&women){
3     int n = men.size(); assert(women.size() == men.size());
4     vector<vector<int>> rank(n, vector<int>(n));
5     for(int w=0; w<n; ++w)for(int i=0; i<n; ++i)rank[w][women[w][i]]=i;
6     vector<int> husband(n, -1), wife(n, -1), next(n); queue<int> free;
7     for (int m = 0; m < n; ++m) free.push(m);
8     while (!free.empty()) {
9         int m = free.front(); free.pop(); int w = men[m][next[m]++];
10        if (husband[w] < 0 || rank[w][m] < rank[w][husband[w]]) {
11            if(husband[w]>0){ wife[husband[w]]=-1; free.push(husband[w]); }
12            husband[w] = m; wife[m] = w;
13        } else free.push(m);
14    }
15    return wife;
16 }

```

3 Cây

3.1 Tree Isomorphism

Độ phức tạp: $\mathcal{O}(n \log n)$

Nhân chuẩn hóa chính xác từ các con; so sánh cây không gốc tại tâm

```

1 // Shared interning dictionary gives exact rooted IDs (not randomized hashes)
2 struct TreeIsomorphism {
3     map<vector<int>, int> ids;
4     vector<int> centers(const vector<vector<int>>& g) const {
5         int n = g.size();
6         if (n < 2) return n ? vector<int> {0}
7             :vector<int> {}; vector<int> degree(n), leaves;

```

```

8   for(int u=0; u<n; ++u)if((degree[u]=g[u].size())<=1)leaves.push_back(u);
9   int remaining = n;
10  while (remaining > 2) {
11      remaining -= leaves.size(); vector<int> next;
12      for(int u:leaves)for(int v:g[u])if(--degree[v]==1)next.push_back(v);
13      leaves.swap(next);
14  }
15  return leaves;
16 }
17 int rooted_id(const vector<vector<int>>& g, int root) {
18     vector<int> parent(g.size(), -1), order {root}, id(g.size());
19     for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(v!=parent[
20         order[i]]){
21         parent[v] = order[i]; order.push_back(v);
22     }
23     for (auto it = order.rbegin(); it != order.rend(); ++it) {
24         int u = *it; vector<int> children;
25         for (int v : g[u]) if (v != parent[u]) children.push_back(id[v]);
26         sort(children.begin(), children.end());
27         auto [pos,inserted]=ids.emplace(children,ids.size()+1); id[u]=pos->
28         second;
29     }
30     return id[root];
31 }
32 bool isomorphic(const vector<vector<int>>&a,const vector<vector<int>>&b){
33     if (a.size() != b.size()) return false;
34     if (a.empty()) return true;
35     auto ca = centers(a), cb = centers(b);
36     if (ca.size() != cb.size()) return false;
37     int id = rooted_id(a, ca[0]);
38     for (int c : cb) if (rooted_id(b, c) == id) return true;
39     return false;
40 }

```

3.2 Centroid Decomposition

Độ phức tạp: $\mathcal{O}(n \log n)$

Phân rã cây thành hệ phân cấp sâu $\mathcal{O}(\log n)$ để truy vấn đường đi bằng D&C

```

1  // Builds centroid parent forest for a tree; original adjacency is unchanged.
2  // on_centroid(c,parent,blocked) may process the current component.
3  struct CentroidDecomposition {
4      vector<int> parent;
5      template<typename Process>CentroidDecomposition(const vector<vector<int>>&g
6          ,Process on_centroid):parent(g.size(),-1){
7          int n = g.size();
8          if (!n) return;
9          vector<bool> blocked(n); vector<int> p(n), size(n);
10         vector<pair<int, int>> tasks { {0, -1} };
11         while (!tasks.empty()) {
12             auto [entry,ancestor]=tasks.back(); tasks.pop_back(); vector<int>order
13             {entry};
14             p[entry] = -1;
15             for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(!blocked[v
16                 ]&&v!=p[order[i]]){
17                 p[v] = order[i]; order.push_back(v);
18             }
19             int total = order.size(), c = entry;
20             for (auto it = order.rbegin(); it != order.rend(); ++it) {
21                 int u = *it; size[u] = 1; int largest = 0;
22                 for (int v : g[u]) if (!blocked[v] && p[v] == u) {
23                     size[u] += size[v]; largest = max(largest, size[v]);
24                 }
25                 if (max(largest, total - size[u]) <= total / 2) c = u;
26             }
27             parent[c]=ancestor; blocked[c]=true; on_centroid(c,ancestor,blocked);
28             for (int v : g[c]) if (!blocked[v]) tasks.push_back( {v, c});
29         }
30     }
31 };

```

3.3 Heavy-Light Decomposition

Độ phức tạp: $\mathcal{O}(n \log^2 n)$

Chia cây thành heavy chain: truy vấn/cập nhật đường đi $\mathcal{O}(\log^2 n)$, cây con $\mathcal{O}(\log n)$

```

1  // Zero-based positions; segments [l,r). rev=true means traverse r-1 down to
2  // l. path_segments returns segments in u-to-v order, supports noncommutative
3  // folds.
4  struct HeavyLight {
5      struct Segment { int l, r; bool reversed; };
6      vector<int> parent, depth, size, heavy, head, pos, vertex;
7      HeavyLight(const vector<vector<int>>&g,int root=0):parent(g.size(),-1),
8          depth(g.size(),size(g.size(),1),heavy(g.size(),-1),head(g.size(),pos(g
9              .size(),vertex(g.size())){
10         if (g.empty()) return;
11         vector<int> order {root};
12         for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(v!=parent[
13             order[i]]){
14             parent[v]=order[i]; depth[v]=depth[order[i]]+1; order.push_back(v);
15         }
16         for(auto it=order.rbegin();it!=order.rend();++it)for(int v:g[*it])if(
17             parent[v]==*it){
18             size[*it] += size[v];
19             if (heavy[*it] < 0 || size[v] > size[heavy[*it]]) heavy[*it] = v;
20         }
21         vector<pair<int, int>> tasks { {root, root} }; int timer = 0;
22         while (!tasks.empty()) {
23             auto [u, h] = tasks.back(); tasks.pop_back();
24             for (; u != -1; u = heavy[u]) {
25                 head[u] = h; pos[u] = timer; vertex[timer++] = u;
26                 for(int v:g[u])if(parent[v]==u&&v!=heavy[u])tasks.push_back( {v,v});
27             }
28         }
29     }
30     int lca(int u, int v) const {
31         while (head[u] != head[v]) {
32             if (depth[head[u]] < depth[head[v]]) swap(u, v);
33             u = parent[head[u]];
34         }
35         return depth[u] < depth[v] ? u : v;
36     }
37     pair<int,int>subtree(int u)const { return {pos[u],pos[u]+size[u]}; }
38     vector<Segment>path_segments(int u,int v,bool edges=false)const {
39         vector<Segment> left, right;
40         while (head[u] != head[v]) {
41             if (depth[head[u]] >= depth[head[v]]) {
42                 left.push_back( {pos[head[u]],pos[u]+1,true}); u=parent[head[u]];
43             }else { right.push_back( {pos[head[v]],pos[v]+1,false}); v=parent[head[
44                 v]]; }
45         }
46         if (depth[u] >= depth[v]) {
47             if (pos[v]+edges<pos[u]+1)left.push_back( {pos[v]+edges,pos[u]+1,true});
48         } else if (pos[u]+edges<pos[v]+1)right.push_back( {pos[u]+edges,pos[v]+1,
49             false});
50         reverse(right.begin(), right.end());
51         left.insert(left.end(), right.begin(), right.end()); return left;
52     }
53 };

```

4 Chuỗi

4.1 Chiến lược giải bài chuỗi

Phân loại bài chuỗi, chu kỳ, LCP, substring phân biệt, palindrome và định lý chính

• **Problem Taxonomy & Core Approches:**

- LCP với toàn chuỗi/tìm một mẫu: Dùng Z-Algorithm hoặc KMP.
- Tìm nhiều mẫu/DP trên tập chuỗi: Dùng Aho-Corasick Automaton.

– Substring phân biệt: Suffix Array ($\frac{n(n+1)}{2} - \sum LCP_i$) hoặc SAM ($\sum len[u] - len[link[u]]$).

– Substring thứ k /LCS của nhiều chuỗi: Dùng Suffix Automaton (SAM) hoặc Suffix Array + LCP.

– Substring đối xứng (palindrome): Dùng Manacher (bán kính tại mỗi vị trí) hoặc EERTREE / PAM (cây palindrome).

– Phép xoay chuỗi nhỏ nhất theo thứ tự từ điển: Chạy Lyndon Factorization (Duval) trên $S + S$.

– So sánh substring $\mathcal{O}(1)$ /chuỗi 2D: Dùng Rolling Hash ($2^{61} - 1$ số nguyên tố Mersenne).

• **Chu kỳ và tính chất border:**

– Chu kỳ nhỏ nhất của chuỗi dài n là $p = n - \pi[n - 1]$ (nếu $n \bmod p == 0 \implies S = (S[0..p - 1])^{n/p}$).

– Fine-Wilf (bổ đề chu kỳ): Nếu S có chu kỳ p, q và $|S| \geq p + q - \gcd(p, q) \implies S$ có chu kỳ $\gcd(p, q)$.

– Dãy border: Mọi border chia thành $\leq \mathcal{O}(\log n)$ cấp số cộng $\implies$ Tối ưu DP border xuống $\mathcal{O}(n \log n)$.

• **Cận lý thuyết:**

– Số palindrome phân biệt trong chuỗi dài n luôn $\leq n$.

– Suffix Automaton của chuỗi dài n có $\leq 2n - 1$ trạng thái và $\leq 3n - 4$ cạnh chuyển.

4.2 Rolling Hash (61-bit Mersenne)

Độ phức tạp: $\mathcal{O}(n)$ dựng, $\mathcal{O}(1)$ truy vấn substring

Hash kép/Mersenne 61-bit ($2^{61} - 1$), base chung do người gọi chọn (so sánh có xác suất lỗi)

```

1  struct RollingHash {
2      using U=uint64_t; static constexpr U MOD=(U(1)<<61)-1; vector<U>h,p;
3      static U mul(U a, U b) {
4          __uint128_t x=(__uint128_t)a*b; U v=(U)(x&MOD)+(U)(x>>61); return v%=MOD?
5              v:MOD;
6      }
7      // Use the same base for hashes to be compared; equality is probabilistic.
8      Randomize the shared base for adversarial inputs.
9      explicit RollingHash(string_view s,U base=911382323):h(s.size()+1),p(s.size
10          ()+1){
11          assert(base > 256 && base < MOD);
12          for (size_t i = 0; i < s.size(); ++i) {
13              h[i+1]=(mul(h[i],base)+(unsigned char)s[i]+1)%MOD; p[i+1]=mul(p[i],base
14                  );
15          }
16      }
17      U get(int l, int r) const {
18          assert(0<=l&&l<=r&&r<(int)h.size()); return (h[r]+MOD-mul(h[l],p[r-l]))%
19              MOD;
20      }
21 };

```

4.3 KMP

Độ phức tạp: $O(n + m)$

Prefix function π và tìm mẫu tuyến tính

```
1 vector<int> prefix_function(string_view s) {
2     vector<int> p(s.size());
3     for (int i = 1; i < (int) s.size(); ++i) {
4         int j = p[i - 1];
5         while (j && s[i] != s[j]) j = p[j - 1];
6         if (s[i] == s[j]) ++j;
7         p[i] = j;
8     }
9     return p;
10 }
11 vector<int> kmp_matches(string_view text, string_view pattern) {
12     vector<int> out;
13     if (pattern.empty()) {
14         for (int i = 0; i <= (int) text.size(); ++i) out.push_back(i);
15         return out;
16     }
17     auto p = prefix_function(pattern); int j = 0;
18     for (int i = 0; i < (int) text.size(); ++i) {
19         while (j && text[i] != pattern[j]) j = p[j - 1];
20         if (text[i] == pattern[j]) ++j;
21         if (j==(int)pattern.size()){ out.push_back(i-j+1); j=p[j-1]; }
22     }
23     return out;
24 }
```

4.4 Z-Algorithm

Độ phức tạp: $O(n + m)$

Z-array: $z[i] = \text{LCP}(S, S[i..n - 1])$ và tìm mẫu qua $S = P + \# + T$

```
1 vector<int> z_function(string_view s) {
2     int n = s.size(); vector<int> z(n);
3     for (int i = 1, l = 0, r = 0; i < n; ++i) {
4         if (i < r) z[i] = min(r - i, z[i - l]);
5         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
6         if (i + z[i] > r) l = i, r = i + z[i];
7     }
8     if (n) z[0] = n;
9     return z;
10 }
```

4.5 Aho Corasick

Độ phức tạp: $O(26 \sum |P_i| + |T| + \text{matches})$

Tìm nhiều mẫu trên trie, dựng failure/exit link bằng BFS

```
1 struct AhoCorasick {
2     struct Node {
3         array<int, 26> next; int link = 0, exit = -1; vector<int> ids;
4         Node() { next.fill(-1); }
5     }; vector<Node> t {}; bool built = false;
6     // Lowercase, nonempty patterns; IDs may repeat across different patterns.
7     void add(string_view s, int id) {
8         assert(!built && !s.empty()); int u = 0;
9         for (char c : s) {
10             assert('a' <= c && c <= 'z'); int x = c - 'a';
11             if (t[u].next[x] < 0) { t[u].next[x] = t.size(); t.emplace_back(); }
12             u = t[u].next[x];
13         }
14         t[u].ids.push_back(id);
15     }
16     void build() {
17         assert(!built); queue<int> q;
18         for (int c = 0; c < 26; ++c) {
19             int v = t[0].next[c];
20             if (v < 0) t[0].next[c] = 0;
21             else q.push(v);
22         }
23         while (!q.empty()) {
```

```
24         int u=q.front(); q.pop(); int f=t[u].link; t[u].exit=t[f].ids.empty()?t
[f].exit:f;
25         for (int c = 0; c < 26; ++c) {
26             int v = t[u].next[c];
27             if (v < 0) t[u].next[c] = t[f].next[c];
28             else { t[v].link = t[f].next[c]; q.push(v); }
29         }
30         built = true;
31     }
32 }
33 // emit(end_exclusive, pattern_id); includes overlaps and duplicate
patterns.
34 template<typename Emit> void scan(string_view s, Emit emit) const {
35     assert(built); int u = 0;
36     for (int i = 0; i < (int) s.size(); ++i) {
37         assert('a' <= s[i] && s[i] <= 'z'); u = t[u].next[s[i] - 'a'];
38         for(int v=u; v>0; v=t[v].exit)for(int id:t[v].ids)emit(i+1,id);
39     }
40 }
41 };
```

4.6 Suffix Array

Độ phức tạp: $O(n \log n)$ dựng, $O(n)$ LCP

Dựng suffix array bằng doubling, tính LCP bằng Kasai

```
1 struct SuffixArray {
2     vector<int> sa, rank, lcp;
3     explicit SuffixArray(string_view s) {
4         int n=s.size(), N=n+1; vector<int>a(N), c(N), p(N), cnt(max(N, 257));
5         for (int i = 0; i < n; ++i) a[i] = (unsigned char) s[i] + 1;
6         for (int x : a) ++cnt[x];
7         for (int i = 1; i < (int) cnt.size(); ++i) cnt[i] += cnt[i - 1];
8         for (int i = N - 1; i >= 0; --i) p[--cnt[a[i]]] = i;
9         int classes = 1;
10        for (int i = 1; i < N; ++i) {
11            if (a[p[i]] != a[p[i - 1]]) ++classes;
12            c[p[i]] = classes - 1;
13        }
14        for (int k = 1; k < N; k *= 2) {
15            vector<int> q(N), d(N); fill(cnt.begin(), cnt.end(), 0);
16            for(int i=0; i<N; ++i){ q[i]=(p[i]-k+N)%N; ++cnt[c[q[i]]]; }
17            for (int i = 1; i < classes; ++i) cnt[i] += cnt[i - 1];
18            for (int i = N - 1; i >= 0; --i) p[--cnt[c[q[i]]]] = q[i];
19            int nc = 1;
20            for (int i = 1; i < N; ++i) {
21                if (pair {c[p[i]], c[p[i] + k] % N})
22                    != pair {c[p[i - 1]], c[p[i - 1] + k] % N}) ++nc;
23                d[p[i]] = nc - 1;
24            }
25            c.swap(d); classes = nc;
26            if (k > N / 2) break;
27        }
28        sa.assign(p.begin() + 1, p.end()); rank.resize(n); lcp.resize(n);
29        for (int i = 0; i < n; ++i) rank[sa[i]] = i;
30        for (int i = 0, h = 0; i < n; ++i) {
31            int r = rank[i];
32            if (!r) { h = 0; continue; }
33            int j = sa[r - 1];
34            while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
35            lcp[r] = h;
36            if (h) --h;
37        }
38    }
39 };
```

• Ứng dụng LCP trong thi đấu:

- LCP của hai suffix bất kỳ: $\text{LCP}(S[i..], S[j..]) = \min_{\max(pos[i], pos[j])}^{\min(pos[i], pos[j])+1} \text{LCP}[k]$ qua RMQ/Sparse Table, truy vấn $O(1)$.
- Substring phân biệt: Tổng substring phân biệt là $\frac{n(n+1)}{2} - \sum_{i=1}^{n-1} \text{LCP}[i]$.

- Substring thứ k theo thứ tự từ điển: Suffix $SA[i]$ đóng góp $n - SA[i] - \text{LCP}[i]$ chuỗi mới, có độ dài $\text{LCP}[i] + 1 \dots n - SA[i]$.
- LCS của hai chuỗi: Dựng SA trên $S_1 + \# + S_2$; đáp án là $\max \text{LCP}[i]$ với $SA[i], SA[i - 1]$ thuộc hai chuỗi khác nhau.
- LCS của K chuỗi: Dựng SA trên $S_1 + \#_1 + \dots + S_K + \#_K$; two pointers/sliding window $[L, R]$ chứa đủ K chuỗi $\implies \min_{k=L+1}^R \text{LCP}[k]$.
- Khoảng xuất hiện mẫu: Các suffix có prefix P tạo đoạn liên tiếp $[L, R]$ trong SA; tìm bằng hai binary search trong $O(|P| \log n)$, số lần $= R - L + 1$.
- Substring dài nhất xuất hiện $\geq K$ lần: $\max_i (\min_{j=i+1}^{i+K-1} \text{LCP}[j])$ bằng monotonic queue/sliding RMQ trong $O(n)$.

4.7 Suffix Automaton

Độ phức tạp: $O(n \log |\Sigma|)$ dựng

DAWG tối tiểu, mỗi đường đi từ gốc ứng với một substring của S

```
1 struct SuffixAutomaton {
2     struct State {
3         int length=0, link=-1; map<unsigned char, int> next; long long ends=0;
4     }; vector<State> states {}; int last = 0;
5     void extend(unsigned char c) {
6         int cur = states.size(); states.emplace_back();
7         states[cur].length=states[last].length+1; states[cur].ends=1; int p=last;
8         while (p>=0 && !states[p].next.count(c)) { states[p].next[c]=cur; p=states[p
].link; }
9         if (p < 0) states[cur].link = 0;
10        else {
11            int q = states[p].next[c];
12            if (states[p].length+1==states[q].length) states[cur].link=q;
13            else {
14                int clone = states.size(); states.push_back(states[q]);
15                states[clone].length=states[p].length+1; states[clone].ends=0;
16                while (p>=0 && states[p].next[c]==q) { states[p].next[c]=clone; p=states[
p].link; }
17                states[q].link = states[cur].link = clone;
18            }
19        }
20        last = cur;
21    }
22    explicit SuffixAutomaton(string_view s = "") {
23        for (unsigned char c : s) extend(c);
24    }
25    bool contains(string_view s) const {
26        int u = 0;
27        for (unsigned char c : s) {
28            auto it = states[u].next.find(c);
29            if (it == states[u].next.end()) return false;
30            u = it->second;
31        }
32        return true;
33    }
34    long long distinct_substrings() const {
35        long long ans = 0;
36        for (int i=1; i<(int)states.size(); ++i) ans+=states[i].length-states[states[
i].link].length;
37        return ans;
38    }
39    // Returned counts do not mutate the automaton; may be recomputed after
extensions.
40    vector<long long> occurrences() const {
41        vector<int> order(states.size()); iota(order.begin(), order.end(), 0);
42        sort(order.begin(), order.end(), [&](int a, int b) {
43            return states[a].length > states[b].length;
44        }); vector<long long> cnt;
45        for (auto& s : states) cnt.push_back(s.ends);
46        for (int u:order) if (states[u].link>=0) cnt[states[u].link]+=cnt[u];
47        return cnt;
48    }
```

```
48 }
49 };
```

4.8 Lyndon Factorization

Độ phức tạp: $O(n)$ thời gian; các đoạn trả về/chuỗi nhân đôi dùng $O(n)$ bộ nhớ

Duval: phân tích chuỗi thành từ Lyndon $s_1 \geq s_2 \geq \dots \geq s_k$ và tìm phép xoay nhỏ nhất

```
1 vector<pair<int, int>> lyndon_factors(string_view s) {
2     vector<pair<int, int>> out; int n = s.size();
3     for (int i = 0; i < n; i++) {
4         int j = i + 1, k = i;
5         while (j < n && (unsigned char) s[k] <= (unsigned char) s[j]) {
6             k = (unsigned char) s[k] < (unsigned char) s[j] ? i : k + 1; ++j;
7         }
8         while (i <= k) { out.push_back( {i, i + j - k}); i += j - k; }
9     }
10    return out;
11 }
12 int minimum_rotation(string_view s) {
13     if (s.empty()) return 0;
14     std::string t=std::string(s)+std::string(s); int n=s.size(),ans=0;
15     for (int i = 0; i < n; i++) {
16         ans = i; int j = i + 1, k = i;
17         while (j < 2 * n && (unsigned char) t[k] <= (unsigned char) t[j]) {
18             k = (unsigned char) t[k] < (unsigned char) t[j] ? i : k + 1; ++j;
19         }
20         while (i <= k) i += j - k;
21     }
22     return ans;
23 }
```

4.9 Manacher

Độ phức tạp: $O(n)$

Tìm bán kính palindrome lớn nhất tại mỗi vị trí

```
1 struct PalindromeRadii { vector<int> odd, even; };
2 PalindromeRadii manacher(string_view s) {
3     int n=s.size(); PalindromeRadii out {vector<int>(n),vector<int>(n)};
4     for (int i = 0, l = 0, r = -1; i < n; ++i) {
5         int k = i > r ? 1 : min(out.odd[l + r - i], r - i + 1);
6         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
7         out.odd[i] = k--;
8         if (i + k > r) l = i - k, r = i + k;
9     }
10    for (int i = 0, l = 0, r = -1; i < n; ++i) {
11        int k = i > r ? 0 : min(out.even[l + r - i + 1], r - i + 1);
12        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
13        out.even[i] = k--;
14        if (i + k > r) l = i - k - 1, r = i + k;
15    }
16    return out;
17 }
```

4.10 Palindrome Tree

Độ phức tạp: $O(n)$

EERTREE: biểu diễn mọi palindrome phân biệt và các link chuyển

```
1 struct PalindromeTree {
2     struct Node {
3         int length, link; map<unsigned char, int> next; long long ends = 0;
4     }; vector<Node> nodes { {-1,0,{},0 } { 0,0,{},0 } }; std::string text; int
5     last=1;
6     int append(unsigned char c) {
7         text += char(c); int i = text.size() - 1, u = last;
8         auto fits = [&](int v) {
9             int j=i-1-nodes[v].length; return j>=0&&(unsigned char)text[j]==c;
```

```
9 };
10 while (!fits(u)) u = nodes[u].link;
11 auto it = nodes[u].next.find(c);
12 if (it != nodes[u].next.end()) last = it->second;
13 else {
14     int v=nodes.size(),length=nodes[u].length+2; nodes.push_back( { length
15     ,1,{},0 });
16     nodes[u].next[c] = v;
17     if (length > 1) {
18         int p = nodes[u].link;
19         while (!fits(p)) p = nodes[p].link;
20         nodes[v].link = nodes[p].next.at(c);
21     }
22     last = v;
23 }
24 ++nodes[last].ends; return last;
25 }
26 vector<long long> occurrences() const {
27     vector<long long> cnt;
28     for (auto& v : nodes) cnt.push_back(v.ends);
29     for(int i=(int)nodes.size()-1; i>=2; --i)cnt[nodes[i].link]+=cnt[i];
30     return cnt;
31 };
```

4.11 De Bruijn (FKM)

Độ phức tạp: $O(k^n)$ thời gian, $O(n)$ bộ nhớ phụ; $O(1)$ khẩu hao mỗi ký tự sinh ra

Sinh dãy vòng chứa mỗi từ dài n đúng một lần: nối các từ Lyndon có độ dài là ước của n theo thứ tự từ điển. Dạng tuyến tính: thêm $n - 1$ ký tự đầu của dãy vòng.

```
1 template<typename Emit> void de_bruijn(int k, int n, Emit emit) {
2     assert(k >= 1 && n >= 1);
3     if (k == 1) { emit(0); return; }
4     vector<int> a(n + 1);
5     function<void(int, int)> go = [&](int t, int p) {
6         if (t > n) {
7             if (n % p == 0) for (int i = 1; i <= p; ++i) emit(a[i]);
8             return;
9         }
10        a[t] = a[t - p]; go(t + 1, p);
11        for (int c = a[t - p] + 1; c < k; ++c) { a[t] = c; go(t + 1, t); }
12    }; go(1, 1);
13 }
```

5 Quy hoạch động (DP)

5.1 Reroot DP

Độ phức tạp: $O(n)$ cho lượt DFS

Template reroot trên cây: DFS hai lượt, chuyển trạng thái bằng prefix/suffix

```
1 // Associative merge, identity; finish(aggregate,u) adds vertex contribution.
2 // lift(state,from,to) transforms a complete state across an edge. O(n)
3 // callbacks.
4 template<typename T,typename Merge,typename Finish,typename Lift>vector<T>
5 reroot(const vector<vector<int>>&g,T identity,Merge merge,Finish finish,
6 Lift lift,int root=0){
7     int n = g.size();
8     if (!n) return {};
9     vector<int> parent(n, -1), order {root};
10    for(size_t i=0;i<order.size();++i){for(int v:g[order[i]])if(v!=parent[order[i]]){
11        parent[v] = order[i]; order.push_back(v);
12    }
13    vector<T> down(n, identity), up(n, identity), answer(n, identity);
14    for (auto it = order.rbegin(); it != order.rend(); ++it) {
15        int u = *it; T value = identity;
16        for(int v:g[u])if(v!=parent[u])value=merge(value,lift(down[v],v,u));
```

```
14     down[u] = finish(value, u);
15 }
16 for (int u : order) {
17     int degree = g[u].size();
18     vector<T>pref(degree+1,identity),suff(degree+1,identity),messages;
19     for(int v:g[u])messages.push_back(v==parent[u]?up[u]:lift(down[v],v,u));
20     for(int i=0; i<degree; ++i)pref[i+1]=merge(pref[i],messages[i]);
21     for(int i=degree-1; i>=0; --i)suff[i]=merge(messages[i],suff[i+1]);
22     answer[u] = finish(pref[degree], u);
23     for(int i=0;i<degree;++i)if(g[u][i]!=parent[u])up[g[u][i]]=lift(finish(
24     merge(pref[i],suff[i+1]),u,u,g[u][i]));
25 }
26 return answer;
27 }
```

5.2 Divide-and-Conquer DP và bất đẳng thức tứ giác

Độ phức tạp: $O(kn \log n)$ với DnC / $O(n^2)$ với Knuth

Bất đẳng thức tứ giác (Monge): $\forall a \leq b \leq c \leq d \implies C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ (Cross $\leq$ Nest).

• **Kiểm tra cục bộ cần và đủ:** Chỉ cần kiểm tra các ô 2×2 kề nhau (tổng rút gọn):

$$C(i, j + 1) - C(i, j) \geq C(i + 1, j + 1) - C(i + 1, j) \quad (\forall i < j)$$

(Tương đương, độ tăng chi phí $\Delta_j(i) = C(i, j + 1) - C(i, j)$ không tăng theo i ; trong trường hợp liên tục $\frac{\partial^2 C}{\partial x \partial y} \leq 0$).

• **Các hàm chi phí thường thỏa IQI:**

– $C(l, r) = f(S_r - S_{l-1})$ với f lồi ($f'' \geq 0$), ví dụ $(S_r - S_{l-1})^2$, $(S_r - S_{l-1})^k$ ($k \geq 1$).

– $C(l, r) = \sum_{k=l}^r w_k = W_r - W_{l-1}$ ($w_k \geq 0$, dấu bằng trong Knuth Optimal BST).

– $C(l, r) = \text{inv}(l, r)$ (số nghịch thế trong đoạn $[l, r]$, hoặc tính chất trên cặp chéo).

– $C(l, r) = |x_r - x_l|^p$ ($p \geq 1$ với x đã sắp xếp), hoặc phương sai/tổng bình phương chênh lệch.

• **Đơn điệu:** Tính chất Monge $\implies \text{opt}(i_1) \leq \text{opt}(i_2)$ với $i_1 \leq i_2$ (DnC: $O(kn \log n)$) và $\text{opt}[i][j - 1] \leq \text{opt}[i][j] \leq \text{opt}[i + 1][j]$ (Knuth: $O(n^2)$).

```
1 // next[i]=min_j<i{previous[j]+cost(j,i)}; cost describes [j,i].
2 // Requires nondecreasing optimal j as i increases. O(n log n) cost calls.
3 template<typename Cost>vector<long long>divide_conquer_layer(const vector<
4     long long>&previous, Cost cost, long long inf=1LL<<60){
5     int n = previous.size() - 1; vector<long long> next(n + 1, inf);
6     auto solve = [&](auto&& self, int l, int r, int lo, int hi)->void {
7         if (l > r) return;
8         int mid = (l + r) / 2, best = lo;
9         for (int j = lo; j <= min(mid - 1, hi); ++j) if (previous[j] < inf){
10            long long value = previous[j] + cost(j, mid);
11            if (value < next[mid]) { next[mid] = value; best = j; }
12        }
13        self(self, l, mid - 1, lo, best); self(self, mid + 1, r, best, hi);
14    };
15    if (n) solve(solve, 1, n, 0, n - 1);
16    return next;
17 }
18 // Interval DP: split [l,r] into two nonempty intervals and add cost(l,r).
19 // Requires Knuth inequalities; O(n^2) time/space. Singleton cost is zero.
20 template<typename Cost> long long knuth_partition(int n, Cost cost) {
21     if (!n) return 0;
22     vector<vector<long long>> dp(n, vector<long long>(n));
23     vector<vector<int>> opt(n, vector<int>(n));
```

```

23 for (int i = 0; i < n; ++i) opt[i][i] = i;
24 for (int len = 2; len <= n; ++len) for (int l = 0; l + len <= n; ++l){
25     int r = l + len - 1; dp[l][r] = 1LL << 60;
26     for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l + 1][r]); ++k) {
27         long long value = dp[l][k] + dp[k + 1][r] + cost(l, r + 1);
28         if (value < dp[l][r]) { dp[l][r] = value; opt[l][r] = k; }
29     }
30 }
31 return dp[0][n - 1];
32 }

```

5.3 Tối ưu DP 1D/1D

Độ phức tạp: $\mathcal{O}(n \log n)$

Tối ưu $dp[i] = \min_{j < i} (dp[j] + cost(j, i))$ thỏa bất đẳng thức tứ giác bằng deque các khoảng tối ưu

```

1 // dp[i]=min_{j<i}(dp[j]+cost(j,i)), dp[0]=initial.
2 // Requires single crossing: a newer candidate wins a suffix of future
  indices.
3 template<typename Cost>vector<long long>monotone_partition_dp(int n,Cost cost
  ,long long initial=0){
4     struct Candidate { int j,l,r; }; deque<Candidate>q; vector<long long>dp(n
  +1);
5     dp[0] = initial;
6     if (n) q.push_back( {0, 1, n});
7     auto eval = [&](int j, int i) { return dp[j] + cost(j, i); };
8     for (int i = 1; i <= n; ++i) {
9         while (q.front().r < i) q.pop_front();
10        dp[i] = eval(q.front().j, i);
11        if (i == n) break;
12        if (q.front().r == i) q.pop_front();
13        else q.front().l = i + 1;
14        while(!q.empty()&&eval(i,q.back().l)<=eval(q.back().j,q.back().l))q.
  pop_back();
15        if (q.empty()) { q.push_back( {i, i + 1, n}); continue; }
16        int l = q.back().l, r = q.back().r + 1;
17        while (l < r) {
18            int mid = (l + r) / 2;
19            if (eval(i, mid) <= eval(q.back().j, mid)) r = mid;
20            else l = mid + 1;
21        }
22        if (l <= n) { q.back().r = l - 1; q.push_back( {i, l, n}); }
23    }
24    return dp;
25 }

```

5.4 Alien DP

Độ phức tạp: $\mathcal{O}(n \log(\max_val))$

DP trên hàm lỗi/lỗm (WQS Binary Search):

- Chi phí lỗi/lỗm $\rightarrow$ Chọn greedy.
- DP lỗi/lỗm $\rightarrow$ Binary search hệ số phạt λ .

```

1 struct PenalizedResult { long long value; int count; };
2 // oracle(lambda) minimizes cost-lambda*count, breaking ties toward MORE
  count.
3 // Exact-count optimum must be discrete convex, attainable; lo/hi bracket
  count=k.
4 template<typename Oracle>long long aliens_exact(int k,long long lo,long long
  hi,Oracle oracle){
5     assert(lo <= hi && oracle(hi).count >= k);
6     while (lo < hi) {
7         long long mid=lo+static_cast<long long>((__int128(hi)-lo)/2);
8         if (oracle(mid).count >= k) hi = mid;
9         else lo = mid + 1;
10    }
11    auto result = oracle(lo); return result.value + lo * k;
12 }

```

5.5 Convex Hull Trick

Độ phức tạp: $\mathcal{O}(n \log n)$ hoặc $\mathcal{O}(n)$ khi hệ số góc đơn điệu

Tối ưu DP với hàm tuyến tính $y = mx + c$

```

1 // Add nonincreasing slopes; query nondecreasing x. Products fit int128;
2 // coefficients fit int64 and their products/differences fit signed int128.
3 struct MonotoneMinHull {
4     struct Line {
5         long long m, b;
6         __int128 at(long long x) const { return __int128(m) * x + b; }
7     }; deque<Line> q;
8     void add(long long m, long long b) {
9         Line c {m, b};
10        if (!q.empty() && q.back().m == m) {
11            if (q.back().b <= b) return;
12            q.pop_back();
13        }
14        while (q.size() > 1) {
15            auto a = q[q.size() - 2], z = q.back();
16            if((__int128(z.b)-a.b)*(__int128(z.m)-c.m)<(__int128(c.b)-z.b)*
  (__int128(a.m)-z.m))break;
17            q.pop_back();
18        }
19        q.push_back(c);
20    }
21    __int128 query(long long x) {
22        assert(!q.empty());
23        while (q.size() > 1 && q[0].at(x) >= q[1].at(x)) q.pop_front();
24        return q.front().at(x);
25    }
26 };

```

5.6 Slope Trick

Độ phức tạp: $\mathcal{O}(n \log n)$

Duy trì hàm lỗi tuyến tính từng đoạn liên tục $f(x)$: cộng $|x - a|$, prefix min, dịch miền bằng hai heap

```

1 // f starts at 0. Represents convex piecewise-linear f, slopes integral.
2 struct SlopeTrick {
3     priority_queue<long long> left;
4     priority_queue<long long,vector<long long>,greater<long long>>right;
5     long long lshift = 0, rshift = 0, value = 0;
6     long long minimum() const { return value; }
7     // Minimizers [left endpoint,right endpoint]; missing endpoints mean
  infinity.
8     pair<optional<long long>, optional<long long>> minimizers() const {
9         return {
10             left.empty()?nullopt:optional<long long>(left.top()+lshift),right.empty
  ()?nullopt:optional<long long>(right.top()+rshift)
11         };
12    }
13    void add_x_minus_a(long long a) {
14        if (!left.empty() && left.top() + lshift > a) {
15            long long x=left.top()+lshift; left.pop(); value+=x-a; left.push(a-
  lshift);
16            right.push(x - rshift);
17            } else right.push(a - rshift);
18    }
19    void add_a_minus_x(long long a) {
20        if (!right.empty() && right.top() + rshift < a) {
21            long long x=right.top()+rshift; right.pop(); value+=a-x; right.push(a-
  rshift);
22            left.push(x - lshift);
23            } else left.push(a - lshift);
24    }
25    void add_abs(long long a) { add_x_minus_a(a); add_a_minus_x(a); }
26    void add_constant(long long c) { value += c; }
27    void prefix_min() { right = {}; }
28    // g(x)=min_{y<=x}f(y).
29    void suffix_min() { left = {}; }
30    // g(x)=min_{x-b<=y<=x+a} f(y), a,b>=0.
31    void window_min(long long a, long long b) {
32        assert(a >= 0 && b >= 0); lshift -= a; rshift += b;
33    }

```

```

34 };

```

5.7 Bitmask DP

- **Matching:** Tập cặp không có thứ tự $\rightarrow$ cố định một đầu, chọn đầu còn lại. $\mathcal{O}(n2^n)$ thời gian, $\mathcal{O}(2^n)$ bộ nhớ.

- **Nén trạng thái:** $i = \text{popcount}(mask) \rightarrow dp[i][mask] \rightarrow dp[mask]$.

5.8 SOS DP

Độ phức tạp: $\mathcal{O}(N \cdot 2^N)$

Sum Over Subsets / Fast Zeta Transform: tính tổng trên submask/supermask trong $\mathcal{O}(N \cdot 2^N)$ ($N \leq 24$)

```

1 // In-place subset/superset zeta transform; inverse needs subtraction.
2 template<typename T>void subset_transform(vector<T>&a,bool supersets=false,
  bool inverse=false){
3     size_t n = a.size(); assert(n && (n & (n - 1)) == 0);
4     for(size_t bit=1;bit<n;bit<=<=1)for(size_t mask=0;mask<n;++mask)if(bool(mask
  &bit)!=supersets){
5         T value = a[mask ^ bit];
6         if (inverse) a[mask] -= value;
7         else a[mask] += value;
8     }
9 }

```

5.9 Tối ưu bằng monotonic queue

Độ phức tạp: $\mathcal{O}(1)$ khấu hao mỗi push/pop

Deque chỉ số tăng dần, tìm min trong sliding window

```

1 // Push strictly increasing indices; expire indices < first_valid.
2 template<typename T> struct MonotoneMinQueue {
3     deque<pair<int, T>> q;
4     void push(int index, T value) {
5         while (!q.empty() && q.back().second >= value) q.pop_back();
6         q.push_back( {index, value});
7     }
8     void expire(int first_valid) {
9         while (!q.empty() && q.front().first < first_valid) q.pop_front();
10    }
11    bool empty() const { return q.empty(); }
12    const T&minimum()const { assert(!q.empty()); return q.front().second; }
13 };

```

5.10 Expected Value Trick

Độ phức tạp: $\mathcal{O}(n\sqrt{n})$

Chọn tối ưu k trong n phần tử đã xáo trộn: bước i dùng heuristic dải ngẫu nhiên (có thể bỏ lỡ tối ưu), chỉ xét trạng thái trong $\left[\frac{i-k}{n} - C\sqrt{n}, \frac{i+k}{n} + C\sqrt{n}\right]$.

```

1 // Randomized band DP for additive pick-k; not a deterministic exact
  guarantee.
2 // Shuffles a COPY; caller provides half-width and RNG.  $\mathcal{O}(n*(width+1))$  work,
   $\mathcal{O}(width+1)$  space.
3 // nullopt means no feasible path stayed in the band; reaching k may still be
  suboptimal.
4 template<typename URBG>optional<long long>banded_pick_k(vector<long long>
  values,int k,int width,URBG&rng){
5     int n = values.size(); assert(0 <= k && k <= n && width >= 0);
6     if (!n) return 0;
7     shuffle(values.begin(),values.end(),rng); constexpr long long neg=-(1LL
  <<60);
8     vector<long long> dp(1, 0); int prev_lo = 0, prev_hi = 0;
9     for (int i = 1; i <= n; ++i) {
10        int center = static_cast<long long>(i) * k / n;

```

```
11     int lo=max( {0,center-width,k-(n-i)}),hi=min( {i,k,center+width});
12     if (lo > hi) return nullptr;
13     vector<long long> next(hi - lo + 1, neg);
14     for (int j = lo; j <= hi; ++j) {
15         if (prev_lo <= j && j <= prev_hi) next[j - lo] = dp[j - prev_lo];
16         if (j&&prev_lo<=j-1&&j-1<=prev_hi&&dp[j-1-prev_lo]!=neg)next[j-lo]=max(
17             next[j-lo],dp[j-1-prev_lo]+values[i-1]);
18     }
19     dp.swap(next); prev_lo = lo; prev_hi = hi;
20     return dp[k-prev_lo]==neg?nullptr:optional<long long>(dp[k-prev_lo]);
21 }
```

6 Toán học

6.1 Công thức số học

Số học modulo, rút gọn số mũ Euler mở rộng, Legendre, Wilson, Frobenius Coin, Floor Trick, CRT, Fibonacci

- Rút gọn số mũ Euler mở rộng $(\forall a, m)$:

$$a^b \equiv a^{(b \bmod \phi(m)) + \phi(m)} \pmod m \quad (b \geq \phi(m))$$

- Công thức Legendre $(v_p(n!))$: $v_p(n!) = \sum_{k=1}^{\infty} \lfloor \frac{n}{p^k} \rfloor = \frac{n - S_p(n)}{p - 1}$.

- Định lý Wilson: $(p - 1)! \equiv -1 \pmod p \iff p$ nguyên tố $\implies (p - 2)! \equiv 1 \pmod p$.

- Frobenius Coin (Chicken McNugget): Với $\gcd(a, b) = 1$, số lớn nhất không biểu diễn được dạng $ax + by$ ($x, y \geq 0$) là $ab - a - b$; số lượng không biểu diễn được là $\frac{(a-1)(b-1)}{2}$.

- Floor Trick $(O(\sqrt{n})$ nhóm):

```
1 for (int l = 1, r; l <= n; l = r + 1) {
2     int v = n / l; r = n / v;
3 }
```

- Định lý số dư Trung Hoa (CRT): $x \equiv a_i \pmod{m_i}$.

$$x \equiv \sum_{i=1}^k a_i M_i (M_i^{-1} \bmod m_i) \pmod M \quad (M = \prod m_i, M_i = M/m_i)$$

- Fibonacci: $\gcd(F_m, F_n) = F_{\gcd(m, n)}, F_{2n} = F_n(2F_{n+1} - F_n)$.

6.2 Like-Euclid Floor Sum

Độ phức tạp: $O(\log m)$

Tính $\sum_{i=0}^{n-1} \lfloor \frac{ai+b}{m} \rfloor$ bằng đổi tọa độ

```
1 // Nonnegative n,a,b; m>0. All intermediates and the result must fit signed
128 bits.
2 __int128 floor_sum(uint64_t n, uint64_t m, uint64_t a, uint64_t b) {
3     assert(m); __int128 ans = 0;
4     for (;;) {
5         if (a >= m) { ans += (__int128) n * (n - 1) / 2 * (a / m); a %= m; }
6         if (b >= m) { ans += (__int128) n * (b / m); b %= m; }
7         __uint128_t y = (__uint128_t) a * n + b;
8         if (y < m) break;
9         n = y / m; b = y % m; swap(m, a);
10    }
11    return ans;
12 }
```

6.3 Rabin-Miller

Độ phức tạp: $O(k \log n)$

Miller-Rabin kiểm tra nguyên tố tất định với $n < 2^{64}$

```
1 using U64 = uint64_t; using U128 = __uint128_t;
2 U64 mul_mod(U64 a, U64 b, U64 m) { return U128(a) * b % m; }
3 U64 pow_mod(U64 a, U64 e, U64 m) {
4     assert(m); U64 r = 1 % m;
5     for(a%=m; e>=>1,a=mul_mod(a,a,m))if(e&1)r=mul_mod(r,a,m);
6     return r;
7 }
8 bool is_prime(U64 n) {
9     if (n < 2) return false;
10    for(U64 p:{2,3,5,7,11,13,17,19,23,29,31,37})if(n%p==0)return n==p;
11    U64 d = n - 1; int s = 0;
12    while (!(d & 1)) d >>= 1, ++s;
13    for(U64 a:{2ULL,325ULL,9375ULL,28178ULL,450775ULL,9780504ULL,1795265022ULL
14    }){
15        if (a % n == 0) continue;
16        U64 x = pow_mod(a, d, n);
17        if (x == 1 || x == n - 1) continue;
18        bool pass = false;
19        for(int r=1;r<s;++r){ x=mul_mod(x,x,n); if(x==n-1){ pass=true; break; } }
20        if (!pass) return false;
21    }
22    return true;
23 }
```

6.4 Sàng nhanh (Linear Sieve)

Độ phức tạp: $O(N)$

Sàng Euler tuyến tính: nguyên tố và ước nguyên tố nhỏ nhất tới N

```
1 struct LinearSieve {
2     vector<int> primes, spf, phi, mu;
3     explicit LinearSieve(int n) : spf(n + 1), phi(n + 1), mu(n + 1) {
4         assert(n >= 0);
5         if (n) phi[1] = mu[1] = 1;
6         for (int i = 2; i <= n; ++i) {
7             if(!spf[i]){ spf[i]=i; primes.push_back(i); phi[i]=i-1; mu[i]=-1; }
8             for (int p : primes) {
9                 if (p > n / i) break;
10                spf[i * p] = p;
11                if (i%p==0){ phi[i*p]=phi[i]*p; mu[i*p]=0; break; }
12                phi[i * p] = phi[i] * (p - 1); mu[i * p] = -mu[i];
13            }
14        }
15    }
16    };
```

6.5 Đếm nguyên tố (Prime Counting)

Độ phức tạp: $O(N^{3/4})$ thời gian, $O(\sqrt{N})$ bộ nhớ

Sàng theo các giá trị thường: đếm nguyên tố $\leq N$

```
1 // Quotient-value sieve: O(N^(3/4)/log N) typical bound, O(sqrt N) storage.
2 uint64_t prime_count(uint64_t n) {
3     if (n < 2) return 0;
4     uint64_t r = sqrtl(n);
5     while ((__uint128_t)(r + 1) * (r + 1) <= n) ++r;
6     while ((__uint128_t) r * r > n) --r;
7     vector<uint64_t> v, g;
8     for (uint64_t l = 1; l <= n; ) {
9         uint64_t x=n/l; v.push_back(x); g.push_back(x-1); uint64_t next=n/x;
10        if (next == n) break;
11        l = next + 1;
12    }
13    auto index=&(uint64_t x)->size_t { return x<=r?v.size()-x:n/x-1; };
14    for (uint64_t p = 2; p <= r; ++p) {
15        if (g[index(p)] == g[index(p - 1)]) continue;
16        uint64_t before = g[index(p - 1)];
17        for(size_t i=0;i<v.size()&&v[i]>=p*p;++i)g[i]-=g[index(v[i]/p)]-before;
```

```
18     }
19     return g[0];
20 }
```

6.6 Pollard Rho

Độ phức tạp: $O(\sqrt[4]{n} \log n)$

Tìm ước không tầm thường của hợp số bằng phát hiện chu trình Brent

```
1 U64 rho_factor(U64 n, mt19937_64 & rng) {
2     if (n % 2 == 0) return 2;
3     if (n % 3 == 0) return 3;
4     for (;;) {
5         U64 c=rng()% (n-1)+1,y=rng()% (n-1)+1,g=1,r=1,x=0,ys=0;
6         auto f=[&](U64 v){ return U64((U128(mul_mod(v,v,n))+c)%n); };
7         while (g == 1) {
8             x = y;
9             for (U64 i = 0; i < r; ++i) y = f(y);
10            for (U64 k = 0; k < r && g == 1; k += 128) {
11                ys = y; U64 q = 1;
12                for(U64 i=0;i<min<U64>(128,r-k);++i){ y=f(y); q=mul_mod(q,x>y?x-y:y-x
13                ,n); }
14                g = gcd(q, n);
15            }
16            r *= 2;
17        }
18        if (g == n) {
19            do { ys = f(ys); g = gcd(x > ys ? x - ys : ys - x, n); }
20            while (g == 1);
21        }
22        if (g < n) return g;
23    }
24 }
25 vector<U64> factorize(U64 n, mt19937_64 & rng) {
26     assert(n >= 1); vector<U64> out;
27     function<void(U64)> go = [&](U64 x) {
28         if (x == 1) return;
29         if (is_prime(x)) { out.push_back(x); return; }
30         U64 d = rho_factor(x, rng); go(d); go(x / d);
31     }; go(n); sort(out.begin(), out.end()); return out;
32 }
```

6.7 Primitive Root

Độ phức tạp: $\mathcal{O}(\text{ans} \cdot \log p)$

Tìm căn nguyên thủy g nhỏ nhất modulo nguyên tố p

```
1 U64 primitive_root(U64 p, mt19937_64 & rng) {
2     assert(is_prime(p));
3     if (p == 2) return 1;
4     auto f=factorize(p-1,rng); f.erase(unique(f.begin(),f.end()),f.end());
5     for (U64 g = 2; g < p; ++g) {
6         bool ok = true;
7         for (U64 q : f) if (pow_mod(g, (p - 1) / q, p) == 1) {
8             ok = false; break;
9         }
10        if (ok) return g;
11    }
12    throw logic_error("prime required");
13 }
```

6.8 Baby-step Giant-Step

Độ phức tạp: $O(\sqrt{m} \log m)$
Tìm x nhỏ nhất thỏa $a^x \equiv b \pmod{m}$ (Discrete Log)

```
1 // Smallest exponent, including non-coprime a,m. Practical m limited by sqrt(m) memory.
2 optional<U64> discrete_log(U64 a, U64 b, U64 m) {
3     assert(m); a %= m; b %= m;
4     if (b == 1 % m) return 0;
5     U64 k = 1 % m, offset = 0;
6     for (U64 g; (g = gcd(a, m)) > 1;) {
7         if (b == k) return offset;
8         if (b % g) return nullopt;
9         b /= g; m /= g; k = mul_mod(k, a / g, m); ++offset;
10    }
11    U64 n = sqrtl(m) + 1; unordered_map<U64, U64> baby; U64 x = b;
12    for (U64 q = 0; q <= n; ++q) { baby[x] = q; x = mul_mod(x, a, m); }
13    U64 step = pow_mod(a, n, m); x = k;
14    for (U64 p = 1; p <= n + 1; ++p) {
15        x = mul_mod(x, step, m); auto it = baby.find(x);
16        if (it != baby.end()) return p * n - it->second + offset;
17    }
18    return nullopt;
19 }
```

6.9 Căn modulo (Tonelli-Shanks)

Độ phức tạp: $O(\log^2 p)$
Tonelli-Shanks: giải $x^2 \equiv a \pmod{p}$

```
1 optional<U64> sqrt_mod(U64 a, U64 p) {
2     assert(is_prime(p)); a %= p;
3     if (p == 2 || a == 0) return a;
4     if (pow_mod(a, (p - 1) / 2, p) != 1) return nullopt;
5     if (p%4==3){ U64 x=pow_mod(a,p/4+1,p); return min(x,p-x); }
6     U64 q = p - 1; int s = 0;
7     while (!(q & 1)) q >>= 1, ++s;
8     U64 z = 2;
9     while (pow_mod(z, (p - 1) / 2, p) != p - 1) ++z;
10    U64 c=pow_mod(z,q,p),x=pow_mod(a,(q+1)/2,p),t=pow_mod(a,q,p);
11    while (t != 1) {
12        int i = 0; U64 u = t;
13        while (u != 1) { u = mul_mod(u, u, p); ++i; }
14        U64 b=pow_mod(c,u,U64(1)<<(s-i-1),p); x=mul_mod(x,b,p); c=mul_mod(b,b,p);
15        t = mul_mod(t, c, p); s = i;
16    }
17    return min(x, p - x);
18 }
```

6.10 Extended GCD

Độ phức tạp: $O(\log \min(a, b))$
Tìm x, y thỏa phương trình $ax + by = \gcd(a, b)$

```
1 struct Bezout { __int128 gcd, x, y, };
2 Bezout extended_gcd(long long a, long long b) {
3     __int128 r = a, s = b, x = 1, u = 0, y = 0, v = 1;
4     while (s) {
5         __int128 q=r/s; tie(r,s)=pair{s,r-q*s}; tie(x,u)=pair{u,x-q*u};
6         tie(y, v) = pair{v, y - q * v};
7     }
8     if (r < 0) r = -r, x = -x, y = -y;
9     return {r, x, y};
10 }
```

6.11 Diophantine

Độ phức tạp: $O(\log \min(a, b))$
Nghiệm tổng quát của phương trình Diophantine tuyến tính $ax + by = c$

```
1 struct DiophantineSolution { bool all_pairs; __int128 x, y, dx, dy, };
2 // Every solution is (x+t*dx,y+t*dy), except all_pairs for 0x+0y=0.
3 optional<DiophantineSolution>diophantine(long long a,long long b,long long c)
4 {
5     auto z = extended_gcd(a, b);
6     if (!z.gcd) {
7         if (c) return nullopt;
8         return DiophantineSolution {true, 0, 0, 0, 0};
9     }
10    if (c % z.gcd) return nullopt;
11    return DiophantineSolution {
12        false, z.x*(c/z.gcd), z.y*(c/z.gcd), b/z.gcd, -(__int128)a/z.gcd
13    };
14 }
```

6.12 Stern-Brocot Tree

Độ phức tạp: $O(\log(p + q))$
Phân số tối giản, mã hóa/giải mã cây, LCA

```
1 using SternPath = vector<pair<char, uint64_t>>;
2 SternPath stern_encode(uint64_t p, uint64_t q) {
3     assert(p && q && gcd(p, q) == 1); SternPath out;
4     while (p != q) {
5         if (p < q) {
6             uint64_t k = (q - 1) / p; out.push_back( {'L', k}); q -= k * p;
7         } else { uint64_t k=(p-1)/q; out.push_back( {'R', k}); p-=k*q; }
8     }
9     return out;
10 }
11 // Fractions and intermediate bounds must fit unsigned 128 bits.
12 pair<__uint128_t, __uint128_t> stern_decode(const SternPath& path) {
13     __uint128_t a = 0, b = 1, c = 1, d = 0;
14     for (auto[dir, k] : path) {
15         assert(k && (dir == 'L' || dir == 'R'));
16         if (dir == 'L') c += k * a, d += k * b;
17         else a += k * c, b += k * d;
18     }
19     return {a + c, b + d};
20 }
21 SternPath stern_lca(const SternPath& a, const SternPath& b) {
22     SternPath out;
23     for(size_t i=0; i<min(a.size(),b.size())&&a[i].first==b[i].first; ++i){
24         out.push_back( {a[i].first, min(a[i].second, b[i].second)});
25         if (a[i].second != b[i].second) break;
26     }
27     return out;
28 }
```

6.13 Hàm nhân tính và Möbius Inversion

Tích chập Dirichlet, nghịch đảo Möbius, tổng GCD/LCM

• Euler Phi & Möbius: $\phi(n) = n \prod_{p|n} (1 - 1/p)$, $\sum_{d|n} \phi(d) = n$, $\sum_{d|n} \mu(d) = [n = 1]$.

• Dirichlet & Inversion: $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$.

$$f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu(d)f(n/d)$$

• Đếm cặp nguyên tố cùng nhau:

$$\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_{d=1}^{\min(n, m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

• Tổng GCD trên mọi cặp:

$$\sum_{i=1}^n \sum_{j=1}^m \gcd(i, j) = \sum_{g=1}^{\min(n, m)} \phi(g) \left\lfloor \frac{n}{g} \right\rfloor \left\lfloor \frac{m}{g} \right\rfloor$$

• Tổng LCM trên mọi cặp $(S(x) = \frac{x(x+1)}{2})$:

$$\sum_{i \leq n, j \leq m} \text{lcm}(i, j) = \sum_g g \sum_d d^2 \mu(d) S\left(\left\lfloor \frac{n}{gd} \right\rfloor\right) S\left(\left\lfloor \frac{m}{gd} \right\rfloor\right)$$

6.14 Công thức tổ hợp

Đẳng thức nhị thức, Hockey-stick, Vandermonde, Inversion, Stars and Bars, Catalan, Stirling, Cayley, LGV

• Đẳng thức nhị thức: $\binom{n}{k} = \binom{n}{n-k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; $k\binom{n}{k} = n\binom{n-1}{k-1}$; $\frac{1}{k+1}\binom{n}{k} = \frac{1}{n+1}\binom{n+1}{k+1}$; $\binom{n}{k}\binom{n}{m} = \binom{n}{m}\binom{n-m}{k-m}$; Đối số trên âm: $\binom{-n}{k} = (-1)^k \binom{n+k-1}{k}$.

• Tổng nhị thức: $\sum_{k=0}^n \binom{n}{k} = 2^n$; $\sum_{k=0}^n (-1)^k \binom{n}{k} = [n = 0]$; $\sum_k \text{chẵn} \binom{n}{k} = \sum_k \text{lẻ} \binom{n}{k} = 2^{n-1}$; $\sum_{k=0}^n k \binom{n}{k} = n2^{n-1}$; $\sum_{k=0}^n k^2 \binom{n}{k} = n(n+1)2^{n-2}$; $\sum_{k=0}^n \frac{1}{k+1} \binom{n}{k} = \frac{2^{n+1}-1}{n+1}$.

• Hockey-Stick & Vandermonde: $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$; $\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$; $\sum_{k=0}^r \binom{n}{k} \binom{n}{r-k} = \binom{m+n}{r} \implies \sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$.

• Binomial Inversion và sai phân: $f(n) = \sum_{k=0}^n \binom{n}{k} g(k) \iff g(n) = \sum_{k=n}^N \binom{n}{k} f(k)$; Dạng tổng phía trên: $f(n) = \sum_{k=n}^N \binom{n}{k} g(k) \iff g(n) = \sum_{k=n}^N (-1)^{k-n} \binom{n}{k} f(k)$; $\sum_{k=0}^n (-1)^{n-k} \binom{n}{k} k^m = n! \begin{Bmatrix} m \\ n \end{Bmatrix}$ (0 nếu $m < n$, $n!$ nếu $m = n$).

• Stars and Bars: $x_i \geq 0 \implies \binom{n+k-1}{k-1}$; $x_i \geq 1 \implies \binom{n-1}{k-1}$; $x_i \geq c_i \implies n' = n - \sum c_i$.

• Derangements: $D_0 = 1, D_1 = 0, D_n = (n - 1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n \approx [n!/e]$.

• Số Catalan $C_n = \frac{1}{n+1} \binom{2n}{n}$: Dãy ngoặc đúng, cây nhị phân đầy đủ có $n + 1$ lá, chia tam giác đa giác lồi $(n + 2)$ đỉnh, đường Dyck không vượt $y = x$, hoán vị sinh bằng một stack.

• Nguyên lý phản xạ (Reflection Principle): Đường đi $(0, 0) \rightarrow (n, m)$ không chạm $y = x + k$ ($k > 0$): $\binom{n+m}{m} - \binom{n+m}{m-k}$.

• Định lý Ballot: A có a phiếu, B có b ($a > b$). Xác suất A luôn dẫn trước nghiêm ngặt: $\frac{a-b}{a+b}$.

• Số Stirling loại 1 và 2: $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] = \left[\begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right] + (n-1) \left[\begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right]$, $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\} + k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\}$. $x^n = \sum_{k=0}^n \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} x^{\underline{k}} \implies \sum_{i=1}^n i^k = \sum_{j=0}^k \left\{ \begin{smallmatrix} k \\ j \end{smallmatrix} \right\} j! \binom{n+1}{j+1}$.

• PIE: $|\bigcup_{i=1}^n A_i| = \sum_{\emptyset \neq J} (-1)^{|J|-1} |\bigcap_{j \in J} A_j|$; Thuộc đúng r tập: $\sum_{m=r}^n (-1)^{m-r} \binom{n}{r} \sum_{|X|=m} |\bigcap_{i \in X} A_i|$.

• Định lý Lucas: $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$ (chữ số cơ số p).

- **Đếm cây và Cayley:** n^{n-2} cây có nhãn, rừng gồm k cây có gốc: kn^{n-k-1} , Matrix Tree Theorem ($\det(L_i)$), bổ đề LGV ($\det(M)$).

6.15 Hàm sinh (Generating Functions) và chuỗi Taylor

OGF, EGF, khai triển Taylor, tích phân số Simpson

- OGF: $A(x) = \sum a_n x^n$; EGF: $A(x) = \sum a_n \frac{x^n}{n!}$.
- $\frac{1}{1-x} = \sum_{n \geq 0} x^n$, $\frac{1}{(1-x)^k} = \sum_{n \geq 0} \binom{n+k-1}{k-1} x^n$.
- $e^x = \sum_{n \geq 0} \frac{x^n}{n!}$, $\ln(1-x) = -\sum_{n \geq 1} \frac{x^n}{n}$, $(1+x)^\alpha = \sum_{n \geq 0} \binom{\alpha}{n} x^n$.
- **Công thức Simpson:** $\int_a^{a+2h} f(x)dx \approx \frac{h}{3}(f(a) + 4f(a+h) + f(a+2h))$.

6.16 Truy hồi tuyến tính (Berlekamp-Massey)

Độ phức tạp: $\mathcal{O}(N^2)$ tìm truy hồi, $\mathcal{O}(N^2 \log K)$ tính số hạng thứ k

Tìm truy hồi tuyến tính tối tiểu từ $2N$ số hạng đầu, tính số hạng thứ K qua $x^K \bmod P(x)$

```

1 // Prime modulus 998244353; s[i]=sum recurrence[j]*s[i-j].
2 vector<int> berlekamp_massey(vector<int> s) {
3     for (auto& x : s) x = (x % MOD + MOD) % MOD;
4     vector<int> C {1}, B {1}; int L = 0, gap = 1; long long previous = 1;
5     for (int i = 0; i < (int) s.size(); ++i) {
6         long long d = (s[i] % MOD + MOD) % MOD;
7         for (int j = 1; j <= L; ++j) d = (d + 1LL * C[j] * s[i - j]) % MOD;
8         if (!d) { ++gap; continue; }
9         auto old = C; long long coef = d * power(previous, MOD - 2) % MOD;
10        if (C.size() < B.size() + gap) C.resize(B.size() + gap);
11        for(int j=0;j<(int)B.size();++j)C[j+gap]=(C[j+gap]-coef*B[j]%MOD+MOD)%MOD
12        ;
13        if(2*L<i){ L=i+1-L; B=move(old); previous=d; gap=1; } else ++gap;
14    }
15    C.resize(L + 1); C.erase(C.begin());
16    return C;
17 }
18 int recurrence_term(const vector<int>&initial,const vector<int>&recurrence,
19                     uint64_t k){
20     int n = recurrence.size();
21     if (k < initial.size()) return(initial[k] % MOD + MOD) % MOD;
22     if (!n) return 0;
23     assert(initial.size() >= (size_t) n);
24     auto combine = [&](const vector<int>& a, const vector<int>& b) {
25         vector<int> c(2 * n - 1);
26         for(int i=0;i<n;++i)for(int j=0;j<n;++j)c[i+j]=(c[i+j]+1LL*a[i]*b[j])%MOD
27         ;
28         for(int i=2*n-2;i>=n;--i)for(int j=1;j<=n;++j)c[i-j]=(c[i-j]+1LL*c[i]*
29         recurrence[j-1])%MOD;
30         c.resize(n); return c;
31     }; vector<int> a(n), x(n); a[0] = 1;
32     if (n == 1) x[0] = recurrence[0];
33     else x[1] = 1;
34     for (; k >= 1, x = combine(x, x)) if (k & 1) a = combine(a, x);
35     long long ans = 0;
36     for(int i=0; i<n; ++i)ans=(ans+1LL*a[i]*initial[i])%MOD;
37     return(ans + MOD) % MOD;
38 }
```

6.17 FWHT

Độ phức tạp: $\mathcal{O}(n \log n)$
Fast Walsh-Hadamard Transform (XOR, AND, OR Convolution)

```

1 enum class Walsh {Xor, And, Or};
2 // Integer inverse XOR requires exact division; values/intermediates must fit
3
4 void fwht(vector<long long>& a, Walsh kind, bool inverse = false) {
5     size_t n = a.size(); assert(n && !(n & (n - 1)));
6     for(size_t len=1;len<n;len*=2)for(size_t i=0;i<n;i+=2*len)for(size_t j=0;j<
7     len;++j){
8         auto& x = a[i + j]; auto& y = a[i + j + len];
9         if (kind == Walsh::Xor) {
10             long long u = x, v = y; x = u + v; y = u - v;
11         }else if (kind==Walsh::Or){ y+=inverse?-x:x; }else { x+=inverse?-y:y; }
12     }
13     if (inverse && kind == Walsh::Xor) for (auto& x : a) {
14         assert(x % (long long) n == 0); x /= n;
15     }
16 }
```

6.18 Lagrange

Độ phức tạp: $\mathcal{O}(n)$ tính giá trị

Nội suy Lagrange tại X từ $n + 1$ điểm cách đều có giá trị $y_0, \dots, y_n$

```

1 // y[i]=f(i), degree < y.size() < prime modulus; input values reduced.
2 U64 lagrange(const vector<U64>& y, U64 x, U64 p) {
3     assert(is_prime(p)&&!y.empty())&&y.size()<p; int n=y.size(); x%=p;
4     if (x < (U64) n) return y[x] % p;
5     vector<U64> pre(n + 1, 1), suf(n + 1, 1), invfact(n, 1);
6     for (int i = 0; i < n; ++i) pre[i + 1] = mul_mod(pre[i], x - i, p);
7     for(int i=n-1; i>0; --i)suf[i]=mul_mod(suf[i+1],x-i,p);
8     U64 fact = 1;
9     for (int i = 1; i < n; ++i) fact = mul_mod(fact, i, p);
10    invfact[n - 1] = pow_mod(fact, p - 2, p);
11    for(int i=n-1; i>0; --i)invfact[i-1]=mul_mod(invfact[i],i,p);
12    U64 ans = 0;
13    for (int i = 0; i < n; ++i) {
14        U64 term = mul_mod(y[i] % p, mul_mod(pre[i], suf[i + 1], p), p);
15        term = mul_mod(term, mul_mod(invfact[i], invfact[n - 1 - i], p), p);
16        if ((n - 1 - i) & 1) term = term * p - term : 0;
17        ans = U64((U128(ans) + term) % p);
18    }
19    return ans;
20 }
```

6.19 NTT (Number Theoretic Transform)

Độ phức tạp: $\mathcal{O}(n \log n)$

NTT lặp với bit-reversal, MOD = 998244353 ($G = 3$)

```

1 const int MOD = 998244353, G = 3;
2 long long power(long long a, long long b) {
3     long long res = 1;
4     for(a%=MOD; b; b>>=1,a=a%MOD)if (b&1)res=res*a%MOD;
5     return res;
6 }
7 long long modInverse(long long n) { return power(n, MOD - 2); }
8 void ntt(vector<int>& a, bool invert) {
9     int n = a.size(); assert(n > 0 && !(n & (n - 1)) && n <= (1 << 23));
10    for (int i = 1, j = 0; i < n; i++) {
11        int bit = n >> 1;
12        for (; j & bit; bit >>= 1) j ^= bit;
13        j ^= bit;
14        if (i < j) swap(a[i], a[j]);
15    }
16    for (int len = 2; len <= n; len <= 1) {
17        long long wlen = power(G, (MOD - 1) / len);
18        if (invert) wlen = modInverse(wlen);
19        for (int i = 0; i < n; i += len) {
20            long long w = 1;
21            for (int j = 0; j < len / 2; j++) {
22                long long u = a[i + j], v = a[i + j + len / 2] * w % MOD;
23                a[i + j] = (u + v >= MOD ? u + v - MOD : u + v);
24                a[i+j+len/2]=(u-v<0?u-v+MOD:u-v); w=w*wlen%MOD;
25            }
26        }
27    }
```

```

27 }
28 if (invert) {
29     long long inv_n = modInverse(n);
30     for (int& x : a) x = x * inv_n % MOD;
31 }
32 }
33 vector<int> convolution_ntt(vector<int> a, vector<int> b) {
34     if (a.empty() || b.empty()) return {};
35     for (auto& x : a) x = (x % MOD + MOD) % MOD;
36     for (auto& x : b) x = (x % MOD + MOD) % MOD;
37     int sz = 1, need = a.size() + b.size() - 1;
38     while (sz < need) sz <= 1;
39     a.resize(sz); b.resize(sz); ntt(a, false); ntt(b, false);
40     for (int i = 0; i < sz; i++) a[i] = 1LL * a[i] * b[i] % MOD;
41     ntt(a, true); a.resize(need); return a;
42 }
```

6.20 FFT (Fast Fourier Transform)

Độ phức tạp: $\mathcal{O}(n \log n)$

FFT Cooley-Tukey trên số phức cho tích chập số thực/tổng quát

```

1 // Rounded integer convolution; coefficient magnitudes must allow reliable
2 double precision.
3 using cd = complex<double>; const double PI = acos(-1);
4 void fft(vector<cd>& a, bool invert) {
5     int n = a.size(); assert(n > 0 && !(n & (n - 1)));
6     for (int i = 1, j = 0; i < n; i++) {
7         int bit = n >> 1;
8         for (; j & bit; bit >>= 1) j ^= bit;
9         j ^= bit;
10        if (i < j) swap(a[i], a[j]);
11    }
12    for (int len = 2; len <= n; len <= 1) {
13        double ang=2*PI/len*(invert?-1:1); cd wlen(cos(ang),sin(ang));
14        for (int i = 0; i < n; i += len) {
15            cd w(1);
16            for (int j = 0; j < len / 2; j++) {
17                cd u = a[i + j], v = a[i + j + len / 2] * w; a[i + j] = u + v;
18                a[i + j + len / 2] = u - v; w *= wlen;
19            }
20        }
21        if (invert) for (cd& x : a) x /= n;
22    }
23    vector<long long>convolution_fft(const vector<int>&a,const vector<int>&b){
24        if (a.empty() || b.empty()) return {};
25        vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
26        int sz = 1, need = a.size() + b.size() - 1;
27        while (sz < need) sz <= 1;
28        fa.resize(sz); fb.resize(sz); fft(fa, false); fft(fb, false);
29        for (int i = 0; i < sz; i++) fa[i] += fb[i];
30        fft(fa, true); vector<long long> res(need);
31        for (int i = 0; i < need; i++) res[i] = round(fa[i].real());
32        return res;
33    }
```

6.21 Công thức Formal Power Series (FPS)

Công thức thao tác đa thức bằng phương pháp Newton và convolution

- **Nghịch đảo** ($Q = P^{-1}$): $Q_k \equiv Q_{k-1}(2 - PQ_{k-1}) \pmod{x^{2^k}}$, khởi đầu $Q_0 = P(0)^{-1}$.

- **Đạo hàm và tích phân:** $P'(x) = \sum ia_i x^{i-1}$, $\int P(x) = \sum \frac{a_i}{i+1} x^{i+1}$.

- **Logarithm:** $\ln P(x) = \int \frac{P'(x)}{P(x)} dx \pmod{x^n}$ (cần $P(0) = 1$).

- **Hàm mũ:** $Q_k \equiv Q_{k-1}(1 - \ln Q_{k-1} + P) \pmod{x^{2^k}}$ (cần $P(0) = 0$).

- **Căn bậc hai:** $Q_k \equiv \frac{1}{2}(Q_{k-1} + PQ_{k-1}^{-1}) \pmod{x^{2^k}}$ ($P(0)$ là thặng dư bậc hai).

• **Lũy thừa:** $P(x)^k = \exp(k \ln P(x))$. Nếu $P(x) = cx^d(1 + Q(x))$, $P(x)^k = c^k x^{kd} \exp(k \ln(1 + Q(x)))$.

• **Chia lấy thương và dư** ($A = BQ + R$): Với $\text{rev}_k(P) = x^k P(1/x)$:

$$\text{rev}_{n-m}(Q) \equiv \text{rev}_n(A) \cdot (\text{rev}_m(B))^{-1} \pmod{x^{n-m+1}}, \quad R = A - BQ$$

• **Taylor Shift** $P(x + c)$: $P(x + c) = \sum_{k=0}^n \frac{x^k}{k!} \sum_{i=k}^n (i! a_i) \frac{c^{i-k}}{(i-k)!}$.
Convolution dãy $A_i = i! a_i$ đảo ngược với $B_j = \frac{c^j}{j!}$.

• **Convolution modulo tùy ý (MTT)**: Tách $A = A_1 B + A_0$, $B = \lceil \sqrt{M} \rceil \approx 32000$. Tính $(A_1 B_1), (A_1 B_0 + A_0 B_1), (A_0 B_0)$ bằng bốn FFT.

6.22 Đại số tuyến tính (Gauss)

Độ phức tạp: $\mathcal{O}(n^3)$

Khử Gauss: định thức, hạng ma trận, hệ phương trình tuyến tính

```
1 struct LinearSolution {
2     int rank; bool consistent; vector<double> particular; vector<vector<double>>
    nullspace;
3 };
4 // Augmented n x (m+1) matrix; pivot tolerance depends on input scaling.
5 LinearSolution gauss(vector<vector<double>>&a, int m, double eps=1e-9) {
6     int n = a.size(), r = 0; vector<int> where(m, -1);
7     for (auto& row : a) assert((int) row.size() == m + 1);
8     for (int c = 0; c < m && r < n; ++c) {
9         int p = r;
10        for(int i=r+1; i<n; ++i)if(abs(a[i][c])>abs(a[p][c]))p=i;
11        if (abs(a[p][c]) <= eps) continue;
12        swap(a[p], a[r]); double t = a[r][c];
13        for (int j = c; j <= m; ++j) a[r][j] /= t;
14        for (int i = 0; i < n; ++i) if (i != r) {
15            t = a[i][c];
16            for (int j = c; j <= m; ++j) a[i][j] -= t * a[r][j];
17        }
18        where[c] = r++;
19    }
20    LinearSolution out { r, true, vector<double>(m), {} };
21    for (int i = r; i < n; ++i) if (abs(a[i][m]) > eps) {
22        out.consistent = false; return out;
23    }
24    for(int c=0; c<m; ++c)if(where[c]>=0)out.particular[c]=a[where[c]][m];
25    else {
26        vector<double> v(m); v[c] = 1;
27        for(int j=0; j<m; ++j)if(where[j]>=0)v[j]=-a[where[j]][c];
28        out.nullspace.push_back(v);
29    }
30    return out;
31 }
32 double determinant(vector<vector<double>>& a, double eps = 1e-9) {
33     int n = a.size(); double ans = 1;
34     for (auto& row : a) assert((int) row.size() == n);
35     for (int c = 0; c < n; ++c) {
36         int p = c;
37         for(int i=c+1; i<n; ++i)if(abs(a[i][c])>abs(a[p][c]))p=i;
38         if (abs(a[p][c]) <= eps) return 0;
39         if (p != c) swap(a[p], a[c]), ans = -ans;
40         double pivot = a[c][c]; ans *= pivot;
41         for (int i = c + 1; i < n; ++i) {
42             double f = a[i][c] / pivot;
43             for (int j = c + 1; j < n; ++j) a[i][j] -= f * a[c][j];
44         }
45     }
46     return ans;
47 }
```

6.23 XOR Linear Basis

Độ phức tạp: $\mathcal{O}(B)$ thêm/truy vấn ($B \leq 60$)

XOR basis: XOR max/min/thứ k của tập con, sinh 2^{size} giá trị phân biệt

```
1 struct XorBasis {
2     array<uint64_t, 64> b {}; int count = 0;
3     bool insert(uint64_t x) {
4         for (int i = 63; i >= 0; --i) if (x >> i & 1) {
5             if (b[i] x ^= b[i];
6             else { b[i] = x; ++count; return true; }
7         }
8         return false;
9     }
10    int rank() const { return count; }
11    bool contains(uint64_t x) const {
12        for (int i = 63; i >= 0; --i) if (x >> i & 1) x ^= b[i];
13        return x == 0;
14    }
15    uint64_t maximum(uint64_t x = 0) const {
16        for (int i = 63; i >= 0; --i) x = max(x, x ^ b[i]);
17        return x;
18    }
19    uint64_t minimum(uint64_t x = 0) const {
20        for (int i = 63; i >= 0; --i) x = min(x, x ^ b[i]);
21        return x;
22    }
23    optional<uint64_t> kth(uint64_t k) const {
24        if (count < 64 && k >= (uint64_t(1) << count)) return nullopt;
25        auto a = b; vector<uint64_t> v;
26        for (int i = 0; i < 64; ++i) if (a[i]) {
27            for (int j = 0; j < i; ++j) if (a[i] >> j & 1) a[i] ^= a[j];
28            v.push_back(a[i]);
29        }
30        uint64_t x = 0;
31        for (int i = 0; i < (int) v.size(); ++i) if (k >> i & 1) x ^= v[i];
32        return x;
33    }
34 };
```

6.24 Burnside Lemma & Pólya Theorem

Đếm lớp tương đương dưới tác động nhóm: necklace, bracelet

• $|Classes| = \frac{1}{|G|} \sum_{g \in G} |Fix(g)| = \frac{1}{|G|} \sum_{g \in G} k^{c(g)}$.

• **Necklace (chỉ n phép xoay):** $N(n, k) = \frac{1}{n} \sum_{d|n} \phi(d) k^{n/d}$.

• **Bracelet (xoay và đối xứng):**

$$- n \text{ lẻ: } B(n, k) = \frac{1}{2} N(n, k) + \frac{1}{2} k^{(n+1)/2}.$$

$$- n \text{ chẵn: } B(n, k) = \frac{1}{2} N(n, k) + \frac{1}{4} (k^{n/2} + k^{n/2+1}).$$

6.25 Chiến lược và suy luận trạng thái game

Độ phức tạp: $\mathcal{O}(V + E)$

Trạng thái thắng/thua, retrograde analysis, đối xứng, chặn lẻ, Sprague-Grundy MEX, biến thể Nim

```
1 const int DRAW = 0, WIN = 1, LOSE = -1;
2 vector<int> retrograde_analysis(int n, const vector<vector<int>>& adj){
3     vector<int> out_deg(n), res(n, DRAW); vector<vector<int>> radj(n); queue<int> q;
4     for (int u = 0; u < n; ++u) {
5         out_deg[u] = adj[u].size();
6         for (int v : adj[u]) radj[v].push_back(u);
7         if (out_deg[u] == 0) { res[u] = LOSE; q.push(u); }
8     }
9     while (!q.empty()) {
10        int u = q.front(); q.pop();
11        for (int p : radj[u]) {
12            if (res[p] != DRAW) continue;
13            if (res[u] == LOSE) {
14                res[p] = WIN; q.push(p);
15            } else if (res[u] == WIN) { if(--out_deg[p]==0){ res[p]=LOSE; q.push(p); }
16        }
17    }
```

```
16     }
17 }
18 return res;
19 }
20 int calc_mex(const vector<int>& vals) {
21     int m = vals.size(); vector<bool> seen(m + 1, false);
22     for (int x : vals) if (0 <= x && x <= m) seen[x] = true;
23     for (int i = 0; i <= m; ++i) if (!seen[i]) return i;
24     return m + 1;
25 }
26 int get_grundy(int u, vector<int>& g, const vector<vector<int>>& adj){
27     if (g[u] != -1) return g[u];
28     vector<int> nxt;
29     for (int v : adj[u]) nxt.push_back(get_grundy(v, g, adj));
30     return g[u] = calc_mex(nxt);
31 }
32 // get_grundy requires a DAG and caller-owned memo initialized to -1.
```

• **Quy tắc trạng thái thắng ($\mathcal{W}/\mathcal{N}$ -pos) và thua ($\mathcal{L}/\mathcal{P}$ -pos):**

– **Trạng thái kết thúc:** Normal play (hết nước đi $\implies \mathcal{L}$); Misère play (hết nước đi $\implies \mathcal{W}$).

– **Trạng thái thắng $\mathcal{W}$:** $\exists v \in \text{next}(u)$ sao cho $v \in \mathcal{L}$ (có ít nhất một nước tới trạng thái $\mathcal{L}$).

– **Trạng thái thua $\mathcal{L}$:** $\forall v \in \text{next}(u)$, $v \in \mathcal{W}$ (mọi nước tới $\mathcal{W}$, hoặc hết nước đi).

– **Chơi tối ưu:** Từ $\mathcal{W}$, chọn nước tới $\mathcal{L}$. Đối thủ từ $\mathcal{L}$ buộc phải trở lại $\mathcal{W}$.

• **Cách suy luận lời giải:**

– **Retrograde BFS ($\mathcal{O}(V + E)$):** Lan truyền kết quả ngược từ trạng thái kết thúc bằng bộ đếm bậc ra; trạng thái chu trình chưa xác định là hòa.

– **Đối xứng/phản chiếu:** Nếu P2 phản chiếu được nước P1 qua tâm/trục, P2 thắng. Nếu P1 đi ở tâm để tạo hai nửa đối xứng, P1 thắng.

– **Bất biến và chặn lẻ:** Theo dõi $I(S) \bmod 2$ (tổng, thành phần, nghịch thế). Điều khiển chặn lẻ để đối thủ gặp trạng thái kết thúc.

– **Chiến lược ghép cặp:** Chia bàn thành các cặp rời (a_i, b_i) . P1 lấy a_i thì P2 lấy ngay b_i .

– **Tìm quy luật từ trường hợp nhỏ:** Mô phỏng $N \leq 20$, tính SG $g(n)$, tìm chu kỳ $g(n) = g(n \bmod T)$ hoặc điều kiện bit.

• **Định lý Sprague-Grundy (game con độc lập):**

– $g(G) = \bigoplus g(G_i)$, với $g(u) = \text{mex}(\{g(v) \mid u \rightarrow v\})$. Thắng $\iff g(G) \neq 0$.

– **Tree Hackenbush (Colon Principle):** Node nhánh $g(u) = \bigoplus_{v \in \text{children}} (g(v) + 1)$.

• **Các biến thể Nim quen thuộc:**

– **Standard Nim:** Tổng XOR $S = \bigoplus a_i$. Thắng $\iff S \neq 0$.

– **Bounded Subtraction Nim** (lấy 1.. k): $g(a) = a \bmod (k + 1) \implies$ Thắng $\iff \bigoplus (a_i \bmod (k + 1)) \neq 0$.

– **Moore’s Nim_k** (lấy từ $\leq k$ đồng): Thua $\iff \forall$ vị trí bit $b, \sum_i \text{bit}_b(a_i) \equiv 0 \pmod{k + 1}$.

– **Staircase Nim:** Tương đương Nim thường trên bậc có chỉ số lẻ $(1, 3, 5, \dots)$.

– Misère Nim (người đi cuối thua):

- * Mọi đồng ≤ 1 : Thắng $\iff$ số đồng có một viên là chẵn ($S = 0$).
- * Có đồng > 1 : giống Nim thường (Thắng $\iff S \neq 0$).

6.26 Xác suất và kỳ vọng

Tính tuyến tính của kỳ vọng, biến chỉ báo, Coupon Collector, bất đẳng thức

- **Tính tuyến tính của kỳ vọng:** $E[X_1 + \dots + X_n] = E[X_1] + \dots + E[X_n]$ ($\forall X_i$).
- **Biến chỉ báo:** $E[X] = \sum P(A_i)$.
- **Phân phối hình học:** $E[X] = 1/p$, $\text{Var}(X) = (1-p)/p^2$.
- **Coupon Collector:** $E[T] = n \sum_{i=1}^n \frac{1}{i} = nH_n \approx n(\ln n + \gamma)$ ($\gamma \approx 0.5772$).
- **Markov:** $P(X \geq a) \leq \frac{E[X]}{a}$; **Chebyshev:** $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$.

6.27 Bignum

Độ phức tạp: $O(n)$ in (n) block

Các block cơ số 10^9 ; tính trung gian bằng 64-bit. In block cao nhất bình thường, các block còn lại đủ 9 chữ số.

```
1 // Nonnegative integer: base 10^9, least-significant block first.
2 // Zero = {0}; otherwise the last block must be nonzero.
3 // Example: 12345678900000000042 -> {42, 234567890, 1}.
4 using bignum = vector<uint32_t>; constexpr uint32_t BASE = 1000000000;
5 ostream & print(const bignum& a, ostream & out = cout) {
6     // nonempty, normalized
7     assert(!a.empty()); out << to_string(a.back());
8     for (size_t i = a.size() - 1; i > 0; --i) {
9         std::string block = to_string(a[i - 1]);
10        out << std::string(9 - block.size(), '0') << block;
11    }
12    return out;
13 }
```

7 Hình học

7.1 Primitive Geometry

Độ phức tạp: $O(1)$ mỗi thao tác

Phép toán 2D cơ bản: điểm, vector, orientation, giao đường thẳng, hình chiếu

```
1 constexpr long double EPS = 1e-12L;
2 struct Point {
3     long double x = 0, y = 0;
4     Point operator + (Point p) const { return {x + p.x, y + p.y}; }
5     Point operator - (Point p) const { return {x - p.x, y - p.y}; }
6     Point operator * (long double k) const { return {x * k, y * k}; }
7     Point operator / (long double k) const { return {x / k, y / k}; }
8 };
9 long double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
10 long double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
11 long double norm2(Point a) { return dot(a, a); }
12 Point perp(Point a) { return {-a.y, a.x}; }
13 int sign(long double x) { return (x > EPS) - (x < -EPS); }
14 struct Line { Point p, d; };
15 optional<Point> line_intersection(Line a, Line b) {
16     long double z = cross(a.d, b.d);
17     if (abs(z) <= EPS) return nullopt;
18     return a.p + a.d * (cross(b.p - a.p, b.d) / z);
19 }
20 Point projection(Point p, Line l) {
```

```
21     assert(norm2(l.d)>0); return l.p+l.d*(dot(p-l.p,l.d)/norm2(l.d));
22 }
23 bool on_segment(Point p, Point a, Point b) {
24     return sign(cross(b - a, p - a)) == 0 && dot(p - a, p - b) <= EPS;
25 }
```

7.2 Convex Hull

Độ phức tạp: $O(n \log n)$

Monotone Chain/Graham Scan: đa giác lồi nhỏ nhất chứa tập điểm

```
1 // CCW hull without duplicate/collinear vertices; exact sort, tolerant
   orientation.
2 vector<Point> convex_hull(vector<Point> p) {
3     sort(p.begin(), p.end(), [](Point a, Point b){ return tie(a.x, a.y) < tie(b.x, b.y); });
4     p.erase(unique(p.begin(), p.end(), [](Point a, Point b) {
5         return a.x == b.x && a.y == b.y;
6     }), p.end());
7     if (p.size() < 3) return p;
8     vector<Point> h;
9     for (Point v : p) {
10        while (h.size() > 1 && cross(h.back() - h[h.size() - 2], v - h.back()) <= EPS) h.
11            pop_back();
12        h.push_back(v);
13    }
14    size_t lower = h.size();
15    for (int i = (int) p.size() - 2; i >= 0; --i) {
16        Point v = p[i];
17        while (h.size() > lower && cross(h.back() - h[h.size() - 2], v - h.back()) <= EPS) h.
18            pop_back();
19        h.push_back(v);
20    }
21    h.pop_back(); return h;
22 }
```

7.3 Rotating Calipers

Độ phức tạp: $O(n)$ trên convex hull không có ba đỉnh thẳng hàng

Antipodal pairs: Cặp đỉnh nằm trên hai đường đồ song song bao quanh hull. Quét cạnh/cross product trả về cặp chỉ số; có thể dùng tìm đường kính.

```
1 // Strict CCW convex hull; emits supporting vertex pairs (duplicates
   permitted).
2 vector<pair<int, int>> antipodal_pairs(const vector<Point>& p) {
3     int n = p.size(); vector<pair<int, int>> out;
4     if (n < 2) return out;
5     if (n == 2) return {
6         {0, 1}
7     }; int j = 1;
8     for (int i = 0; i < n; ++i) {
9         int k=(i+1)%n; auto area=[&](int v){ return cross(p[k]-p[i], p[v]-p[i]); };
10        while (area((j + 1) % n) > area(j) + EPS) j = (j + 1) % n;
11        out.push_back( {i, j}); out.push_back( {k, j});
12        if (abs(area((j + 1) % n) - area(j)) <= EPS) {
13            out.push_back( {i, (j+1)%n}); out.push_back( {k, (j+1)%n});
14        }
15    }
16    return out;
17 }
18 long double diameter_squared(const vector<Point>& h) {
19     long double ans = 0;
20     for(auto [i, j]:antipodal_pairs(h)) ans=max(ans, norm2(h[i]-h[j]));
21     return ans;
22 }
```

7.4 Minkowski Sum

Độ phức tạp: $O(|P| + |Q|)$

Minkowski sum của hai đa giác lồi. Khoảng cách nhỏ nhất: $\text{dist}(P, Q) = \text{dist}(O, P - Q)$

```
1 // Strict CCW convex polygons. Degenerate hulls use the explicit fallback.
2 vector<Point> minkowski_sum(vector<Point> a, vector<Point> b) {
3     if (a.empty() || b.empty()) return {};
4     if (a.size() < 3 || b.size() < 3) {
5         vector<Point> s;
6         for (auto x : a) for (auto y : b) s.push_back(x + y);
7         return convex_hull(s);
8     }
9     auto rotate_min = [](vector<Point>& p) {
10        auto it = min_element(p.begin(), p.end(), [](Point x, Point y) {
11            return tie(x.y, x.x) < tie(y.y, y.x);
12        }); rotate(p.begin(), it, p.end());
13    }; rotate_min(a); rotate_min(b); int n=a.size(), m=b.size(), i=0, j=0;
14    Point cur = a[0] + b[0]; vector<Point> out {cur};
15    while (i < n || j < m) {
16        Point u = a[(i + 1) % n] - a[i % n], v = b[(j + 1) % m] - b[j % m];
17        int c = i == n ? -1 : j == m ? 1 : sign(cross(u, v));
18        if (c >= 0 && i < n) { cur = cur + u; ++i; }
19        if (c <= 0 && j < m) { cur = cur + v; ++j; }
20        out.push_back(cur);
21    }
22    out.pop_back(); return out;
23 }
```

7.5 Point in Convex Polygon

Độ phức tạp: $O(\log n)$

Kiểm tra điểm trong đa giác lồi bằng binary search góc cực từ P_0

```
1 // -1 outside, 0 boundary, 1 inside; strict CCW polygon.
2 int point_in_convex(const vector<Point>& p, Point q) {
3     int n = p.size();
4     if (!n) return -1;
5     if (n == 1) return norm2(q - p[0]) <= EPS * EPS ? 0 : -1;
6     if (n == 2) return on_segment(q, p[0], p[1]) ? 0 : -1;
7     int a=sign(cross(p[1]-p[0], q-p[0])), b=sign(cross(p.back()-p[0], q-p[0]));
8     if (a < 0 || b > 0) return -1;
9     if (!a) return on_segment(q, p[0], p[1]) ? 0 : -1;
10    if (!b) return on_segment(q, p[0], p.back()) ? 0 : -1;
11    int l = 1, r = n - 1;
12    while (r - l > 1) {
13        int m = (l + r) / 2;
14        if (cross(p[m] - p[0], q - p[0]) >= 0) l = m;
15        else r = m;
16    }
17    return sign(cross(p[r] - p[1], q - p[1]));
18 }
```

7.6 Line Hull Intersection

Độ phức tạp: $O(n)$ tiền xử lý, $O(\log n)$ truy vấn

Tiền xử lý pháp tuyến cạnh; binary search đỉnh cực trị và cạnh bị cắt

```
1 // 0(n) preprocessing, 0(log n) query. Strict CCW hull, n>=3.
2 struct LineHullIntersection {
3     vector<Point> p; vector<pair<long double, int>> normals;
4     int extreme(Point d) const {
5         long double a = atan2(d.y, d.x);
6         auto it=lower_bound(normals.begin(), normals.end(), a, [](auto x, long double y){
7             return x.first < y;
8         }); return it == normals.end() ? normals[0].second : it->second;
9     }
10    explicit LineHullIntersection(vector<Point> h) : p(move(h)) {
11        assert(p.size() >= 3);
12        for (int i = 0; i < (int) p.size(); ++i) {
13            Point e = p[(i + 1) % p.size()] - p[i], out = {e.y, -e.x};
14            normals.push_back( {atan2(out.y, out.x), i});
15        }
16        auto it = min_element(normals.begin(), normals.end());
```

```

17     rotate(normals.begin(), it, normals.end());
18 }
19 vector<Point> intersect(Line l) const {
20     assert(norm2(l.d) > 0); Point normal = perp(l.d);
21     int hi = extreme(normal), lo = extreme(normal * -1), n = p.size();
22     auto side = [&](int i) { return cross(l.d, p[i] - l.p); };
23     if (side(hi) < -EPS || side(lo) > EPS) return {};
24     // A supporting line may touch a vertex or coincide with one whole edge.
25     if (abs(side(hi)) <= EPS || abs(side(lo)) <= EPS) {
26         int u = abs(side(hi)) <= EPS ? hi : lo; vector<Point> hits {p[u]};
27         for(int v: {(u+n-1)%n, (u+1)%n}) if (abs(side(v)) <= EPS) hits.push_back(p[v]);
28     };
29     return hits;
30 }
31 vector<Point> out;
32 for (auto ends : {
33     pair {lo, hi}, pair {hi, lo}
34 }) {
35     int a=ends.first, b=ends.second, steps=(b-a+n)%n, L=0, R=steps;
36     long double initial = side(a);
37     if (abs(initial) <= EPS) { out.push_back(p[a]); continue; }
38     while (R - L > 1) {
39         int m = (L + R) / 2; long double v = side((a + m) % n);
40         if ((v > 0) == (initial > 0)) L = m;
41         else R = m;
42     }
43     Point x=p[(a+L)%n], y=p[(a+R)%n]; auto hit=line_intersection(l, {x, y-x});
44     if (hit) out.push_back(hit);
45     else if (abs(side((a + R) % n)) <= EPS) out.push_back(y);
46 }
47 if(out.size()==2&&norm2(out[0]-out[1])<=EPS*EPS)out.pop_back();
48 return out;
49 };

```

7.7 Polygon Raycast

Độ phức tạp: $O(n)$

Raycasting ngang kiểm tra điểm trong đa giác đơn bất kỳ

```

1 int point_in_polygon(const vector<Point>& p, Point q) {
2     bool inside = false;
3     for (int i = 0, n = p.size(); i < n; ++i) {
4         Point a = p[i], b = p[(i + 1) % n];
5         if (on_segment(q, a, b)) return 0;
6         if ((a.y > q.y) != (b.y > q.y)) {
7             long double x = a.x + (b.x - a.x) * (q.y - a.y) / (b.y - a.y);
8             if (x > q.x) inside = !inside;
9         }
10    }
11    return inside ? 1 : -1;
12 }

```

7.8 Half-plane Intersection

Độ phức tạp: $O(n \log n)$

Giao các nửa mặt phẳng trái bằng deque, cắt trong hình vuông giới hạn do người gọi chọn

```

1 // Left side of each directed line, clipped to [-bound, bound]^2.
2 // Empty or lower-dimensional intersections return empty. Choose bound
3 // explicitly.
4 vector<Point> halfplane_intersection(vector<Line> lines, long double bound) {
5     assert(bound > 0);
6     lines.insert(lines.end(), {
7         {-bound, -bound}, {1, 0}
8     }, { {bound, -bound}, {0, 1} }, { {bound, bound}, {-1, 0} }, { {-bound, bound}, {0, -1} });
9 }
10 auto angle = [](Line l) {
11     long double a=atan2(l.d.y, l.d.x); return a<0?a+2*acosl(-1.L):a;
12 }; sort(lines.begin(), lines.end(), [&](Line a, Line b) { return angle(a)<angle(b); });

```

```

13 deque<Line> q; auto outside=[](Line l, Point p){ return cross(l.d, p-l.p)<=-EPS; };
14 for (Line l : lines) {
15     assert(norm2(l.d) > 0);
16     while (q.size() > 1) {
17         auto p = line_intersection(q[q.size() - 2], q.back());
18         if (!p || outside(l, *p)) q.pop_back();
19         else break;
20     }
21     while (q.size() > 1) {
22         auto p = line_intersection(q[0], q[1]);
23         if (!p || outside(l, *p)) q.pop_front();
24         else break;
25     }
26     if (!q.empty() && abs(cross(q.back().d, l.d)) <= EPS) {
27         if (dot(q.back().d, l.d) < 0) return {};
28         if (outside(l, q.back().p)) q.pop_back();
29         else continue;
30     }
31     q.push_back(l);
32 }
33 while (q.size() > 2) {
34     auto p = line_intersection(q[q.size() - 2], q.back());
35     if (!p || outside(q.front(), *p)) q.pop_back();
36     else break;
37 }
38 while (q.size() > 2) {
39     auto p = line_intersection(q[0], q[1]);
40     if (!p || outside(q.back(), *p)) q.pop_front();
41     else break;
42 }
43 if (q.size() < 3) return {};
44 vector<Point> out;
45 for (int i = 0; i < (int) q.size(); ++i) {
46     auto p = line_intersection(q[i], q[(i + 1) % q.size()]);
47     if (!p) return {};
48     out.push_back(*p);
49 }
50 long double area = 0;
51 for(int i=0; i<(int)out.size(); ++i) area+=cross(out[i], out[(i+1)%out.size()]);
52 if (abs(area) <= EPS) return {};
53 return out;
54 }

```

7.9 Smallest Enclosing Circle

Độ phức tạp: $O(n)$ kỳ vọng

Thuật toán ngẫu nhiên Welzl tìm đường tròn bao nhỏ nhất

```

1 struct Circle { Point center; long double radius; };
2 Circle diameter_circle(Point a, Point b) {
3     Point c = (a + b) / 2; return {c, sqrtl(norm2(a - c))};
4 }
5 bool contains(Circle c, Point p) {
6     return norm2(p-c.center)<=c.radius*c.radius+EPS*max(1.L, c.radius*c.radius);
7 }
8 Circle circle_through(Point a, Point b, Point c) {
9     Point u = b - a, v = c - a; long double d = 2 * cross(u, v);
10    if (abs(d) <= EPS) {
11        Circle ans = diameter_circle(a, b);
12        for(auto x:{diameter_circle(a, c), diameter_circle(b, c)}) if(x.radius>ans.radius)ans=x;
13        return ans;
14    }
15    Point o=a+Point {v.y*norm2(u)-u.y*norm2(v), u.x*norm2(v)-v.x*norm2(u)}
16    /d; return {o, sqrtl(norm2(o - a))};
17 }
18 Circle smallest_enclosing_circle(vector<Point> p, mt19937_64 &rng) {
19     shuffle(p.begin(), p.end(), rng); Circle c { {0, 0}, -1 };
20     for(int i=0; i<(int)p.size(); ++i) if(c.radius<0 || !contains(c, p[i])) {
21         c = {p[i], 0};
22         for (int j = 0; j < i; ++j) if (!contains(c, p[j])) {
23             c = diameter_circle(p[i], p[j]);
24             for(int k=0; k<j; ++k) if(!contains(c, p[k])) c=circle_through(p[i], p[j], p[k]);

```

```

25     }
26 }
27 if (p.empty()) c.radius = 0;
28 return c;
29 }

```

7.10 Circle Tangents

Độ phức tạp: $O(1)$

Tìm tối đa bốn tiếp tuyến chung của hai đường tròn

```

1 // Tangency points on circles; nullopt means infinitely many (identical circles).
2 optional<vector<pair<Point, Point>>> circle_tangents(Point a, long double ra, Point b, long double rb) {
3     assert(ra >= 0 && rb >= 0); Point d = b - a; long double z = norm2(d);
4     if (z <= EPS * EPS) {
5         if (abs(ra - rb) <= EPS) return nullopt;
6         return vector<pair<Point, Point>> {};
7     }
8     vector<pair<Point, Point>> out;
9     for (int s : {-1, 1}) {
10        long double r = ra - s * rb, h = z - r * r;
11        if (h < -EPS) continue;
12        h = sqrtl(max(0.L, h));
13        for (int side : {-1, 1}) {
14            if (side == 1 && h <= EPS) continue;
15            Point n=(d+r*perp(d)*(h*side))/z; pair<Point, Point> t {a+n*ra, b+n*(s*rb)};
16            bool dup = false;
17            for(auto old:out) if(norm2(old.first-t.first)+norm2(old.second-t.second)<=EPS*EPS) dup=true;
18            if (!dup) out.push_back(t);
19        }
20    }
21    return out;
22 }

```

7.11 Công thức hình học

Định lý phẳng, đường tròn, tâm tam giác, phép nghịch đảo và phép toán 3D

• **Biến đổi tuyến tính** $\leftrightarrow$ **ma trận**: Mỗi ánh xạ tuyến tính $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ có $A = [T(e_1) \ \cdots \ T(e_n)]$ sao cho $T(x) = Ax$. Các dạng 2D thường dùng:

– Quay góc θ : $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$; co giãn: $\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$.

– Đối xứng qua $Ox, Oy, y = x$: $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$.

– Shear (trượt): $\begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}$ hoặc $\begin{pmatrix} 1 & 0 \\ k & 1 \end{pmatrix}$; chiếu lên vector đơn vị u :

$$P = uu^T.$$

– Tịnh tiến là affine, không tuyến tính: $x' = Ax + b$. Trong tọa độ thuần nhất, $\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$.

– Hợp thành $T_2 \circ T_1$ ứng với $A_2 A_1$; lặp một biến đổi k lần ứng với A^k .

• **Định lý Pick**: $S = I + \frac{B}{2} - 1 \implies I = S - \frac{B}{2} + 1$, với $B = \sum \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$.

• **Công thức Euler cho đồ thị phẳng**: $V - E + F = 1 + C$ ($V - E + F = 2$ với đồ thị phẳng liên thông).

• **Định lý Ptolemy**: Tứ giác nội tiếp $ABCD \iff AC \cdot BD = AB \cdot CD + BC \cdot AD$.

• **Định lý Stewart:** $b^2m + c^2n = a(d^2 + mn)$ (cevia d chia cạnh a thành m, n).

• **Công thức Bretschneider** (Diện tích tứ giác bất kỳ):

$$S = \sqrt{(p-a)(p-b)(p-c)(p-d) - abcd \cos^2\left(\frac{A+C}{2}\right)}$$

(*Tứ giác nội tiếp $\implies \cos(\frac{A+C}{2}) = 0$, trở thành công thức Brahmagupta*).

• **Định lý Descartes** (bốn đường tròn tiếp xúc từng đôi):

$$\left(\sum_{i=1}^4 k_i\right)^2 = 2 \sum_{i=1}^4 k_i^2 \quad \left(k_i = \pm \frac{1}{R_i}\right)$$

• **Nghịch đảo đường tròn (Circle Inversion):** Tâm O , bán kính R . $P \mapsto P'$ thỏa $OP \cdot OP' = R^2$. Đổi đường thẳng không qua $O \longleftrightarrow$ đường tròn qua O ; bảo toàn góc và tiếp xúc.

• **Giao hai đường tròn:** Đặt $d = |O_1 - O_2|$. Có giao khi $|R_1 - R_2| \leq d \leq R_1 + R_2$, góc $\alpha = \arccos\left(\frac{R_1^2 + d^2 - R_2^2}{2R_1d}\right)$.

• **Đường tròn Apollonius:** Quỹ tích $\frac{PA}{PB} = k$ ($k \neq 1$) là đường tròn có đường kính nối hai điểm chia trong và ngoài AB theo tỉ số k .

• **Tâm ngoại tiếp O và trực tâm H :** Đặt $d_B = x_B^2 + y_B^2 - x_A^2 - y_A^2$, $d_C = x_C^2 + y_C^2 - x_A^2 - y_A^2$:

$$x_O = \frac{d_B(y_C - y_A) - d_C(y_B - y_A)}{D}$$

$$y_O = \frac{(x_B - x_A)d_C - (x_C - x_A)d_B}{D}$$

$$D = 2[(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)]$$

$$H = A + B + C - 2O$$

• **Phép toán 3D:**

– Cross product 3D: $\vec{A} \times \vec{B} = (A_y B_z - A_z B_y, A_z B_x - A_x B_z, A_x B_y - A_y B_x)$.

– Thể tích tứ diện: $V = \frac{1}{6} |((B - A) \times (C - A)) \cdot (D - A)|$.

– Mặt phẳng qua P_0 có pháp tuyến $\vec{n}$: $\vec{n} \cdot (P - P_0) = 0$; Khoảng cách điểm đến mặt phẳng: $\frac{|\vec{n} \cdot (P - P_0)|}{|\vec{n}|}$.

7.12 AREA

Độ phức tạp: $\mathcal{O}(n \log n)$

Diện tích hợp các hình chữ nhật bằng sweep line và segment tree

```
1 struct Rectangle { long long x1, y1, x2, y2; };
2 __int128 rectangle_union_area(const vector<Rectangle>& rects) {
3     struct Event { long long x,y1,y2; int delta; }; vector<Event>e; vector<long
         long>ys;
4     for (auto r : rects) {
5         assert(r.x1 <= r.x2 && r.y1 <= r.y2);
6         if (r.x1 == r.x2 || r.y1 == r.y2) continue;
7         e.push_back( {r.x1,r.y1,r.y2,1}); e.push_back( {r.x2,r.y1,r.y2,-1});
8         ys.push_back(r.y1); ys.push_back(r.y2);
9     }
10    if (e.empty()) return 0;
11    sort(ys.begin(),ys.end()); ys.erase(unique(ys.begin(),ys.end()),ys.end());
```

```
12    sort(e.begin(),e.end(),[](auto a,auto b){ return a.x<b.x; }); int n=ys.size
    ()-1;
13    vector<int> cover(4 * n); vector<__int128> len(4 * n);
14    function<void(int,int,int,int,int,int)>update=[&](int u,int l,int r,int a,
        int b,int d){
15        if (b <= l || r <= a) return;
16        if (a <= l && r <= b) cover[u] += d;
17        else { int m=(l+r)/2; update(u*2,l,m,a,b,d); update(u*2+1,m,r,a,b,d); }
18        len[u]=cover[u]?(__int128)ys[r]-ys[l]:r-l==1?0:len[u*2]+len[u*2+1];
19    }; __int128 ans = 0; long long prev = e[0].x;
20    for (auto v : e) {
21        ans += len[l] * ((__int128) v.x - prev);
22        update(1,0,n,lower_bound(ys.begin(),ys.end(),v.y1)-ys.begin(),lower_bound
            (ys.begin(),ys.end(),v.y2)-ys.begin(),v.delta);
23        prev = v.x;
24    }
25    return ans;
26 }
```

7.13 Manhattan MST

Độ phức tạp: $\mathcal{O}(n \log n)$

Manhattan MST bằng quét tám miền góc

```
1 struct ManhattanEdge { int u, v; long long weight; };
2 // Coordinates, sums/differences, distances and total weight must fit long
   long.
3 vector<ManhattanEdge>manhattan_candidates(vector<pair<long long,long long>>p)
   {
4     auto original=p; int n=p.size(); vector<int>ids(n); iota(ids.begin(),ids.
       end(),0);
5     vector<ManhattanEdge> edges;
6     for (int direction = 0; direction < 4; ++direction) {
7         sort(ids.begin(), ids.end(), [&](int i, int j) {
8             return p[i].first + p[i].second < p[j].first + p[j].second;
9         }); map<long long, int> sweep;
10        for (int i : ids) {
11            auto[x, y] = p[i];
12            for (auto it = sweep.lower_bound(-y); it != sweep.end(); ) {
13                int j = it->second; auto[u, v] = p[j];
14                if (x - u < y - v) break;
15                edges.push_back( {
16                    i,j,llabs(original[i].first-original[j].first)+llabs(original[i].
                        second-original[j].second)
17                }); it = sweep.erase(it);
18            }
19            sweep[-y] = i;
20        }
21        for (auto& [x, y] : p) if (direction & 1) x = -x;
22        else swap(x, y);
23    }
24    return edges;
25 }
26 pair<long long,vector<ManhattanEdge>>manhattan_mst(const vector<pair<long
   long,long long>>&p){
27     auto e = manhattan_candidates(p);
28     sort(e.begin(),e.end(),[](auto a,auto b){ return a.weight<b.weight; });
29     vector<int>parent(p.size(),size(p.size(),1); iota(parent.begin(),parent.
       end(),0);
30     function<int(int)> find = [&](int u) {
31         return parent[u] == u ? u : parent[u] = find(parent[u]);
32     }; long long total = 0; vector<ManhattanEdge> tree;
33     for (auto edge : e) {
34         int a = find(edge.u), b = find(edge.v);
35         if (a == b) continue;
36         if (size[a] < size[b]) swap(a, b);
37         parent[b]=a; size[a]+=size[b]; total+=edge.weight; tree.push_back(edge);
38     }
39     return {total, tree};
40 }
```

7.14 Planar Dual Graph

Độ phức tạp: $\mathcal{O}(E \log E)$

Từ embedding phẳng liên thông với cạnh thẳng sang đồ thị đối ngẫu: tách mặt (kể cả mặt ngoài) và cạnh dual bằng sắp góc các half-edge (dart).

```
1 struct PlanarDual {
2     vector<vector<int>>>face_darts; vector<int>>face_of; vector<pair<int,int>>>
       dual_edges;
3     int outer_face = -1;
4 };
5 // Connected straight-line embedding, no crossings/overlapping rays/self-
   loops.
6 // Bridges yield loops in the dual. Isolated single vertex has one empty face
   .
7 PlanarDual planar_dual(const vector<Point>&p,const vector<pair<int,int>>&e
   dges){
8     int n = p.size(), m = edges.size(); PlanarDual out;
9     if(!m){ if(n){ out.face_darts.push_back( {}); out.outer_face=0; } return
       out; }
10    vector<vector<int>>>adj(n); vector<int>>from(2*m),to(2*m),pos(2*m),next(2*m);
11    for (int i = 0; i < m; ++i) {
12        auto[u,v]=edges[i]; assert(u!=v); from[2*i]=to[2*i+1]=u; to[2*i]=from[2*
        i+1]=v;
13        adj[u].push_back(2 * i); adj[v].push_back(2 * i + 1);
14    }
15    auto upper = [](Point a){ return a.y > 0 || (a.y == 0 && a.x >= 0); };
16    for (int u = 0; u < n; ++u) {
17        sort(adj[u].begin(), adj[u].end(), [&](int a, int b) {
18            Point x=p[to[a]]-p[u],y=p[to[b]]-p[u];if(upper(x)!=upper(y))return
                upper(x)>upper(y);return cross(x,y)>0;
19        });
20        for (int j = 0; j < (int) adj[u].size(); ++j) pos[adj[u][j]] = j;
21    }
22    for (int d = 0; d < 2 * m; ++d) {
23        int v=to[d],k=adj[v].size(); next[d]=adj[v][ (pos[d-1]+k-1)%k];
24    }
25    out.face_of.assign(2 * m, -1);
26    long double minimum = numeric_limits<long double>::infinity();
27    for (int d = 0; d < 2 * m; ++d) if (out.face_of[d] < 0) {
28        int id=out.face_darts.size(); vector<int>>face; long double area=0; int v=
        d;
29        do {
30            out.face_of[v]=id; face.push_back(v); area+=cross(p[from[v]],p[to[v]]);
                v=next[v];
31        }
32        while (v != d);
33        out.face_darts.push_back(move(face));
34        if (area < minimum) minimum = area, out.outer_face = id;
35    }
36    for(int i=0;i<m;++i)out.dual_edges.push_back( {out.face_of[2*i],out.face_of
        [2*i+1]});
37    return out;
38 }
```

8 Tiện ích và kỹ thuật

8.1 Template

```
1 #include<bits/stdc++.h>
2 bool VERBOSE = false;
3 // Use cerr (not std::cerr); false skips log expressions too.
4 #define cerr if (!VERBOSE) {} else std::cerr
5 #define FOR(i, a, b) for (int i = a; i <= b; i++)
6 #define FORD(i, a, b) for(int i = a; i >= b; i--)
7 #define endl '\n'
8 #define pb push_back
9 #define pf push_front
10 #define fastio ios_base::sync_with_stdio(NULL); cin.tie(NULL); cout.tie(NULL);
11
12 //Data structure
13 #define vi vector<int>
14 #define int long long
15 #define ii pair<int, int>
16 #define iii pair<int, ii>
17
18 //Bitmask (absolutely have a () outside of every thing)
```

```
19 #define LSB(x) (x & (-x))
20 #define ON(x, i) ((x >> i)&1)
21 #define OFF(x, i) !ON(x, i)
22 #define SET(x, i) ((x) |= (1LL << (i)))
23 #define UNSET(x, i) ((x) &= ~(1LL << (i)))
24
25 template<typename T> bool maximize(T &a, T b){ if (a < b){ a = b; return
true; } return false; }
26 template<typename T> bool minimize(T &a, T b){ if (a > b){ a = b; return
true; } return false; }
27 using namespace std;
28
29 const int MOD = 1e9 + 7;
30 void add(int &a, int b){
31     a += b;
32     if (a >= MOD) a -= MOD;
33 }
34
35 void sub(int &a, int b){
36     a -= b;
37     if (a < 0) a += MOD;
38 }
39 void solve(){
40
41 }
42 // Remember: return 0; at the end of main.
```

8.2 Debugging

```
1 #define what_is(x) cerr << #x << " = " << x << endl;
2 #define deb(...) logger(#__VA_ARGS__, __VA_ARGS__)
3 template<typename ...Args>
4 void logger(string vars, Args&&... values) {
5     cout << "{ " << vars << " } = { ";
6     string delim = "";
7     (... , (cout << delim << values << "},", delim = " , "));
8 }
```

8.3 Script biên dịch và chạy

Cần g++ trong PATH (Windows: MinGW-w64). Truyền tên file nguồn; dừng nếu biên dịch lỗi.

```
1 # Linux / macOS: save as run.sh
2 #!/bin/sh
3 [ "$#" -eq 1 ] || { echo 'Usage: sh run.sh file.cpp' >&2; exit 1; }
4 src="$1"
5 exe="${src%.*.exe}"
6 g++ -std=c++17 -O2 -Wall "$src" -o "$exe" || exit 1
7 case "$exe" in /*) ;; *) exe="./$exe" ;; esac
8 "$exe"
9 # Run: sh run.sh "my solution.cpp"
10 # Files: sh run.sh main.cpp < input.txt > output.txt
11
12 :: Windows: save as run.bat (omit the .sh part)
13 @echo off
14 if "%~1"==" " (echo Usage: run.bat file.cpp & exit /b 1)
15 g++ -std=c++17 -O2 -Wall "%~1" -o "%~dpn1.exe"
16 if errorlevel 1 exit /b 1
17 "%~dpn1.exe"
18 :: Run in cmd: run.bat "my solution.cpp"
19 :: Files: run.bat main.cpp < input.txt > output.txt
```

8.4 Phân tích biểu thức (Expression Parsing)

```
1 #include <bits/stdc++.h>
2
3 // + - * / ^, parentheses, unary signs; ^ is right-associative.
4 // Throws invalid_argument on syntax errors, domain_error on undefined
arithmetic.
5 long double evaluate_expression(const std::string& text) {
6     std::vector<long double> values;
7     std::vector<char> ops;
8     auto precedence = [](char op) {
```

```
9         if (op == '^') return 4;
10        if (op == 'n' || op == 'p') return 3;
11        if (op == '*' || op == '/') return 2;
12        if (op == '+' || op == '-') return 1;
13        return 0;
14    };
15    auto syntax = [] { throw std::invalid_argument("expression syntax"); };
16    auto apply = [&] {
17        char op = ops.back(); ops.pop_back();
18        if (values.empty()) syntax();
19        long double b = values.back(); values.pop_back();
20        if (op == 'n' || op == 'p') {
21            values.push_back(op == 'n' ? -b : b); return;
22        }
23        if (values.empty()) syntax();
24        long double a = values.back(); values.pop_back();
25        if (op == '/') && b == 0) throw std::domain_error("division by zero");
26        long double result = op == '+' ? a + b : op == '-' ? a - b :
27            op == '*' ? a * b : op == '/' ? a / b : std::pow(a, b);
28        if (!std::isfinite(result)) throw std::domain_error("nonfinite result");
29        values.push_back(result);
30    };
31    bool operand = true;
32    for (size_t i = 0; i < text.size(); ) {
33        char c = text[i];
34        if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue; }
35        if (std::isdigit(static_cast<unsigned char>(c)) || c == '.') {
36            if (!operand) syntax();
37            char* end;
38            long double value = std::strtold(text.c_str() + i, &end);
39            if (end == text.c_str() + i || !std::isfinite(value)) syntax();
40            i = static_cast<size_t>(end - text.c_str());
41            values.push_back(value); operand = false; continue;
42        }
43        ++i;
44        if (c == '(') {
45            if (!operand) syntax();
46            ops.push_back(c); continue;
47        }
48        if (c == ')') {
49            if (operand) syntax();
50            while (!ops.empty() && ops.back() != '(') apply();
51            if (ops.empty()) syntax();
52            ops.pop_back(); operand = false; continue;
53        }
54        if (c != '+' && c != '-' && c != '*' && c != '/' && c != '^') syntax();
55        if (operand) {
56            if (c != '+' && c != '-') syntax();
57            ops.push_back(c == '-' ? 'n' : 'p'); continue;
58        }
59        while (!ops.empty() && ops.back() != '(' &&
60            precedence(ops.back()) > precedence(c) ||
61            precedence(ops.back()) == precedence(c) && c != '^')) apply();
62        ops.push_back(c); operand = true;
63    }
64    if (operand) syntax();
65    while (!ops.empty()) {
66        if (ops.back() == '(') syntax();
67        apply();
68    }
69    if (values.size() != 1) syntax();
70    return values.back();
71 }
```

8.5 Lập lịch Johnson

Lập lịch trên hai máy.

```
1 #include <bits/stdc++.h>
2
3 struct FlowShopJob { int id; long long first, second; };
4 struct FlowShopSchedule { std::vector<int> order; long long makespan; };
5
6 // Nonnegative durations; unique job IDs. O(n log n).
7 FlowShopSchedule johnson_schedule(std::vector<FlowShopJob> jobs) {
8     std::sort(jobs.begin(), jobs.end(), [](const auto& a, const auto& b) {
```

```
9         bool early_a = a.first <= a.second, early_b = b.first <= b.second;
10        if (early_a != early_b) return early_a;
11        long long x = early_a ? a.first : a.second;
12        long long y = early_b ? b.first : b.second;
13        if (x != y) return early_a ? x < y : x > y;
14        return a.id < b.id;
15    });
16    FlowShopSchedule result({}, 0);
17    long long first_finish = 0;
18    for (const auto& job : jobs) {
19        result.order.push_back(job.id);
20        first_finish += job.first;
21        result.makespan = std::max(result.makespan, first_finish) + job.second;
22    }
23    return result;
24 }
```

8.6 Monte Carlo và Las Vegas

• **Monte Carlo** (Thời gian cố định, xác suất lỗi $\leq \epsilon$; lặp k lần để giảm lỗi xuống $(1 - p)^k$):

– **Freivalds** ($AB \stackrel{?}{=} C$): Kiểm tra $P = A(Bv) - Cv$ với $v \in \{0, 1\}^N$ ngẫu nhiên trong $O(N^2)$. Nếu $AB \neq C$, $\Pr(P = 0) \leq 1/2$. Lặp k lần $\implies$ lỗi $\leq 2^{-k}$.

– **Identity Testing / PIT (Schwartz-Zippel)**: Đa thức khác 0 bậc d , $P(x_1..x_n)$, tại $r \in S^n$ ngẫu nhiên có $\Pr(P(r) = 0) \leq d/|S|$. General matching: kiểm tra $\det(\text{Tutte Matrix}) \neq 0$ trong $O(V^3)$.

– **Fingerprinting / Hashing**: Ánh xạ $v \mapsto r_v \in [0, 2^{64} - 1]$. Hash tập bằng $\bigoplus r_v$, multiset bằng $\sum cnt(v)r_v \pmod{2^{64}}$, dãy bằng rolling hash 61-bit. Xác suất va chạm $\leq 2^{-64}$.

– **Color Coding** ($k \leq 8$): Tìm đồ thị con/đường đi k đỉnh bằng tô ngẫu nhiên k màu. Xác suất mục tiêu đủ màu là $k!/k^k \geq e^{-k}$. DP trên màu mất $O(2^k(V + E))$. Lặp $O(e^k \ln(1/\epsilon)) \approx 300$ lần để lỗi $< 10^{-6}$.

• **Las Vegas** (Luôn đúng, thời gian ngẫu nhiên, kỳ vọng $1/p$ lần thử):

– **Tìm và kiểm chứng witness**: Nếu nghiệm có mật độ $\geq p$, chọn ứng viên ngẫu nhiên rồi kiểm tra trong $O(T) \implies$ thời gian kỳ vọng $O(T/p)$. Ví dụ: Chọn phần tử ngẫu nhiên để tìm majority $> n/2$ trong kỳ vọng $O(n)$ ($p > 1/2 \implies \leq 2$ lần thử); chọn cặp ngẫu nhiên tìm ước chung.

– **Cấu trúc và chọn ngẫu nhiên**: Pivot QuickSelect (kỳ vọng $O(n)$), priority Treap ngẫu nhiên (chiều cao kỳ vọng $O(\log n)$), Pollard Rho (kỳ vọng $O(n^{1/4})$).

8.7 Lịch (Calendar)

```
1 #include <bits/stdc++.h>
2
3 // Valid Gregorian date, year >= 1. Returns Sunday=0, ..., Saturday=6.
4 int day_of_week(int year, int month, int day) {
5     if (month < 3) { month += 12; --year; }
6     int y = year % 100, century = year / 100;
7     int h = (day + 13 * (month + 1) / 5 + y + y / 4 +
8         century / 4 + 5 * century) % 7;
9     return (h + 6) % 7;
10 }
```

8.8 Josephus

```

1 #include <bits/stdc++.h>
2
3 // n,k >= 1; survivor is zero-indexed. O(n), O(1) space.
4 uint64_t josephus_linear(uint64_t n, uint64_t k) {
5     uint64_t result = 0;
6     for (uint64_t size = 1; size < n; ++size)
7         result = static_cast<uint64_t>((__uint128_t(result) + k) % (size + 1));
8     return result;
9 }
10
11 // Batch elimination, small k: O(k log n); O(n) fallback for k>n.
12 // Iterative stack avoids deep recursion; n <= INT64_MAX.
13 uint64_t josephus_fast(uint64_t n, uint64_t k) {
14     assert(n && k && n <= uint64_t(INT64_MAX));
15     if (k == 1) return n - 1;
16     std::vector<uint64_t> sizes;
17     uint64_t reduced = n;
18     while (reduced >= k) {
19         sizes.push_back(reduced);
20         reduced -= reduced / k;
21     }
22     uint64_t result = josephus_linear(reduced, k);
23     while (!sizes.empty()) {
24         uint64_t size = sizes.back(); sizes.pop_back();
25         if (result < size % k) result += size - size % k;
26         else { result -= size % k; result += result / (k - 1); }
27     }
28     return result;
29 }

```

8.9 Kỹ thuật interactive

```

1 #include <bits/stdc++.h>
2
3 // Protocol adapter: supply serialization and invalid-reply rules for the task.
4 template<class Reply>
5 class InteractiveSession {
6     std::istream& input;
7     std::ostream& output;
8     size_t limit, used = 0;
9 public:
10     InteractiveSession(std::istream& in, std::ostream& out, size_t budget)
11         : input(in), output(out), limit(budget) {}
12     size_t queries() const { return used; }
13     template<class Write, class Valid>
14     Reply ask(Write write, Valid valid) {
15         if (used == limit) throw std::runtime_error("query budget exhausted");
16         write(output); output << '\n' << std::flush; ++used;
17         Reply reply;
18         if (!(input >> reply) || !valid(reply))
19             throw std::runtime_error("invalid judge reply");
20         return reply;
21     }
22     template<class Write>
23     void answer(Write write) { write(output); output << '\n' << std::flush; }
24 };
25
26 // Monotone predicate: test(lo)=false, test(hi)=true; known endpoints not
27 // queried.
28 template<class Test>
29 long long first_true(long long lo, long long hi, Test test) {
30     assert(lo < hi);
31     while ((__int128_t(hi) - lo > 1) {
32         long long mid = static_cast<long long>((__int128_t(lo) + (__int128_t(hi) -
33             lo) / 2);
34         if (test(mid)) hi = mid; else lo = mid;
35     }
36     return hi;
37 }
38
39 // Adjacent values differ by 1; read(lo)<=target<=read(hi), lo<hi.
40 template<class Read>
41 long long find_unit_step_value(long long lo, long long hi, long long target,
42     Read read) {
43     while ((__int128_t(hi) - lo > 1) {
44         long long mid = static_cast<long long>((__int128_t(lo) + (__int128_t(hi) -
45             lo) / 2);

```

```

42     auto value = read(mid);
43     if (value == target) return mid;
44     if (value < target) lo = mid; else hi = mid;
45 }
46 return read(lo) == target ? lo : hi;
47 }
48
49 // Distinct values, n>=2; better(i,j) returns true iff value[i]>value[j].
50 // Returns [maximum index, second-maximum index]; balanced tournament.
51 template<class Better>
52 std::pair<int, int> top_two(int n, Better better) {
53     assert(n >= 2);
54     std::vector<std::vector<int>> defeated(n);
55     std::vector<int> round(n);
56     std::iota(round.begin(), round.end(), 0);
57     while (round.size() > 1) {
58         std::vector<int> next;
59         for (size_t i = 0; i < round.size(); i += 2) {
60             int winner = round[i];
61             if (i + 1 < round.size()) {
62                 int loser = round[i + 1];
63                 if (!better(winner, loser)) std::swap(winner, loser);
64                 defeated[winner].push_back(loser);
65             }
66             next.push_back(winner);
67         }
68         round.swap(next);
69     }
70     int best = round[0], second = defeated[best][0];
71     for (size_t i = 1; i < defeated[best].size(); ++i)
72         if (better(defeated[best][i], second)) second = defeated[best][i];
73     return {best, second};
74 }

```

- **Kiểm tra ba phía:** So sánh hai nhóm A, B cùng kích thước; khi có một mục tiêu và baseline bằng nhau đã biết, trả lời xác định mục tiêu ở A, B hoặc nhóm còn lại C .
- **Gộp truy vấn:** Hỏi $\gcd(X, \prod p_i^{e_i})$ để khôi phục số mũ các nguyên tố đã chọn; tránh tràn tích. Suy ra nhóm cuối khi biết tổng.
- **Chuẩn hóa:** Với $1 \leq x \leq 10^d$, tổng chữ số $S(x(10^d - 1)) = 9d$; đưa trạng thái về $9d$, rồi cộng mục tiêu $-9d$.
- **Top hai theo chi phí:** Xáo trộn phần tử còn lại; so với phần tử nhì trước phần tử nhất để ít truy vấn cực đại thật.
- **Đỉnh cực đại/lan truyền:** Cache giá trị đã hỏi, giữ witness cho đỉnh còn khả thi. Dò mìn: số mìn còn lại = 0 thì các ô chưa biết an toàn; bằng số ô chưa biết thì tất cả là mìn. Tiếp tục trừ các phương trình chồng nhau.

8.10 Random, Bit Tricks, Bitset

```

1 #include <bits/stdc++.h>
2
3 // Calls visit(submask), including zero, in descending order.
4 template<class Visit>
5 void for_each_submask(uint64_t mask, Visit visit) {
6     uint64_t sub = mask;
7     for (;;) {
8         visit(sub);
9         if (sub == 0) break;
10        sub = (sub - 1) & mask;
11    }
12 }
13
14 // Gosper: n<=64, k<=n. Calls visit(mask) for every k-element subset.
15 template<class Visit>
16 void for_each_k_subset(unsigned n, unsigned k, Visit visit) {
17     assert(k <= n && n <= 64);
18     if (k == 0) { visit(uint64_t(0)); return; }
19     if (k == 64) { visit(UINT64_MAX); return; }
20     uint64_t mask = (uint64_t(1) << k) - 1;

```

```

21     while (n == 64 || mask < (uint64_t(1) << n)) {
22         visit(mask);
23         uint64_t low = mask & (uint64_t(0) - mask), next = mask + low;
24         if (next == 0) break;
25         mask = next | (((next ^ mask) >> 2) / low);
26     }
27 }
28
29 // Return 64 for zero; avoid undefined __builtin_ctzll(0).
30 unsigned trailing_zeros(uint64_t x) { return x ? __builtin_ctzll(x) : 64; }
31 unsigned leading_zeros(uint64_t x) { return x ? __builtin_clzll(x) : 64; }
32 unsigned popcount(uint64_t x) { return __builtin_popcountll(x); }
33 uint64_t lowest_bit(uint64_t x) { return x & (uint64_t(0) - x); }
34 // Permutation/matching DP: next position = popcount(mask), no extra step state

```

8.11 Biến đổi bài toán (Transformations)

Ghi chú gốc

- **Khi bí:** Đọc lại đề! Xét trường hợp nhỏ ($N \leq 6$) để tìm quy luật/bất biến; suy ra độ phức tạp ($N \leq 20 \rightarrow 2^N, 500 \rightarrow N^3, 5000 \rightarrow N^2, 2 \cdot 10^5 \rightarrow N \log N$).
- **Tối ưu $\rightarrow$ quyết định (Binary Search):** Nếu điều kiện $P(X)$ đơn điệu, binary search X qua kiểm tra khả thi.
- **Hoán vị toàn cục $\rightarrow$ đổi chỗ cục bộ:** Lập luận trao đổi greedy: đổi $(i, j) \rightarrow (j, i)$ cải thiện mục tiêu $\iff f(i, j) < f(j, i)$.
- **Ràng buộc $\rightarrow$ Graph / Flow / Cut:** 2-SAT, difference constraints ($x_j - x_i \leq w$), Project Selection (min-cut), Hall, Dilworth (DAG path cover = $|V| - \text{MaxMatching}$).
- **Đại số $\rightarrow$ hình học/tính lồi:** Đường bao trên/dưới, Convex Hull, Slope Trick, Minkowski sum, CHT.
- **Trung bình có trọng số $\rightarrow$ Convex Hull:** Xem λ_i là tỉ lệ trộn ($\lambda_i \geq 0, \sum \lambda_i = 1$). Khi đó $x = \sum \lambda_i x_i$ nằm giữa các đầu vào: hai điểm cho đoạn AB , ba điểm cho miền tam giác, nhiều điểm cho convex hull. Mọi điểm trong convex hull đều biểu diễn được như vậy. Trong 1D: $x \in [\min x_i, \max x_i]$. Chuẩn hóa trọng số dương: $\lambda_i = w_i / \sum_j w_j$.
- **Tỉ số tổng có trọng số $\rightarrow$ trung bình tỉ số:** Với $w_i \geq 0, b_i > 0, \frac{\sum w_i a_i}{\sum w_i b_i}$ là trung bình có trọng số của a_i/b_i , trọng số mới $w_i b_i$. Vì vậy nằm trong $[\min_i (a_i/b_i), \max_i (a_i/b_i)]$. Không trọng số: $w_i = 1$.
- **Trộn trước $\rightarrow$ Jensen:** Với f lồi (hình cái bát), tính sau khi trộn rẻ hơn: $f(\sum \lambda_i x_i) \leq \sum \lambda_i f(x_i)$. Đọc là $f(\text{trung bình}) \leq \text{trung bình của } f$.
- **Đoạn toàn cục $\rightarrow$ Divide & Conquer:** Chia $[l, r] \rightarrow [l, m]$ và $[m + 1, r]$, cộng đóng góp chéo (CDQ D&C, D&C DP).
- **Xóa offline $\rightarrow$ thêm (đảo thời gian):** Đảo thứ tự truy vấn $Q \rightarrow 1$ để đổi xóa thành thêm DSU. Game DP: đi ngược từ trạng thái kết thúc.
- **Chia miền tìm kiếm (Meet-in-Middle):** Chia $N = N/2 + N/2$, giảm $2^N \rightarrow 2^{N/2}$ bằng hash table/two pointers.
- **Đúng k nhóm $\rightarrow$ hệ số phạt Lagrange:** Tối ưu $\text{cost} + \lambda m$, m là số nhóm. Binary search λ khi m đơn điệu; cần tính lồi và xử lý hòa để khôi phục đúng k . Bài *đếm* đúng k tập dùng Binomial Inversion/PIE từ số cách thuộc ít nhất k tập.
- **Tổng toàn cục $\rightarrow$ đóng góp phần tử:** Đổi góc nhìn: cộng đóng góp mỗi phần tử/cạnh/cặp (u, v) trên mọi trạng thái hợp lệ (Fubini).
- **Chia theo ngưỡng ($\sqrt{N}$ Heavy/Light):** Nhóm $\leq B$: duyệt trực tiếp; $> B$: tối đa N/B nhóm, cache riêng. Cân bằng chi phí theo B ; khác kỹ thuật Mo.

• **Nâng cấp cuối** → **chặn trạng thái**: Bỏ nâng cấp cuối, lập lại kế hoạch còn lại; nếu chi phí tăng $\leq$ giá nâng cấp thì bỏ được. Với $L \leq N$, $k > \lceil \sqrt{N} \rceil$: $\lceil L/(k-1) \rceil - \lceil L/k \rceil \leq 1$. Giá nâng cấp ≥ 1 , mỗi điểm dừng giá 1 $\implies$ chặn tài nguyên ở $\lceil \sqrt{N} \rceil$.

• **Gom nhóm thương (Quotient Grouping)**: $\lfloor N/k \rfloor$ có $O(\sqrt{N})$ giá trị khác nhau. Với $1 \leq l \leq N$, nhóm cùng thương kết thúc ở $r = \lfloor N/\lfloor N/l \rfloor \rfloor$; tiếp theo $l = r + 1$.

• **Đếm chính xác** → **bù/PIE**: Đếm trạng thái hợp lệ qua tổng – không hợp lệ, Stars & Bars, PIE.

• **Impartial Game** → **Nim Sum**: Chuyển game trên DAG thành XOR các giá trị Sprague-Grundy ($SG(u) = \text{mex}\{SG(v)\}$).

• **Undirected Vertex Geography** → **Matching**: Quân bắt đầu tại s ; hai người luân phiên đi tới hàng xóm chưa thăm, xóa đỉnh vừa rời. Người đầu thắng iff s thuộc mọi maximum matching của G , tương đương $\nu(G) > \nu(G-s)$.

• **Tổng quát** → **trường hợp đặc biệt**: Giải giới hạn dễ trước ($A_i \in \{0,1\}$, cây, $N \leq 6$) để tìm quy luật rồi tổng quát hóa.

• **Vét cạn** → **Branch & Bound**: Cắt nhánh DFS bằng cận hợp lệ.

• **Manhattan** $\leftrightarrow$ **Chebyshev** (xoay 45°): $(x,y) \mapsto (x+y, x-y) \implies |x_1-x_2|+|y_1-y_2| = \max(|x'_1-x'_2|, |y'_1-y'_2|)$. Tách hai chiều thành hai trục 1D độc lập.

• **Phân rã max/min và trị tuyệt đối**: $\max(A,B) = \frac{A+B+|A-B|}{2}$, $\min(A,B) = \frac{A+B-|A-B|}{2}$; $|x-a| = \max(x-a, a-x)$. Với một khoảng cách, xét bốn cặp dấu $(\pm x \pm y)$. Để tối thiểu $\sum_i (|x-a_i|+|y-b_i|)$, tách hai tổng 1D rồi chọn median của a_i và b_i .

• **Đếm theo ngưỡng/lớp (Fubini)**: $\sum_{i=1}^n A_i = \sum_{v=1}^{\max A} \text{count}(A_i \geq v)$ (cộng giá trị $\longleftrightarrow$ đếm phần tử $\geq v$).

• **Bình phương** → **tổng số lẻ**: $x^2 = 1 + 3 + 5 + \dots + (2x-1) = \sum_{i=1}^x (2i-1)$.

• **Dãy chữ số 1** → **tổng cấp số nhân**: $\underbrace{11\dots 1}_{i \text{ chữ số}} = 1+10+\dots+10^{i-1} = \frac{10^i-1}{9}$.

• **Dãy tăng/giảm** → **biến không âm**: Bỏ ràng buộc thứ tự bằng chênh lệch kề. Nếu $A_1 \leq \dots \leq A_n$, đặt $D_i = A_i - A_{i-1} \geq 0$; nếu $A_1 \geq \dots \geq A_n$, đặt $D_i = A_{i-1} - A_i \geq 0$ ($i \geq 2$). Thứ tự nghiêm ngặt cần $D_i \geq 1$. Khôi phục $A_i = A_1 + \sum_{j=2}^i D_j$ (tăng), hoặc $A_i = A_1 - \sum_{j=2}^i D_j$ (giảm).

• **Range Add** → **mảng hiệu/Prefix Sum**: $A_i = \sum_{j=1}^i D_j$ ($D_i = A_i - A_{i-1}$); cộng đoạn $[l,r] \rightarrow$ hai cập nhật điểm trên D . Tổng tính $\rightarrow S_r - S_{l-1}$.

• **Đoạn/mọi cặp** → **điều kiện kề**: Sau sắp xếp, $\min_{i \neq j} |A_i - A_j| = \min_i (A_{i+1} - A_i)$; $A_j - A_i \geq (j-i)D \iff A'_{i+1} \geq A'_i$ ($A'_i = A_i - iD$). Đoạn dài ≥ 2 có majority ($> 50\%$) luôn chứa witness dài 2 (x,x) hoặc 3 (x,y,x) .

• **Tích** → **tổng**: $ab = \binom{a+b}{2} - \binom{a}{2} - \binom{b}{2}$; $\prod A_i \rightarrow \sum \log A_i$ (hoặc log xác suất).

• **Đăng thức bitwise**: $A+B = (A \oplus B) + 2(A \text{ AND } B) = (A \text{ OR } B) + (A \text{ AND } B)$; $A \oplus B = (A \text{ OR } B) - (A \text{ AND } B)$. Lật bit hai số kề: $\sum_{i=1}^n \text{popcount}((i-1) \oplus i) = 2n - \text{popcount}(n)$.

• **Kỹ thuật ngẫu nhiên (Monte Carlo / Las Vegas)**: Xem §8.6. Nghiệm mật độ p : Monte Carlo lập k lần, lỗi $\leq (1-p)^k$; Las Vegas kiểm chứng trong kỳ vọng $O(T/p)$. Fingerprinting ($\bigoplus r_v, \sum r_v$) kiểm tra tập/multiset; Color Coding tách đồ thị con; Schwartz-Zippel kiểm tra tồn tại đại số (Tutte).

• **Dirichlet/Pigeonhole (4-Sum $a_x + a_y = a_z + a_w$)**: Các giá trị nguyên dương phân biệt: Có tối đa $2 \max A$ giá trị tổng. Nếu $\binom{N}{2} > 2 \max A$, pigeonhole bảo đảm hai cặp trùng tổng. Hash tổng $\rightarrow$ cập chỉ số trong $O(N^2)$; tính phân biệt của các giá trị bảo đảm bốn chỉ số khác nhau.

• **Boyer-Moore Majority Vote**: Tìm phần tử majority ($> n/2$ lần) trong $O(n)$ thời gian, $O(1)$ bộ nhớ bằng một bộ đếm. Antichain lớn nhất theo Sperner: $\binom{n}{\lfloor n/2 \rfloor}$.

Bổ sung chọn lọc

• **Cấp số cộng** → **miền tâm (LFAC2 2011A)**: Tìm hình chữ nhật có từng hàng/cột là cấp số cộng. Đánh dấu cửa sổ 3×3 hợp lệ; hình chữ nhật tâm $h \times w$ ứng với diện tích gốc $(h+2)(w+2)$, *không phải* hw . Histogram; cạnh dài 1 hoặc 2 xử lý riêng.

• **Hợp đoạn** → **chỉ hai hàng xóm (LFAC2 2011E)**: Di chuyển đoạn $[x-l, x+l]$, đếm điểm nguyên trong hợp. Vì các đoạn *cùng độ dài*, phần chỉ x phủ là $[\max(x-l, p+l+1), \min(x+l, q-l-1)]$ với hai tâm kề $p < x < q$. Cập nhật đóng góp riêng, không dựng lại hợp.

• **Chia cây** → **đếm cạnh (LFAC2 2011J)**: Chia cây n đỉnh thành nhóm liên thông cỡ $k, k \mid n$. Khả thi iff có đúng $n/k-1$ cây con không phải gốc có size chia hết k ; cắt toàn bộ các cạnh ấy. Điều kiện chia toàn cục trở thành histogram subtree size.

• **Thực hiện thuận, lập kế hoạch ngược (LFAC2 2012A)**: Mua loại i được tặng mỗi loại $j < i$ còn hàng. Thực hiện mua tăng loại, nhưng DP giảm loại: mọi ảnh hưởng từ loại cao xuống loại thấp chỉ còn *tổng số lần mua t* . Không cần giữ vector tồn kho.

• **Nhân phần dư** → **cộng số mũ (LFAC2 2012E)**: Đếm giá trị biểu thức modulo nguyên tố p , mỗi biến dùng một lần. Viết phần dư khác 0 thành g^i : phép nhân thành cyclic convolution modulo $p-1$ trên chỉ số. Xử lý 0 riêng; modulo đếm số cách khác modulo giá trị.

• **Một lựa chọn cần nhiều con: AND-Dijkstra (LFAC2 2012H)**: Chặt đầu Hydra phí $u > 0$ sinh multiset đầu con, hoặc trả phí diệt vĩnh viễn. Không relax cha từ *một* con; chờ *tất cả* con được chốt rồi cộng chi phí. Phí dương khiến chi phí cha lớn hơn từng con, cho phép thứ tự Dijkstra dù loại đầu có cycle.

• **Bỏ thứ tự sau khi chứng minh dạng chuẩn (LFAC2 2012K)**: Bắn luồng trên vòng đuối thỏ tại đó, đẩy thỏ hai bên ra xa. Với $k < \lceil n/2 \rceil$, có tối ưu bắn các luồng không kề. Thỏ *ban đầu* ở i bị đuối iff bắn i hoặc cả hai hàng xóm; mô phỏng lịch thành điều kiện ba bit cục bộ.

• **Xóa online** → **union dual (LFAC2 2013B)**: Xóa đường lưới, input tiếp phụ thuộc đáp án trước nên không đảo thời gian. Xóa cạnh primal tương ứng ghép hai mặt dual, kể cả mặt ngoài. Hai mặt đã cùng DSU trước xóa iff cạnh là cầu. Bài cần tách liên thông đổi thành bài chỉ union.

• **Xóa hai đoạn** → **tương tác thưa (LFAC2 2013D)**: Hủy hai cuộc biểu tình để mở nhiều đường nhất. Chỉ vùng phủ 1 hoặc 2 lần có thể được mở: gain $= u_i + u_j + w_{ij}$, u_i là phủ riêng, w_{ij} là phủ riêng của cặp. Sweep chỉ tạo $O(n)$ cặp có $w_{ij} > 0$, thay vì xét n^2 cặp.

• **Ràng buộc mọi đường** → **bóc lá (LFAC2 2013F)**: Đặt nhiều camera nhất, mỗi đường trên cây gặp tối đa k camera. Exchange về phía lá; mỗi lớp bóc lá tiêu tốn tối đa hai vị trí trên một đường. Chọn $\lfloor k/2 \rfloor$ lớp, thêm một đỉnh còn lại nếu k lẻ.

• **Đến sớm chưa chắc trội (LFAC2 2013H)**: Đi qua đảo có thời gian bắt bẫy, chỉ được chờ lúc an toàn. Hai lần đến cùng đảo khác khoảng an toàn *không* thay thế nhau. Trạng thái phải là $(v, \text{khoảng an toàn})$; trong cùng khoảng mới chỉ cần giữ lần đến sớm nhất.

• **Hamilton** → **ghép chu trình (LFAC2 2014F)**: Lưới chắn có lỗ 2×2 đủ xa nhau/biên. Dựng chu trình từng mảnh, rồi ghép theo cây khung của các mảnh kề bằng thay hai cạnh đối diện. Cần cổng biên phân biệt; tính khả thi đến từ cấu trúc lỗ, không từ thuật toán Hamilton tổng quát.

• **Tổng giao cặp** → **moment độ phủ (LFAC2 2013K)**: Kỳ vọng giao hai hình chữ nhật: tại điểm phủ c lần có $\binom{c}{2}$ cặp. Vì vậy tổng giao $= \frac{1}{2} \int (c^2 - c)$; sweep chỉ cần giữ hai moment $\int c, \int c^2$, rồi chia số cặp, không liệt kê cặp hình.

• **Tiếp xăng** → **đổi trọng số cạnh (LFAC2 2014B)**: Hai đầu là trạm, bình b , được đổ đầy ở trạm trung gian. Gọi $d_S(v)$ là khoảng cách tối trạm gần nhất; thay $w(u,v)$ bằng $d_S(u) + w + d_S(v)$. Đi được iff hai đầu liên thông qua cạnh mới $\leq b$: trạng thái nhiên liệu biến thành truy vấn DSU.

• **Không hereditary** → **cố định cực trị (LFAC2 2014G)**: Đoạn dài nhất có min/max duy nhất; đoạn con có thể không hợp lệ nên sliding window sai. Cố định minimum i , chặn bởi phần tử gần nhất $\leq a_i$. Gọi M là max trong miền đó: chỉ cần thử lần xuất hiện M gần nhất bên trái/phải, mở rộng tới M khác hoặc biên.

• **Chia miền** → **giảm một nửa số mũ (LFAC2 2014H)**: Tem RGB ngắn nhất phủ chuỗi có k wildcard; chồng tại wildcard được khác màu. $L \geq n/2$: chỉ cần prefix/suffix tương thích. $L \leq n/2$: hai đầu dài L rời nhau, chọn đầu ít wildcard hơn, tối đa $\lfloor k/2 \rfloor$; liệt kê $3^{k/2}$ thay vì 3^k .

• **Sự kiện hoán vị** → **đoạn tăng trên vòng (LFAC2 2014I)**: Giờ t_i , người i bắn p_i nếu còn sống; p là hoán vị. Trên mỗi cycle, thời gian giảm là điểm reset, đoạn tăng luân phiên chết/sống; đoạn dài l góp $\lfloor l/2 \rfloor$ người sống. Đổi một giờ chỉ đổi hai biên, không cần mô phỏng lại mọi sự kiện.

• **Giao nhiều vùng** → **một witness (LFAC2 2014J)**: Trên cây có gốc, mỗi vùng liên thông có một đỉnh cao nhất. Nếu giao khác rỗng, *đỉnh sâu nhất trong các đỉnh cao nhất* thuộc mọi vùng. Chỉ thử ứng viên ấy; vùng walk $a \rightarrow b$ ngăn sách d được kiểm tra bằng $\text{dist}(a,x) + \text{dist}(x,b) \leq d$.

• **Bước nhảy** → **lattice (CRRC I)**: Nhảy $(\pm m, \pm k)$, hoán vị trực, $m^2 + k^2 = n$; đếm ô tối được trong hình chữ nhật, được đi ra ngoài. Với $d = \text{gcd}(m,k)$, tọa độ phải chia hết d ; nếu $m/d, k/d$ cùng lẻ, hai thương tọa độ phải cùng parity. Các cặp cùng n cùng loại parity, nên gộp bằng GCD; đếm bội/chẵn lẻ, không BFS theo diện tích. Không có bước khác 0 thì chỉ gốc.

• **Factorial: nhánh nhỏ và nhánh khó (CRRC K)**: Tìm $a!/(b!c!)$ $= N$, chọn $b \geq c$, đặt $d = a-b$. Nếu $d \geq c$, $N \geq \binom{2c}{c}$ chặn c ; nhánh $d < c$ phải khai thác chia hết theo prime valuations. Với $N \leq 10^9$, editorial dùng $d \leq 20$; cutoff thực nghiệm, chưa chứng minh ở đây.

8.12 Checklist trước khi nộp

- **Đã đọc giới hạn và edge case** ($N = 0, 1$, âm, mất liên thông)?

- Đã reset/xóa trạng thái toàn cục giữa các test case?
- Đúng giới hạn mảng, chỉ số từ 0 hay 1?
- Có tràn số nguyên (dùng long long/ __int128)?

- Hàm thiếu **return** (Undefined Behavior)?
- Độ chính xác số thực/so sánh epsilon (**eps = 1e-9**)?
- Có fast I/O (cin.tie(NULL), '\n' thay endl)?